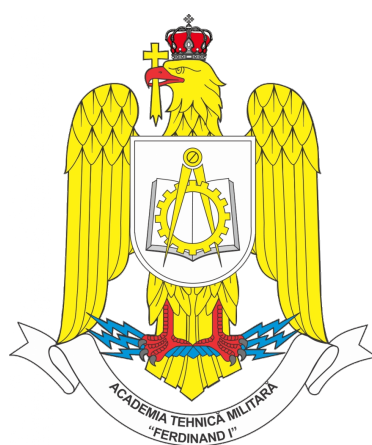


România
Ministerul Apărării Naționale
Academia Tehnică Militară "*Ferdinand I*"

Facultatea de Sisteme Informatice și Securitate Cibernetică
Calculatoare și Sisteme Informatice pentru Apărare și Securitate
Națională



Soluție inteligentă de analiză a înregistrărilor video

Coordonator Științific

Lect. dr. ing. Stelian SPÎNU

Absolvent

Sd. Sg. Maj. Estera DUMITRU

Conține _____ file
Inventariat sub numărul _____
Cu poziția din indicator _____
Cu termen de păstrare _____

București
2026

Mulțumiri pentru persoanele care au sprijinit procesul de realizare a acestei lucrări

Referatul Coordonatorului Științific

Referatul reprezintă un text în care coordonatorul științific sumarizează, ulterior finalizării conținutului efectiv, efortul pe care l-ați depus și trage concluzii cu privire la gradul de realizare a obiectivelor propuse inițial (cele prezentate în cadrul detalierii). De regulă, acest referat nu depășește cele 4 pagini alocate în acest document.

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

Tema Proiectului de Diplomă

Tema proiectului de diplomă (sau detalierea) reprezintă un text realizat de coordonatorul științific, în lunile următoare propunerii titlului proiectului de diplomă către facultate, în care detaliază nevoia reală a implementării unui astfel de proiect și modul în care se dorește a fi implementat. În plus, poate prezenta structura pe capitole a viitoarei lucrări, anexele ce vor fi incluse și sursele bibliografice din care studentul se va informa. De regulă, această detaliere nu depășește cele 4 pagini alocate în acest document.

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

NECLASIFICAT

Rezumat

Prezenta lucrare abordează problema automatizării supravegherii video prin dezvoltarea unui sistem inteligent de analiză a fluxurilor video în timp real. Motivația cercetării constă în limitările monitorizării realizate exclusiv de operatori umani, a căror capacitate de a monitoriza simultan multiple fluxuri video scade pe durata supravegherii prelungite. În acest context, este propusă și implementată o aplicație software integrată, capabilă să asiste procesul de supraveghere prin identificarea automată a unor evenimente și obiecte de interes.

Obiectivul principal al lucrării constă în proiectarea și implementarea unei arhitecturi software unificate care integrează cinci sarcini de viziune artificială asupra acelorași fluxuri video: recunoaștere facială cu autorizare pe zone și profilare demografică, recunoașterea optică a plăcuțelor de înmatriculare, detecția armelor, detecția incendiilor și a fumului, precum și recunoașterea acțiunilor umane. Contribuția principală constă în integrarea eficientă a acestor componente într-un flux unitar de procesare, completată de antrenarea a două modele dedicate pentru scenarii insuficient acoperite public.

Din punct de vedere arhitectural, sistemul este implementat ca o aplicație monolitică, organizată în jurul unui flux concurent de procesare alcătuit din șapte fire de execuție: un fir pentru captură, cinci fire dedicate modulelor de analiză și un fir responsabil de fuziunea rezultatelor. Modelele sunt încărcate o singură dată în memorie și partajează resursele GPU, iar procesarea asincronă bazată pe politica „ultimul cadru disponibil” permite menținerea unei latențe sub 100 ms per cadru. Componenta de backend este dezvoltată utilizând framework-ul FastAPI și oferă un API REST împreună cu un canal WebSocket pentru transmiterea în timp real a cadrelor procesate, în timp ce interfața utilizator este implementată în React și include autentificare bazată pe JWT și control al accesului pe roluri (RBAC).

Sistemul integrează modele consacrate din domeniul viziunii artificiale, precum InsightFace, EasyOCR, detectoare din familia YOLO și modelul SlowFast. În plus, au fost dezvoltate două componente proprii: un detector de arme bazat pe YOLOv8m, ajustat fin pe un set de date combinat, care obține un scor mAP@0,5 de 0,947, și un modul de recunoaștere a acțiunilor bazat pe SlowFast-R50, antrenat pe un set de date creat special pentru identificarea actelor de vandalism, atingând o acuratețe de 94,33%. Rezultatele experimentale demonstrează performanța modulelor dezvoltate și fezabilitatea arhitecturii propuse, capabilă să funcționeze în timp real pe echipamente accesibile.

Abstract

This thesis addresses the automation of video surveillance through an intelligent system for real-time video stream analysis. The motivation stems from the limitations of human operators, whose ability to monitor multiple video streams simultaneously decreases over prolonged surveillance. To address this challenge, the thesis proposes and implements an integrated software application capable of assisting the surveillance process by automatically identifying relevant events and objects of interest.

The primary objective is the design and implementation of a unified software architecture integrating five computer vision tasks on the same video streams: facial recognition with zone-based access authorization and demographic profiling, automatic license plate recognition, weapon detection, fire and smoke detection, and human action recognition. The main contribution lies in the efficient integration of these heterogeneous components into a unified processing pipeline, complemented by the training of two custom models for scenarios that are insufficiently addressed by publicly available pretrained solutions.

From an architectural perspective, the system is implemented as a monolithic application built around a concurrent processing pipeline consisting of seven execution threads: one thread for video capture, five threads dedicated to the analysis modules, and one thread responsible for result fusion. All models are loaded into memory only once and share GPU resources, while asynchronous execution combined with a "latest available frame" policy enables the system to maintain an end-to-end latency below 100 ms per frame. The backend is implemented using the FastAPI framework, providing a REST API and a WebSocket endpoint for real-time streaming of annotated frames, whereas the user interface is developed in React and incorporates JWT-based authentication together with role-based access control (RBAC).

The proposed system integrates well-established computer vision models, including InsightFace, EasyOCR, YOLO-based detectors, and the SlowFast architecture. In addition, two custom components were developed: a YOLOv8m-based weapon detector, fine-tuned on a combined dataset and achieving an mAP@0.5 score of 0.947, and a SlowFast-R50-based human action recognition module trained on a custom dataset specifically created for vandalism detection, achieving an accuracy of 94.33%. The experimental results demonstrate the feasibility of the proposed integrated architecture capable of operating in real time on affordable hardware platforms.

Cuprins

1	Introducere	1
1.1	Contextul actual al supravegherii video	1
1.2	Motivația alegerii temei	2
1.3	Problema abordată	3
1.4	Obiectivele lucrării	4
1.5	Structura lucrării	5
2	Stadiul Actual al Sistemelor Inteligente de Supraveghere Video	7
2.1	Evoluția sistemelor de supraveghere video	7
2.2	Soluții comerciale existente	8
2.3	Soluții open-source și academice	10
2.4	Stadiul cercetării în domeniile relevante	13
2.4.1	Recunoașterea facială	13
2.4.2	Detecția de obiecte	14
2.4.3	Recunoașterea acțiunilor umane	14
2.4.4	Recunoașterea optică a caracterelor (OCR) și a plăcuțelor de înmatriculare	15
2.4.5	Sinteză și diferențiere	15
3	Analiza Cerințelor și Proiectarea Sistemului	17
3.1	Analiza cerințelor	17
3.1.1	Cerințe funcționale	17
3.1.2	Cerințe non-funcționale	21
3.2	Cazuri de utilizare	22
3.3	Arhitectura generală a sistemului	27
3.3.1	Modelul client-server	28
3.3.1.1	Clientul	28
3.3.1.2	Serverul	28
3.3.1.3	Sursele video	29
3.3.1.4	Persistența	29
3.3.2	Diagrama de componente	29
3.3.3	Diagrama de implementare (deployment)	33
3.3.3.1	Nodul client (browser)	33
3.3.3.2	Nodul server cu GPU	33

3.3.3.3	Nodul bază de date	33
3.3.3.4	Camere IP	33
3.4	Proiectarea pipeline-ului multi-threaded	34
3.4.1	Cele 7 fire de execuție	36
3.4.1.1	Firul 1 – Captura video	36
3.4.1.2	Firul 2 – Recunoaștere facială și analiză demo- grafică	36
3.4.1.3	Firul 3 – Plăcuțe de înmatriculare	36
3.4.1.4	Firul 4 – Foc și fum	36
3.4.1.5	Firul 5 – Recunoaștere acțiuni umane	37
3.4.1.6	Firul 6 – Arme	37
3.4.1.7	Firul 7 – Fuziune	37
3.4.1.8	Justificarea separării	37
3.4.2	Mecanismele de sincronizare	38
3.4.2.1	Cozi sigure pentru concurență (<i>thread-safe</i>)	38
3.4.2.2	Partajarea cadrului curent	38
3.4.2.3	Mecanismul <i>ultimul cadru disponibil</i>	39
3.4.2.4	Stare partajată protejată prin lock-uri	39
3.4.3	Justificarea procesării paralele	39
3.4.3.1	Utilizarea GPU-ului	39
3.4.3.2	Măsurarea latențelor de inferență	40
3.4.3.3	Latența percepută per modul	41
3.4.3.4	Dezactivarea selectivă	41
3.4.3.5	Tolerare la eșec	41
3.4.3.6	De ce nu microservicii separate	41
3.5	Proiectarea API-ului REST	42
3.5.1	Convenții de denumire	42
3.5.2	Catalogul endpoint-urilor pe categorii	42
3.5.3	Structura răspunsurilor	46
3.5.4	Coduri de eroare	46
3.6	Proiectarea canalului WebSocket	47
3.6.1	Endpoint dedicat pentru streaming	47
3.6.2	Formatul mesajelor	47
3.6.3	Cadența de transmitere	48
3.6.4	Heartbeat și detectarea deconectării	50
3.6.5	Reconectare	50
3.7	Proiectarea bazei de date	50
3.7.1	Alegerea sistemului de baze de date	50
3.7.2	Diagrama entitate-relație	51
3.7.3	Tabelele principale	53
3.7.4	Chei primare și străine	54
3.7.5	Indexarea	54
3.8	Proiectarea sistemului de control al accesului	55
3.8.1	Modelul RBAC	55
3.8.2	Matricea de permisiuni	56
3.8.3	Hashing-ul parolelor	56

3.8.4	Gestiunea sesiunii (JWT)	57
3.9	Proiectarea logicii de alarmare	58
3.9.1	Modelul evenimentelor: detecție, jurnal și alarmă	58
3.9.2	Reguli de declanșare	59
3.9.3	Captura asociată (snapshot)	62
3.9.4	Niveluri de severitate și mapping pe tipuri de eveniment	63
3.9.5	Deduplicarea temporală	63
3.10	Proiectarea interfeței utilizator	65
3.10.1	Structura de navigare	66
3.10.2	Principii UX	66
3.11	Limitări ale proiectării	67
3.11.1	Limitări arhitecturale și de scalabilitate	67
3.11.2	Limitări de performanță și de procesare	68
3.11.3	Limitări ale modulelor de detecție	68
3.11.4	Limitări ale modelului de date	69
3.11.5	Limitări de confidențialitate și de gestiune a sesiunilor	69
4	Implementarea Modulelor de Inteligență Artificială	70
4.1	Privire de ansamblu și justificarea alegerii modelelor	70
4.2	Modulul de recunoaștere facială	74
4.2.1	Detecția fețelor și extragerea reprezentărilor vectoriale	75
4.2.1.1	Configurarea execuției pe GPU	75
4.2.1.2	Gestionarea bibliotecilor CUDA și a TensorFlow	77
4.2.1.3	Înregistrarea persoanelor	77
4.2.2	Indexarea cu FAISS FlatL2	77
4.2.2.1	Maparea identităților	78
4.2.2.2	Persistența și reconstrucția indexului	78
4.2.2.3	Siguranța accesului concurent	78
4.2.3	Strategia de matching	78
4.2.3.1	De la distanța L2 la similaritatea cosinus	79
4.2.4	Optimizarea: omiterea analizei demografice pentru fețele cunoscute	80
4.3	Analiza demografică și emoțională	81
4.3.1	Estimarea vârstei și genului	81
4.3.2	Recunoașterea emoțiilor cu Mini-Xception	82
4.3.3	Preprocesarea crop-ului facial	82
4.3.4	Inferența pe CPU	83
4.4	Modulul de recunoaștere a plăcuțelor de înmatriculare	83
4.4.1	Detecția plăcuțelor cu YOLOv8	84
4.4.2	Up-scaling-ul plăcuțelor mici	84
4.4.3	Pipeline-ul de preprocesare pentru OCR	86
4.4.4	Apelul EasyOCR	86
4.4.5	Fallback pe imaginea color	87
4.4.6	Tracking-ul plăcuțelor între cadre	87
4.4.7	Validarea formatului	87
4.4.8	Votarea temporală	88

4.5	Modulul de detecție foc și fum	90
4.5.1	Modelul YOLOv10 pre-antrenat	90
4.5.2	Pipeline-ul de filtrare în cinci niveluri	92
4.5.2.1	Praguri de încredere per clasă	93
4.5.2.2	Filtre de dimensiune	93
4.5.2.3	Verificarea cromatică HSV	94
4.5.2.4	Confirmarea temporală prin tracking IoU	94
4.5.2.5	Cooldown de alertă pe locație	95
4.5.3	Flag-urile <i>confirmed</i> și <i>alert</i>	96
4.6	Modulul de detecție a armelor (model ajustat fin propriu)	97
4.6.1	Justificarea unei singure clase <i>weapon</i>	98
4.6.2	Selectarea modelului de bază	98
4.6.3	Seturile de date utilizate	98
4.6.4	Preprocesarea și unificarea	101
4.6.5	Strategia de denumire (<i>zen_ / rf_</i>)	102
4.6.6	Parametrii de antrenare	102
4.6.7	Augmentările folosite	102
4.6.8	Rezultatele antrenării	103
4.6.9	Discuție asupra rezultatelor	106
4.7	Modulul de recunoaștere a acțiunilor umane (ajustare fină SlowFast)	106
4.7.1	Justificarea alegerii SlowFast R50	107
4.7.2	Construcția setului de date	108
4.7.2.1	Clasa <i>normal</i>	108
4.7.2.2	Clasa <i>fight</i>	108
4.7.2.3	Clasa <i>vandalism</i> – set de date propriu	108
4.7.2.3.1	Justificarea creării unui set de date propriu	108
4.7.2.3.2	Etapa 1: Scraping automat YouTube.	108
4.7.2.3.3	Etapa 2: Filtrare manuală.	109
4.7.2.3.4	Etapa 3: Editare în OpenShot.	109
4.7.2.3.5	Setul final.	109
4.7.3	Împărțirea în seturile de antrenare și validare	109
4.7.4	Pregătirea modelului pentru ajustarea fină	110
4.7.4.1	Formatul intrării (contractul SlowFast)	110
4.7.5	Parametrii de antrenare	111
4.7.6	Rezultatele	113
4.7.7	Analiza per clasă	115
4.7.7.1	Integrarea în aplicație	115
4.8	Fuziunea rezultatelor pe cadru	117
4.8.1	Dispecerizarea cadrelor și procesarea în paralel	117
4.8.2	Sincronizarea rezultatelor după <i>frame_id</i>	118
4.8.3	Compunerea cadrului adnotat	119
4.8.4	Generarea alarmelor și deduplicarea	120
4.8.5	Difuzarea către clienți prin WebSocket	120
4.9	Contribuții proprii	121

4.9.1	Contribuții de proiectare și arhitectură	121
4.9.2	Contribuții algoritmice și de procesare	124
4.9.3	Contribuții privind modelele antrenate și seturile de date	125
5	Implementarea Aplicației Web	127
5.1	Organizarea codului backend	127
5.1.1	Injectarea dependențelor și validarea cererilor	128
5.1.2	Middleware, jurnalizare și interacțiunea cu firele de execuție	128
5.2	Concretizarea proiectării: persistență, autentificare, sincroniza- rea camerelor	129
5.2.1	Stratul de persistență	129
5.2.2	Implementarea autentificării și a controlului de rol	129
5.2.3	Sincronizarea camerei dinamice cu registrul de camere	131
5.3	Arhitectura frontend-ului React	131
5.4	Paginile aplicației	132
5.4.1	Monitorizare live și controlul modulelor (Dashboard)	133
5.4.2	Administrarea entităților (operații CRUD)	133
5.4.3	Paginile operaționale: alarme, jurnale și statistici	134
6	Testarea și Evaluarea Sistemului	136
6.1	Metodologia de testare	136
6.2	Configurația hardware folosită	136
6.3	Evaluarea pipeline-ului integrat	136
6.3.1	FPS la procesare concurentă	136
6.3.2	Utilizarea memoriei GPU și RAM	136
6.3.3	Latența end-to-end	136
6.4	Scenarii de test funcțional	136
6.5	Discuție asupra false pozitivelor și false negativelor	136
6.6	Limitări observate	136
7	Concluzii și Direcții de Dezvoltare	137
7.1	Sinteza rezultatelor	137
7.2	Obiective atinse vs. obiective propuse	137
7.3	Contribuții originale	137
7.4	Limitări ale soluției actuale	137
7.5	Direcții de dezvoltare viitoare	137
	Bibliografie	138

Listă de figuri

3.1	Diagrama use-case UML a sistemului de supraveghere inteligent, cu actorii Administrator și Operator în raport cu cele 10 cerințe funcționale, grupate pe domenii (detectie AI, interacțiune cu utilizatorul, administrare).	20
3.2	Diagrama detaliată a cazurilor de utilizare (UC1–UC12), cu actorii Administrator și Operator, cazurile partajate și cele rezervate Administratorului, precum și relațiile include (autentificare) și extend (rezolvare alarmă).	24
3.3	Diagramă bloc de ansamblu a sistemului, organizată pe cele trei straturi logice (Prezentare, Aplicație, Persistență) și protocoalele de comunicare dintre componente: HTTPS/REST și WebSocket între client și backend, RTSP/HTTP/MJPEG pentru fluxurile camerelor, apeluri în proces către modelele AI de pe GPU și acces SQL la PostgreSQL prin driver-ul psycopg2 . Indexul FAISS este menținut în memoria backend-ului și reconstruit din PostgreSQL la pornire. . . .	28
3.4	Diagrama de componente UML a backend-ului. Modulul Captură furnizează cadre (prin interfața <i>Cadru</i>) către Modulul AI. Acesta din urmă este o componentă complexă, alcătuită din cele cinci detectoare independente (recunoaștere facială, plăcuțe de înmatriculare, foc/fum, arme și acțiuni umane), care expune contractul comun <i>Detector</i> către Modulul Fuziune. Rolul Fuziunii este de a difuza cadrele adnotate către componenta de Gestiune WebSocket și de a salva alarmele și jurnalele generate. În paralel, API-ul REST realizează operațiile de tip CRUD. Ambele componente depind de Nivelul de Persistență, format din baza de date PostgreSQL și indexul FAISS (reconstruit la fiecare pornire). Contractele dintre componente sunt ilustrate vizual prin interfețe de tip lollipop	32

- 3.5 Diagrama de implementare (deployment) UML a sistemului. Nodul *Client (Browser)* – în una sau mai multe instanțe ($\{1..m\}$) – servește aplicația React ca artefact static și comunică cu serverul prin HTTP și *WebSocket* pe portul 8000. Nodul *Server GPU* găzduiește procesul *app.py* (FastAPI + Uvicorn, cu cele ~ 7 fire de execuție), modelele AI încărcate în VRAM (InsightFace, cele două instanțe YOLOv8 pentru plăcuțe și arme, SlowFast-R50 și modelul de foc/fum) prin *onnxruntime-gpu*, modelul de emoție FER ținut deliberat pe CPU, indexul FAISS în memoria RAM și baza de date **PostgreSQL** (acces prin TCP/SQL pe portul 5432, indexul FAISS fiind reconstruit din ea la pornire). *Camerele IP* (CAM-01..CAM-n, $\{1..n\}$) furnizează fluxuri RTSP/MJPEG/HTTP. 34
- 3.6 Diagrama de flux a pipeline-ului multi-threaded. Cele **șapte fire de execuție** sunt reprezentate ca dreptunghiuri, iar cele unsprezece cozi *queue.Queue(maxsize=10)* ca cilindri. Fluxul cadrelor pornește de sus, de la *CaptureThread* (care, prin *_dispatch_frame*, depune câte o copie *frame.copy()* în cozile de *intrare*), coboară prin cele cinci coloane paralele de fire AI – fiecare adnotat cu latența de inferență măsurată (Tabelul 3.1) – ale căror rezultate converg, prin cozile de *ieșire*, către *MergingThread*. Acesta aplică politica *ultimul cadru disponibil* (folosind *raw_frame_queue* ca *fallback*), declanșează alarmele și difuzează cadrul adnotat prin *_broadcast_frame* în cozile per-client *client_queues*, de unde este preluat de clienții *WebSocket*. Politica de saturare diferă în funcție de direcția cozii: *omitere* la intrare (cadrul nou este abandonat dacă firul detectorului are deja un backlog) și *drop-oldest* la ieșire (se scoate cel mai vechi rezultat pentru a-l insera pe cel nou). 35
- 3.7 Diagramă de secvență UML a ciclului de viață al unei conexiuni *WebSocket*: autentificare (validarea token-ului JWT, cu închidere 1008 la eșec), handshake (*accept()*), alocarea cozii proprii în *client_queues*, bucla de transmitere la ~ 30 FPS (firul de fuziune produce prin *_broadcast_frame*, endpoint-ul consumă neblocaant și trimite mesajele *video_frame*) și deconectarea, cu eliminarea cozii din dicționar. 49
- 3.8 Diagrama entitate-relație (ERD) a schemei PostgreSQL *facial_recognition*, cu cele zece tabele implementate în *database_manager.py*. Cheile primare (*id*) sunt subliniate, iar cele cinci constrângeri **UNIQUE** sunt marcate cu asterisc. Liniile punctate evidențiază legăturile logice neimpuse prin FK: cele patru câmpuri *camera_id* către *cameras* și relația persoană–zonă, denormalizată ca atribut multi-valoare *authorized_zones TEXT[]* pe *persons* (nu ca entitate asociativă). Tabelul *users* este independent. 52

3.9	Modelul conceptual al evenimentelor și ciclul de viață al unei alarme. O <i>deteție</i> brută, volatilă, devine relevantă și este înregistrată ca <i>jurnal</i> (istoric pasiv, deduplicare 5 s); subsetul critic generează în plus o <i>alarmă</i> (stare unresolved , severitate, snapshot, deduplicare 30 s). Operatorul triază alarma (UC11) într-una din cele două stări terminale: resolved (alarmă rezolvată – incident real tratat) sau false_alarm (alarmă falsă). Jurnalul și alarma sunt canale <i>paralele</i> : o detecție relevantă este jurnalizată, iar dacă este și critică, este escaladată ca alarmă.	60
3.10	Diagrama de flux a deciziei de alarmare pentru un cadru. Pornind de la rezultatele agregate în firul de fuziune, cele șase verificări sunt evaluate <i>independent</i> și secvențial, astfel încât un singur cadru poate genera mai multe alarme. Fiecare ramură stabilește tipul, severitatea și dedup_subject , după care alarma trece printr-o poartă comună: cooldown de 30 s pe cheia (cameră , tip , subiect) (sub alarm_lock) și un <i>backstop</i> în baza de date împotriva duplicatelor recente nerezolvate, înainte de persistare și difuzare în interfață.	65
4.1	Imagine de ansamblu a fluxului de procesare al modulelor de inteligență artificială. Cadrele capturate din fluxul video sunt transmise, în paralel, către cele cinci module de inteligență artificială; sub fiecare modul este indicat modelul (sau biblioteca) pe care se bazează. Rezultatele converg în etapa de fuziune, care le recombina pe fiecare cadru, generează alarmele și jurnalele și difuzează cadrul adnotat către operatori. Pentru claritate, diagrama rămâne la nivel de ansamblu; detalierea firelor de execuție, a coșurilor și a mecanismului de fuziune se găsește în Figura 3.6 și în Secțiunea 4.8.	71
4.2	Fluxul de procesare al modulului de recunoaștere facială. Fiecare cadru este analizat de InsightFace într-o singură trecere, care produce simultan, pentru toate fețele, caseta de încadrare, embedding-ul de 512 dimensiuni și estimările demografice. Pentru fiecare față, embedding-ul este căutat în indexul FAISS, iar identitatea se stabilește pe baza unui prag dublu (distanță $L_2 < 1,0$ și încredere $> 0,4$). Fețele cunoscute (ramura verde) sunt jurnalizate ca evenimente de acces, iar atributele demografice sunt omise deliberat; fețele necunoscute (ramura portocalie) sunt caracterizate demografic, iar emoția este clasificată suplimentar de rețeaua Mini-Xception pe CPU. Indexul FAISS este reconstruit determinist din baza de date.	76
4.3	Fluxul de procesare al modulului de recunoaștere a plăcuțelor. Fiecare plăcuță detectată de YOLOv8 este decupată cu o margine de context, mărită dacă este prea mică și supusă unui lanț de patru operații de preprocesare. Doar după consensul pe minimum trei cadre se interoghează baza de date și se decide autorizarea (casetă verde) sau respingerea (casetă roșie).	85

4.4	Cascada de filtrare foc/fum. Detectiile YOLOv10 parcurg cinci etape: încredere, dimensiune, analiză HSV, agregare temporală IoU și <i>cooldown</i> . Rezultatele sunt progresiv respinse (gri), în așteptare (portocaliu), confirmate vizual (verde) sau marcate ca alertă (roșu). Indicatorii confirmed și alert separă starea validată de evenimentul declanșat.	91
4.5	Fluxul de date și antrenare a detectorului de arme. Cele patru scripturi execută secvențial: (1) descărcarea seturilor Zenodo și Roboflow; (2) unificarea lor (remapare pe o clasă, reținerea exemplelor negative și formatare YOLO); (3) antrenarea modelului YOLOv8m (best.pt); (4) evaluarea pe datele de test. Ponderile validate sunt ulterior integrate în aplicația de supraveghere.	99
4.6	Distribuția casetelor de încadrare (<i>bounding boxes</i>) pe clasele originale ale celor două seturi de date, anterior remapării la categoria unică weapon . Cromatica indică proveniența și tipologia datelor: albastru pentru cele cinci clase de arme din setul Zenodo, portocaliu pentru clasele de arme reținute din SOHAS (pistol și knife), respectiv gri pentru clasele non-armă din SOHAS (excluse la unificare și reutilizate drept exemple negative). Categoria knife , comună ambelor surse, este reprezentată distinct. Valorile afișate deasupra barelor indică volumul total de adnotări, cumulat pentru partițiile de antrenare, validare și testare.	100
4.7	Curbele de antrenare ale detectorului de arme pe parcursul celor 100 de epoci. Rândul superior: componentele funcției de cost pe partiția de antrenare (box_loss , cls_loss , dfl_loss), precizia și sensibilitatea. Rândul inferior: aceleași componente de cost pe partiția de validare, împreună cu metricile mAP@0,5 și mAP@0,5:0,95. Punctele reprezintă valorile per epocă, iar linia punctată redă varianta netezită. Scăderea bruscă a costului de antrenare în jurul epocii 90 corespunde dezactivării augmentării <i>mosaic</i> în ultimele zece epoci.	104
4.8	Curba precizie-sensibilitate (<i>precision-recall</i>) obținută de detectorul de arme pe setul de validare. Aria de sub curbă reflectă valoarea mAP@0,5 = 0,947. Precizia rămâne cvasi-constantă, aproape de nivelul maxim, până la o sensibilitate de $\approx 0,9$, demonstrând capacitatea sistemului de a identifica majoritatea țintelor cu o incidență minimă a alarmelor false.	105
4.9	Procesul modulului de recunoaștere a acțiunilor umane. Setul de date este alcătuit din trei clase: normal , fight și vandalism . Clipurile sunt împărțite la nivel de fișier în antrenare și validare, iar fiecare clip este transformat, conform contractului SlowFast, într-o pereche de intrări Slow/Fast. Modelul SlowFast R50 pre-antrenat pe Kinetics-400 primește un strat nou de clasificare cu trei ieșiri și este ajustat fin în două faze. Modelul rezultat (best_model.pth) este apoi folosit la inferență în aplicație, unde clasifică, prin aceeași pregătire a intrărilor, clipuri de două secunde.	107

- 4.10 Evoluția antrenării modelului de recunoaștere a acțiunilor pe durata a 28 de epoci. Stânga: funcția de cost de antrenare (albastru) și de validare (roșu). Dreapta: scorul macro-F1 obținut pe setul de validare. Perturbarea din jurul epocilor 4–6 corespunde deblocării parametrilor rețelei de bază (tranziția de la ponderi fixe în epocile 1–5 la optimizare globală începând cu epoca 6). Ponderile optime s-au obținut la epoca 21, mecanismul de oprire timpurie (*early stopping*) declanșându-se la epoca 28. 114
- 4.11 Matricea de confuzie normalizată a modelului de recunoaștere a acțiunilor, evaluată pe partiția de validare. Liniile indică apartenența reală la clasă (*ground truth*), în timp ce coloanele reprezintă predicțiile modelului, valorile fiind normalizate pe fiecare linie. Dominanța valorilor de pe diagonala principală confirmă acuratețea ridicată a clasificării, erorile reziduale manifestându-se cu precădere sub forma unor confuzii între clasele **normal** și **fight**. 116

NECLASIFICAT

NECLASIFICAT

Listă de Abrevieri

AI	<i>Artificial Intelligence</i>
API	<i>Application Programming Interface</i>
CNN	<i>Convolutional Neural Network</i>
CPU	<i>Central Processing Unit</i>
CRNN	<i>Convolutional Recurrent Neural Network</i>
FAISS	<i>Facebook AI Similarity Search</i>
FPS	<i>Frames Per Second</i>
GPU	<i>Graphics Processing Unit</i>
HSV	<i>Hue, Saturation, Value</i>
IoU	<i>Intersection over Union</i>
JWT	<i>JSON Web Token</i>
mAP	<i>mean Average Precision</i>
OCR	<i>Optical Character Recognition</i>
ONNX	<i>Open Neural Network Exchange</i>
RBAC	<i>Role-Based Access Control</i>
REST	<i>Representational State Transfer</i>
YOLO	<i>You Only Look Once</i>

NECLASIFICAT

NECLASIFICAT

Capitolul 1:

Introducere

1.1 Contextul actual al supravegherii video

Supravegherea video a evoluat de la un instrument folosit aproape exclusiv în domeniul militar și industrial la o componentă obișnuită a infrastructurii urbane și rezidențiale. Scăderea costurilor camerelor video și dezvoltarea tehnologiilor de comunicație au transformat aceste sisteme din soluții specializate în produse accesibile unui număr tot mai mare de utilizatori. Amploarea acestei extinderi este ilustrată de faptul că, la nivel global, numărul camerelor de supraveghere instalate a depășit un miliard, conform estimărilor centralizate în studiile de specialitate [1]. Și în România se observă o extindere a sistemelor de monitorizare, atât în spațiile publice, cât și în mediul rezidențial, în contextul respectării reglementărilor privind protecția datelor cu caracter personal [2]. Prin urmare, dezvoltarea acestor tehnologii aduce atât beneficii practice, cât și provocări de natură etică și juridică.

Pe lângă creșterea numărului de sisteme instalate, s-a schimbat și modul în care acestea sunt utilizate. Sistemele clasice de supraveghere aveau un rol în principal pasiv: imaginile erau stocate și analizate doar după producerea unui incident [3]. Evoluția algoritmilor de viziune artificială și a metodelor de învățare profundă a permis însă dezvoltarea unor sisteme capabile să proceseze automat fluxurile video în timp real. Astfel, accentul s-a mutat de la simpla înregistrare a imaginilor către identificarea rapidă a evenimentelor relevante și sprijinirea intervenției imediate.

Această schimbare răspunde unei probleme importante: volumul foarte mare de date video depășește capacitatea operatorilor umani de a le analiza eficient. În centrele de monitorizare clasice, un singur operator gestionează simultan zeci de fluxuri video. Studiile din literatura de specialitate demonstrează că monitorizarea continuă este o activitate solicitantă, iar nivelul de atenție și capacitatea de detectare a evenimentelor scad semnificativ după o perioadă relativ scurtă de activitate [4, 5]. În plus, majoritatea imaginilor surprind situații obișnuite, fără incidente, ceea ce face dificilă menținerea atenției pe termen lung. Din acest motiv, utilizarea unor sisteme automate de analiză video devine o soluție necesară.

În acest context, sistemele inteligente de supraveghere reprezintă o direcție firească de dezvoltare. Acestea pot analiza automat conținutul video și pot semnala operatorilor doar situațiile care necesită atenție. Soluția software propusă în cadrul acestei lucrări de licență se înscrie în această tendință, având ca obiectiv creșterea eficienței procesului de monitorizare prin utilizarea tehnicilor moderne de analiză video.

1.2 Motivația alegerii temei

Diferența majoră dintre volumul uriaș de date video generate și capacitatea limitată de atenție a operatorilor umani, detaliată în Secțiunea 1.1, subliniază nevoia evidentă pentru sisteme de supraveghere inteligente. Totuși, această necesitate nu justifică prin ea însăși dezvoltarea unei soluții noi, mai ales că piața oferă deja numeroase produse comerciale. Prin urmare, motivația acestei lucrări pornește tocmai de la o analiză a limitărilor pe care le au sistemele actuale, la care se adaugă un interes personal și tehnic pentru combinarea domeniului viziunii artificiale cu cel al securității fizice.

O mare parte dintre soluțiile comerciale disponibile în prezent sunt sisteme *închise* și *proprietary*. Acest model de afaceri creează o *dependență puternică față de producător* și atrage după sine costuri mari pentru achiziție și licențe. Însă cel mai mare dezavantaj constă în faptul că aceste sisteme funcționează ca o „cutie neagră”: algoritmi iau decizii în fundal, iar utilizatorul nu are acces la modul lor de funcționare și nici nu le poate ajusta parametrii [6]. O astfel de rigiditate face ca sistemele să fie greu de adaptat la situații specifice sau de integrat cu echipamente deja existente, limitând flexibilitatea de care au nevoie utilizatorii.

În contrast cu această abordare, lucrarea de față își propune să demonstreze că se poate construi un sistem de supraveghere inteligent și competitiv, bazat pe principii *accesibile, deschise și flexibile*. *Accesibilitatea* este asigurată prin utilizarea exclusivă a tehnologiilor cu sursă deschisă (open-source) consacrate, ceea ce elimină costurile legate de licențiere [7]. *Deschiderea* oferă transparență totală asupra fluxului de procesare a datelor, permițând verificarea fiecărei etape. Totodată, *flexibilitatea* este obținută printr-o arhitectură modulară, în care diferiții algoritmi de detecție funcționează independent; aceștia pot fi activați, dezactivați sau înlocuiți cu ușurință, astfel încât sistemul să se poată adapta în timp la noile cerințe ale beneficiarului.

Utilitatea practică a unei astfel de arhitecturi poate fi observată în multiple domenii. De exemplu, în **mediul militar**, o soluție independentă elimină riscul trimerii datelor sensibile către servere externe (cloud) [8]. În cazul **instituțiilor publice**, un sistem fără costuri de licențiere respectă bugetele alocate, dar și regulile stricte de transparență și auditare. În același timp, în **mediul privat**, arhitectura oferă libertatea de a personaliza funcționalitățile în funcție de nevoile specifice ale fiecărei afaceri.

Nu în ultimul rând, alegerea temei a fost motivată de dorința de a aborda o problemă reală, cu un impact concret asupra siguranței publice. De asemenea,

dezvoltarea acestui proiect a facilitat parcurgerea integrală a ciclului de creare a unei aplicații complexe de viziune artificială: de la selecția și antrenarea modelelor, până la construirea unei arhitecturi software solide și implementarea unei interfețe intuitive pentru utilizatorul final.

1.3 Problema abordată

Pornind de la contextul descris în Secțiunea 1.1 și de la motivația detaliată în Secțiunea 1.2, problema centrală a acestei lucrări se poate rezuma astfel: *cum se poate construi un sistem integrat de supraveghere, capabil să analizeze imagini în timp real și să îndeplinească simultan mai multe sarcini diferite de procesare, menținând totodată o viteză de reacție optimă și o interfață ușor de folosit pentru operatorii umani?* Această întrebare reunește mai multe provocări distincte, iar găsirea unei soluții care să le îmbine armonios reprezintă esența acestui proiect.

Primul aspect major este **diversitatea și complexitatea sarcinilor de analiză**. Un sistem de supraveghere eficient nu se poate rezuma la o singură funcție, ci trebuie să extragă simultan mai multe tipuri de informații din scenele monitorizate. În funcție de scopul lor, aceste funcționalități pot fi împărțite în următoarele categorii:

- *analiza identității*—detectarea fețelor, recunoașterea persoanelor cunoscute și alertarea în cazul prezenței unor indivizi neautorizați în zone restricționate;
- *caracterizarea demografică*—estimarea unor trăsături precum vârsta, genul și starea emoțională, utilă atât pentru realizarea de statistici, cât și pentru o mai bună înțelegere a contextului vizual;
- *identificarea vehiculelor*—localizarea și citirea plăcuțelor de înmatriculare pentru a facilita controlul accesului auto;
- *analiza comportamentului*—recunoașterea acțiunilor umane relevante pentru securitate, punând accent pe incidentele agresive, cum ar fi altercațiile sau actele de vandalism;
- *detectarea amenințărilor*—identificarea rapidă a obiectelor periculoase (de exemplu, arme de foc) și a situațiilor de urgență (incendii sau degajări de fum).

Dificultatea majoră nu constă doar în abordarea separată a acestor sarcini — deși, în absența unor soluții cu sursă deschisă pre-antrenate, pentru două dintre acestea a fost necesară antrenarea unor modele dedicate — ci, mai ales, în *integrarea lor simultană* într-un singur sistem. Acesta trebuie să proceseze aceleași fluxuri video fără ca modulele componente să concureze excesiv pentru resursele hardware disponibile.

A doua provocare majoră este **necesitatea funcționării în timp real**. Pentru ca un sistem de securitate să aibă un rol proactiv, el trebuie să analizeze

fluxurile video la o viteză cât mai apropiată de cea a înregistrării efective. Doar așa un eveniment critic poate fi semnalat exact atunci când are loc, evitându-se întârzierile care ar anula utilitatea practică a alarmei. Acest lucru impune limitări stricte privind timpul de procesare pentru fiecare cadru. Situația este complicată de faptul că modelele moderne de inteligență artificială consumă multe resurse de calcul, iar rularea lor simultană poate duce la întârzieri majore. Prin urmare, problema devine una de *arhitectură*: cum pot fi coordonate mai multe modele complexe astfel încât timpul de răspuns să rămână suficient de scurt pentru o vizualizare fluidă, folosind în același timp echipamente hardware obișnuite și accesibile.

Al treilea aspect esențial se referă la **utilitatea interfeței pentru operatorul uman**. Deoarece scopul sistemului este să sprijine munca operatorului, și nu să îl înlocuiască complet, rezultatele analizelor trebuie afișate într-un mod care să îi ghideze rapid atenția către problemele reale. Chiar dacă un algoritm identifică perfect evenimentele, dacă generează prea multe notificări repetitive sau nu oferă un context clar, el nu rezolvă problema inițială. De aceea, modul de prezentare a datelor este vital: imaginile trebuie redată și marcate în timp real, iar alarmele trebuie gestionate inteligent, grupând evenimentele similare pentru a nu aglomera ecranul, permițând astfel luarea unor decizii rapide și corecte.

În concluzie, obiectivul acestei lucrări este unul dual: pe de o parte, dezvoltarea și rafinarea unor sarcini individuale prin antrenarea unor modele proprii și integrarea unor algoritmi de optimizare; pe de altă parte, construirea unei arhitecturi software robuste. Aceasta din urmă are rolul de a reuni armonios toate modulele menționate într-un sistem unitar, rapid și ușor de utilizat. Modul în care această provocare este descompusă în obiective concrete și, ulterior, soluționată, va fi detaliat în secțiunea următoare, precum și în capitolele dedicate proiectării și implementării.

1.4 Obiectivele lucrării

Pentru a răspunde provocărilor formulate în Secțiunea 1.3, prezenta lucrare își propune atingerea unor obiective fundamentale care să acopere întregul ciclu de dezvoltare a unui sistem de supraveghere inteligentă. Obiectivul central constă în proiectarea și implementarea unei arhitecturi software unitare, capabile să integreze și să orchestreze simultan multiple sarcini complexe de viziune artificială pe aceleași fluxuri video. O cerință esențială a acestei arhitecturi este optimizarea consumului de resurse hardware, astfel încât analiza să se desfășoare în timp real, garantând o viteză de reacție compatibilă cu monitorizarea live.

Pe lângă componenta de procesare propriu-zisă a imaginilor, un alt obiectiv major vizează dezvoltarea unei logici aplicative inteligente. Aceasta presupune construirea unui mecanism de alarmare capabil să evalueze dinamic evenimentele detectate, raportându-le la reguli de securitate prestabilite. Scopul acestui demers este reducerea drastică a alertelor false sau redundante, precum și implementarea unei structuri coerente pentru gestionarea tuturor entităților monitorizate de sistem.

Totodată, lucrarea își propune să abordeze și limitările inerente ale modelelor pre-antrenate disponibile în prezent. Acolo unde soluțiile existente nu pot acoperi scenarii specifice de securitate, se urmărește colectarea și structurarea unor date de antrenament proprii, urmate de dezvoltarea și calibrarea unor modele de detecție dedicate.

Nu în ultimul rând, toate aceste capabilități tehnice urmează să fie unificate într-o interfață destinată utilizatorului final. Conceperea acestei platforme web presupune integrarea unui sistem de control al accesului bazat pe roluri, având ca scop final simplificarea interacțiunii umane cu sistemul și direcționarea eficientă a atenției operatorilor către incidentele de interes major.

1.5 Structura lucrării

Pentru a facilita parcurgerea documentului, această secțiune trece în revistă conținutul capitolelor următoare, oferind cititorului o hartă a modului în care problema formulată este abordată progresiv, de la fundamentarea teoretică până la evaluarea soluției finale.

Capitolul 2, *Stadiul actual al sistemelor inteligente de supraveghere video*, analizează soluțiile existente pe piață și în literatura de specialitate, evidențiind atât avantajele, cât și limitările acestora. Această trecere în revistă justifică necesitatea proiectului și clarifică poziția sistemului propus în raport cu tehnologiile actuale.

Capitolul 3, *Analiza cerințelor și proiectarea sistemului*, definește cerințele funcționale și nefuncționale, identifică utilizatorii și scenariile de utilizare. De asemenea, sunt descrise deciziile arhitecturale esențiale — fluxul de procesare concurentă, modelul de gestionare a evenimentelor, structura bazei de date și protocoalele de comunicație — care fundamentează implementarea ulterioară.

Capitolul 4, *Implementarea modulelor de inteligență artificială*, este dedicat realizării practice a fiecărui algoritm de analiză (recunoașterea facială și profilarea demografică, citirea plăcuțelor de înmatriculare, detectarea armelor, a incendiilor și a acțiunilor umane). Aici sunt detaliați pașii de antrenare a modelelor proprii, împreună cu algoritmi de procesare și decizie asociați, capitolul încheindu-se cu o sinteză a contribuțiilor personale (Secțiunea 4.9).

Capitolul 5, *Implementarea aplicației web*, descrie componenta cu care interacționează utilizatorul final. Sunt prezentate arhitectura de backend (dezvoltată utilizând framework-ul FastAPI), interfața grafică (realizată în React), mecanismele de autentificare și control al accesului bazat pe roluri, precum și modul în care imaginile și alertele sunt transmise operatorului în timp real.

Capitolul 6, *Testarea și evaluarea sistemului*, analizează performanțele soluției dezvoltate în raport cu obiectivele inițiale. Sunt prezentate metricile de precizie ale modelelor antrenate, timpii de răspuns măsurați pe întregul flux integrat de procesare și comportamentul general al aplicației în scenarii practice de utilizare.

Capitolul 7, *Concluzii și direcții de dezvoltare*, oferă o sinteză a rezultatelor obținute, evaluează măsura în care au fost îndeplinite obiectivele lucrării și propune posibile direcții pentru extinderea și îmbunătățirea sistemului în viitor.

Capitolul 2:

Stadiul Actual al Sistemelor Inteligente de Supraveghere Video

2.1 Evoluția sistemelor de supraveghere video

Dezvoltarea sistemelor de securitate și supraveghere video a fost marcată de o evoluție tehnologică continuă și accelerată, transformând supravegherea dintr-un simplu instrument pasiv de înregistrare, într-un ecosistem proactiv, capabil de analiză și decizie în timp real. Această tranziție istorică s-a desfășurat în mai multe etape majore, fiecare etapă apărând ca răspuns la limitările tehnologice ale generației anterioare: trecerea de la sistemele analogice la cele bazate pe IP, integrarea algoritmilor de analiză a conținutului video (VCA) și, cel mai recent, revoluția adusă de rețelele neurale și deep learning.

Inițial, sistemele de supraveghere în circuit închis (CCTV) operau exclusiv analogic, bazându-se pe transmiterea semnalului video continuu prin cabluri coaxiale către unități locale de stocare, precum DVR-urile (Digital Video Recorder). Deși și-au dovedit utilitatea zeci de ani, aceste arhitecturi prezentau limitări structurale severe în ceea ce privește scalabilitatea infrastructurii, calitatea imaginilor captate și vulnerabilitatea echipamentelor fizice la sabotaj. Paradigma a fost complet schimbată odată cu adoptarea la scară largă a camerelor IP, care au permis integrarea echipamentelor video în infrastructurile rețelilor de date existente (LAN/WAN) [9]. Această migrare a facilitat nu doar o îmbunătățire radicală a rezoluției (prin introducerea senzorilor HD și multi-megapixel), ci a oferit și capabilități de management centralizat și monitorizare la distanță. Cu toate acestea, literatura de specialitate subliniază că tranziția către mediul digital a generat noi provocări ingineresti, în special legate de limitările lățimii de bandă (bandwidth) necesare pentru transmiterea fluxurilor video de înaltă rezoluție și de riscurile de securitate cibernetică inerente rețelilor conectate la internet [9].

În ciuda imaginilor de înaltă fidelitate oferite de camerele IP, eficiența operațională a sistemelor de supraveghere rămânea fundamental dependentă de resursa umană. Monitorizarea pasivă, în care operatorii umani trebuiau să urmărească simultan numeroase ecrane, era inefficientă și predispusă la erori majore

din cauza oboselii și a pierderii concentrării. Pentru a depăși acest obstacol, industria a dezvoltat sisteme de tip Video Content Analysis (VCA), marcând tranziția de la o supraveghere pasivă la una activă, orientată spre detectarea evenimentelor [10]. VCA a dotat sistemele cu capacitatea de a alerta automat operatorii pe baza unor scenarii predefinite, precum încălcarea unui perimetru (line crossing), detectarea obiectelor abandonate, numărarea persoanelor sau identificarea comportamentelor neobișnuite (loitering) [11]. Aceste sisteme au reprezentat un progres major față de tehnologiile timpurii de detecție a mișcării (Video Motion Detection), care erau adesea declanșate fals de factori de mediu (vânt, schimbări de iluminare sau mișcarea vegetației). Totuși, algoritmi VCA clasici se bazau pe reguli stricte, programate manual, ceea ce îi făcea inflexibili și le limita performanța în medii vizuale complexe, foarte aglomerate sau dinamice.

Următorul salt tehnologic major, care a redefinit complet capabilitățile ecosistemului de supraveghere, a fost inserția inteligenței artificiale prin intermediul algoritmilor de Deep Learning. Spre deosebire de sistemele VCA tradiționale, care depindeau de „extragerea manuală a caracteristicilor” (manual feature extraction) realizată de programatori, modelele de tip Deep Learning—în special Rețelele Neurale Convoluționale (CNN)—au introdus capacitatea de a învăța autonom direct din seturi masive de date [3]. Această inovație a eliminat barierele algoritmilor bazați pe reguli, permițând sistemelor să clasifice obiectele cu o precizie excepțională, reușind să facă distincția clară, în timp real, între un om, un vehicul sau un animal.

Mai mult decât simpla clasificare și filtrare a alarmelor false, Deep Learning-ul a transformat modul în care datele video sunt utilizate. Prin analizarea unor volume mari de fluxuri video și identificarea unor tipare complexe de comportament, sistemele moderne dotate cu rețele neurale pot recunoaște automat situații cu potențial de risc și abateri de la comportamentul normal, semnalându-le operatorilor în momentul producerii lor [3]. Prin urmare, camera de supraveghere a evoluat dintr-un simplu senzor optic, într-un „nod inteligent” (edge device) capabil de procesare locală avansată, integrat într-un ecosistem informațional proactiv și automatizat.

2.2 Soluții comerciale existente

Piața sistemelor de supraveghere video este dominată în prezent de soluții comerciale robuste, dezvoltate de companii consacrate. Deși aceste platforme oferă capabilități avansate de inteligență artificială, ele funcționează ca sisteme închise (*closed-source*), impunând adesea restricții legate de hardware și costuri recurente semnificative. În continuare sunt analizate trei dintre cele mai reprezentative produse din această categorie, pentru a evidenția limitările care motivează abordarea propusă în prezenta lucrare.

1. Verkada

Verkada este o platformă modernă de securitate, nativă în cloud (SaaS - *Software as a Service*), recunoscută pentru ușurința în instalare și managementul centralizat [13].

- **Capabilități AI declarate:** Sistemul integrează algoritmi de analiză video la nivel de procesor (*edge computing*) capabili de detectarea persoanelor și a vehiculelor, recunoașterea atributelor faciale și a culorii hainelor, recunoașterea plăcuțelor de înmatriculare (LPR) și detectarea comportamentelor neobișnuite [13].
- **Model de licențiere:** Utilizarea platformei necesită plata unei licențe software anuale pentru fiecare cameră în parte (cuprinsă adesea între 200 și 1800 USD per an/dispozitiv).
- **Limitări observate:** Cea mai mare constrângere este *dependența totală de hardware-ul producătorului (vendor lock-in)*. Platforma software Verkada funcționează exclusiv cu camerele produse de ei [14]. În plus, costul total de proprietate (TCO - *Total Cost of Ownership*) crește masiv pe termen lung din cauza licențelor recurente, iar dependența de cloud poate fi un dezavantaj pentru instituțiile care impun izolarea strictă a datelor.

2. Avigilon (Motorola Solutions)

Avigilon este unul dintre liderii tradiționali ai pieței de sisteme de management video (VMS), oferind soluții hibride și *on-premise* destinate mediului *enterprise* [15].

- **Capabilități AI declarate:** Tehnologia sa de vârf, *Avigilon Appearance Search*, permite căutarea rapidă a persoanelor și vehiculelor în zeci de fluxuri video simultan [16]. Oferă, de asemenea, LPR, detecția mișcărilor neobișnuite (*Unusual Motion Detection*) și integrare profundă cu sistemele de control al accesului.
- **Model de licențiere:** Licențierea este modulară, presupunând o achiziție inițială substanțială (per server și per canal/cameră), plus costuri suplimentare pentru modulele de analiză AI avansată și actualizări de mentenanță.
- **Limitări observate:** Deși software-ul poate accepta camere de la terți (prin protocolul ONVIF), obținerea performanțelor AI maxime și utilizarea anumitor algoritmi necesită achiziționarea fie a camerelor Avigilon cu AI integrat, fie a unor echipamente de procesare dedicate (*Avigilon AI Appliances*) [15]. Sistemul funcționează ca o „cutie neagră” și presupune bugete inaccesibile utilizatorilor de rând sau organizațiilor mici.

3. BriefCam

BriefCam este o soluție software pură, faimoasă pentru inovația sa în domeniul *Video Synopsis* (rezumarea video), capabilă să suprapună evenimentele din momente diferite pe un singur fundal static pentru a condensa ore de înregistrări în câteva minute de vizionare [17].

- **Capabilități AI declarate:** Oferă analiză video avansată în timp real și post-eveniment: extragerea datelor demografice (vârstă, gen), LPR, recunoaștere facială, clasificarea precisă a obiectelor și analiza comportamentală complexă (direcție, staționare, viteză) [17].
- **Model de licențiere:** Costurile sunt foarte ridicate, sistemul fiind destinat aplicațiilor critice (guvernamentale, aeroporturi, orașe inteligente). Licențierea se face pe pachete de bază la care se adaugă costuri per canal video procesat și per utilizator concurent.
- **Limitări observate:** Pe lângă modelul opac de procesare a datelor, algoritmi săi complecși impun o barieră hardware majoră. Sistemul necesită achiziționarea unor servere *on-premise* extrem de puternice, echipate cu multiple plăci video dedicate (GPU) din gama enterprise, crescând masiv complexitatea instalării și a mentenanței [18].

Concluzii privind soluțiile comerciale

Deși produsele prezentate mai sus demonstrează nivelul ridicat de performanță la care a ajuns viziunea artificială aplicată în securitate, ele confirmă limitările identificate în Secțiunea 1.2. Dependența de hardware proprietar (precum în cazul Verkada și, parțial, Avigilon), costurile uriașe de licențiere și lipsa de transparență a modului în care rețelele neuronale iau decizii justifică pe deplin nevoia unei alternative. Soluția dezvoltată în cadrul prezentei lucrări demonstrează că funcționalitățile esențiale — precum identificarea vehiculelor, detectarea armelor și analiza comportamentului — pot fi implementate utilizând exclusiv tehnologii cu sursă deschisă (*open-source*), rezultând o arhitectură transparentă, adaptabilă și independentă de costuri de licențiere.

2.3 Soluții open-source și academice

Alternativa la sistemele comerciale o reprezintă comunitatea cu sursă deschisă (*open-source*) și cercetarea academică. Aceste inițiative elimină costurile de licențiere și dependența de hardware-ul proprietar, dar vin adesea cu limitări în ceea ce privește optimizarea pentru cazuri de utilizare foarte specifice sau lipsa unei arhitecturi software moderne, pregătite pentru utilizatori finali. Mai jos sunt analizate trei exemple relevante.

1. Frigate NVR

Frigate este unul dintre cele mai populare sisteme de tip NVR (*Network Video Recorder*) din comunitatea open-source, fiind construit nativ în jurul conceptului de inteligență artificială [19].

- **Capabilități AI declarate:** Rulează inferență AI complet locală, evitând transmiterea datelor în cloud. Este capabil să analizeze fluxuri video în timp real și este optimizat pentru a folosi acceleratoare hardware precum Google Coral Edge TPU, Intel OpenVINO sau NVIDIA TensorRT [19]. Oferă integrare nativă profundă cu platforme de automatizare precum Home Assistant.
- **Model de licențiere:** Complet gratuit și open-source (licență MIT).
- **Limitări față de soluția propusă:** Principala limitare a platformei constă în faptul că modelele AI furnizate implicit, antrenate în general pe seturi de date precum COCO, sunt concepute pentru detecția unor categorii de obiecte de interes general. Implementarea unor funcționalități specializate presupune integrarea și configurarea unor modele personalizate suplimentare. Astfel, în timp ce Frigate este orientat în principal către scenarii de monitorizare și automatizare specifice mediului *smart home*, soluția propusă în această lucrare este dezvoltată pentru a răspunde cerințelor specifice aplicațiilor de securitate și supraveghere inteligentă.

2. ZoneMinder

ZoneMinder este una dintre cele mai cunoscute platforme VMS (Video Management System) din ecosistemul open-source, fiind dezvoltată și întreținută în mod continuu de peste două decenii [20].

- **Capabilități AI declarate:** Platforma oferă suport pentru un număr mare de camere IP prin protocolul RTSP și include funcționalități avansate pentru gestionarea înregistrărilor și filtrarea evenimentelor pe baza unor zone definite de utilizator. În versiunile recente, au fost integrate și funcționalități de recunoaștere a obiectelor prin intermediul unor mecanisme externe bazate pe scripturi și *webhook*-uri (de exemplu, servere de evenimente care interoghează scripturi Python) [20].
- **Model de licențiere:** Gratuit și open-source (licență GPL).
- **Limitări față de soluția propusă:** ZoneMinder utilizează o arhitectură monolitică, implementată preponderent în C++ și PHP. Deși oferă un set extins de funcționalități, integrarea componentelor de inteligență artificială se realizează prin mecanisme suplimentare, externe fluxului principal de procesare. În multe configurații, analiza bazată pe AI este declanșată doar după detectarea unei modificări la nivelul imaginii (detecția clasică a mișcării), ceea ce poate introduce întârzieri suplimentare în procesarea evenimentelor. În contrast, sistemul propus în această lucrare, construit

pe baza tehnologiilor FastAPI și React, adoptă o arhitectură separată pentru backend și frontend și efectuează inferența direct asupra fluxului video, reducând astfel latența procesării.

3. Implementări academice bazate pe YOLO (Custom Pipelines)

O mare parte din literatura de specialitate modernă explorează utilizarea algoritmilor de detecție din familia YOLO (versiunile v5, v8, v10) specific pentru domeniul securității.

- **Capabilități AI declarate:** Cercetările academice demonstrează că modelele YOLO antrenate pe seturi de date specifice ating o precizie excelentă (mAP, F1-Score) și un FPS ridicat în detecția armelor albe sau de foc, identificând amenințările în fracțiuni de secundă direct din imagini CCTV [21, 22].
- **Model de licențiere:** Surse deschise (adesea scripturi publicate pe GitHub în scop de cercetare) [22].
- **Limitări față de soluția propusă:** Majoritatea cercetărilor academice se opresc la stadiul de „Proof of Concept” (PoC). Soluțiile prezentate în lucrări științifice sunt, de regulă, simple scripturi izolate de Python care analizează videoclipuri preînregistrate sau cadre individuale [21, 22]. Acestora le lipsește complet un ecosistem aplicativ real: nu dețin un server backend care să gestioneze mai multe conexiuni RTSP, nu oferă o bază de date pentru stocarea și alertarea evenimentelor și nici o interfață grafică interactivă. Chiar și abordările academice mai avansate, care integrează un sistem complet de tip IoT cu notificări și control hardware [23], rămân prototipuri proof-of-concept dedicate unui scenariu restrâns, fără a oferi un produs software generic end-to-end. Soluția propusă depășește acest stadiu teoretic, transformând modelele AI într-un produs software complet end-to-end.

Așa cum reiese din analiza realizată, soluțiile existente din ecosistemul open-source prezintă o serie de limitări în raport cu obiectivele urmărite în această lucrare. Unele platforme sunt concepute ca soluții generale de supraveghere video și nu sunt optimizate pentru scenarii specifice, în timp ce altele se bazează pe arhitecturi software dezvoltate pentru cerințe tehnologice anterioare sau sunt prezentate exclusiv în context experimental și de cercetare. Prin integrarea unor modele YOLO personalizate într-o stivă tehnologică modernă, bazată pe FastAPI pentru serviciile backend și React pentru interfața utilizatorului, proiectul propus urmărește să combine performanța modelelor de detecție validate în literatura de specialitate cu cerințele de scalabilitate, modularitate și utilizare practică specifice unei aplicații software complete.

2.4 Stadiul cercetării în domeniile relevante

Spre deosebire de Secțiunile 2.2 și 2.3, care au analizat *produse* integrate de supraveghere, prezenta secțiune adoptă o perspectivă orientată pe *sarcini*, trecând în revistă literatura recentă din cele patru domenii de viziune artificială pe care se fundamentează sistemul propus: recunoașterea facială, detecția de obiecte, recunoașterea acțiunilor umane și recunoașterea optică a caracterelor (OCR) aplicată plăcuțelor de înmatriculare. Scopul nu este o sinteză exhaustivă a fiecărui domeniu, ci poziționarea deciziilor tehnice luate în această lucrare (prezentate în detaliu în Capitolul 4) în raport cu direcțiile consacrate și cu rezultatele de referință din cercetarea actuală. Pentru fiecare domeniu sunt evidențiate atât abordările dominante, cât și diferența esențială față de demersul de față: faptul că, în lucrările analizate, fiecare sarcină este tratată în mod izolat, în timp ce contribuția prezentei lucrări constă în integrarea tuturor acestor sarcini într-un singur flux de procesare concurrent, în timp real.

2.4.1 Recunoașterea facială

Recunoașterea facială bazată pe învățare profundă a fost redefinită de introducerea funcțiilor de cost cu margine unghiulară, care optimizează direct separabilitatea reprezentărilor vectoriale (*embeddings*) în spațiul cosinus. Reperul domeniului îl constituie *ArcFace* [24], care introduce o margine unghiulară aditivă în funcția de pierdere, producând vectori de trăsături cu o discriminare inter-clasă superioară abordărilor anterioare bazate pe pierderi softmax sau triplet. Sinteze recente ale domeniului [25] confirmă faptul că, deși arhitecturile bazate pe *transformer* câștigă teren, rețelele convoluționale antrenate cu pierderi de tip ArcFace rămân soluția dominantă în aplicațiile practice, datorită echilibrului dintre acuratețe, robustețe la variații de postură și iluminare și cost computațional redus la inferență. Studii comparative dedicate [26] evaluează modele precum ArcFace și FaceNet în condiții degradate (rezoluție redusă, iluminare slabă, ocluzii parțiale) — exact condițiile specifice scenariilor de supraveghere —, evidențiind comportamentul robust al reprezentărilor antrenate cu margine unghiulară.

Soluția adoptată în prezenta lucrare se înscrie direct în această linie consacrată: modulul de recunoaștere facială (Secțiunea 4.2) utilizează pachetul `buffalo_s` al framework-ului InsightFace, ale cărui reprezentări vectoriale de 512 dimensiuni sunt antrenate cu funcția de cost ArcFace pe setul de date Glint360K. Spre deosebire de majoritatea lucrărilor citate, care se concentrează pe maximizarea acurateții de verificare pe seturi de referință (LFW, CASIA-WebFace), accentul prezentei lucrări nu cade pe rafinarea arhitecturii neuronale — preluată în forma sa originală —, ci pe *integrarea* acesteia într-un flux complet de identificare în timp real: indexarea vectorilor cu FAISS, o strategie de decizie pe bază de prag dublu (distanță L_2 și scor de încredere) și un mecanism de autorizare spațială per individ. Astfel, contribuția se mută de la nivelul modelului la nivelul sistemului care îl valorifică.

2.4.2 Detecția de obiecte

Detecția de obiecte în timp real este dominată de familia de modele YOLO (*You Only Look Once*), care tratează detecția ca o problemă unică de regresie, atingând un compromis optim între viteză și acuratețe. Literatura recentă oferă numeroase analize comparative ale evoluției acestei familii [27, 28], urmărind tranziția de la arhitecturile bazate pe ancore către cele *anchor-free* (începând cu YOLOv8) și, ulterior, către eliminarea completă a etapei de suprimare a non-maximelor (NMS) în YOLOv10, ceea ce reduce latența de inferență. Aceste sinteze documentează faptul că variantele intermediare (de tip *medium*) oferă cel mai bun raport între capacitatea de predicție și costul computațional pentru implementări în timp real.

În cadrul prezentei lucrări, familia YOLO este utilizată pentru trei sarcini distincte: detecția plăcuțelor de înmatriculare (YOLOv8), detecția focului și a fumului (YOLOv10) și detecția armelor, sarcină pentru care, în absența unui model public suficient de robust, a fost antrenat prin transfer de învățare un detector YOLOv8m dedicat unei singure clase (**weapon**), pornind de la integrarea a două seturi de date eterogene; modelul rezultat atinge o valoare mAP@0,5 de 0,947 (Secțiunea 4.6). Această valoare este comparabilă cu rezultatele raportate în literatura academică dedicată detecției armelor din imagini CCTV [21, 22]. Diferența esențială față de aceste lucrări, evidențiată deja în Secțiunea 2.3, este că ele se opresc la stadiul de validare a unui model izolat, în timp ce detectorul antrenat aici funcționează ca un modul printre altele într-un sistem integrat, suprapunând peste detecția brută straturi suplimentare de filtrare și decizie (de exemplu, cascada de filtrare în cinci niveluri pentru foc și fum, Secțiunea ??), tocmai pentru a reduce alarmele fals pozitive în condiții reale de operare.

2.4.3 Recunoașterea acțiunilor umane

Recunoașterea acțiunilor umane în context de securitate este tratată în literatură preponderent ca o problemă de detecție a violenței sau a comportamentului anormal. Arhitectura de referință pentru modelarea spațio-temporală este *Slow-Fast* [29], care separă explicit o ramură *Slow* (cadență redusă, pentru semantica spațială) de o ramură *Fast* (cadență ridicată, pentru mișcarea fină), obținând rezultate de top pe seturi de referință precum Kinetics. Pentru evaluarea metodelor de detecție a violenței în scene reale, setul de date *RWF-2000* [30] a devenit un reper, fiind alcătuit din 2000 de clipuri provenite de la camere de supraveghere; sinteze sistematice recente ale domeniului [31] confirmă rolul central al acestui set și tendința migrării de la descriptorii manuali către arhitecturi profunde spațio-temporale.

Modulul de recunoaștere a acțiunilor din prezenta lucrare (Secțiunea 4.7) adoptă această linie consacrată, ajustând fin prin transfer de învățare un model SlowFast-R50 pre-antrenat pe Kinetics-400, pentru trei clase țintă (**normal**, **fight**, **vandalism**), și atingând o acuratețe de 94,33%. În raport cu literatura, această componentă aduce două diferențe notabile. În primul rând, clasele **normal** și **fight** provin din setul RWF-2000 consacrat, asigurând comparabi-

litatea cu rezultatele publicate. În al doilea rând, întrucât clasa **vandalism** este insuficient reprezentată în resursele publice — spre deosebire de violența interpersonală, intens studiată —, pentru aceasta a fost constituit un set de date propriu de 614 secvențe video validate manual. Astfel, demersul nu se limitează la reproducerea unei arhitecturi existente pe un set standard, ci extinde domeniul de aplicabilitate către o categorie de incidente relevantă pentru supraveghere, dar slab acoperită de literatura actuală.

2.4.4 Recunoașterea optică a caracterelor (OCR) și a plăcuțelor de înmatriculare

Recunoașterea automată a plăcuțelor de înmatriculare (ALPR) este abordată în literatura recentă printr-o arhitectură în două etape consacrată: un detector de obiecte localizează plăcuța în cadru, după care un motor OCR extrage textul [32, 33]. Pentru etapa de detecție, variantele YOLO domină datorită vitezei lor, iar pentru etapa de citire sunt frecvent utilizate motoare OCR de uz general precum EasyOCR (bazat pe arhitecturile CRAFT pentru detecția textului și CRNN pentru recunoaștere) [32]. O temă recurentă în aceste lucrări o constituie dificultatea recunoașterii în condiții dificile — rezoluție redusă, neclaritate de mișcare (*motion blur*), iluminare neuniformă —, care impune etape suplimentare de preprocesare a imaginii.

Modulul de recunoaștere a plăcuțelor implementat în prezenta lucrare (Secțiunea 4.4) adoptă aceeași arhitectură în două etape (YOLOv8 pentru detecție, EasyOCR pentru citire), validată de literatură. Contribuția specifică nu se află la nivelul modelelor preluate, ci în lanțul de prelucrare construit în jurul lor pentru a face față exact condițiilor dificile semnalate de cercetare: întrucât plăcuțele apar tipic la lățimi de doar 116–151 px, fluxul include o etapă explicită de mărire a rezoluției prin interpolare bicubică, urmată de filtrare bilaterală, egalizare adaptivă a histogramei (CLAHE) și accentuarea conturilor, precum și o procedură de rezervă (*fallback*) pe imaginea color. În plus, în loc să accepte rezultatul OCR dintr-un singur cadru, decizia finală este obținută printr-un mecanism de urmărire spațială (IoU) și votare temporală ponderată cu scorul de încredere, validată suplimentar printr-o expresie regulată conformă standardului românesc. Această stabilizare temporală, mai puțin întâlnită în lucrările care evaluează ALPR pe imagini statice, este esențială pentru funcționarea fiabilă pe un flux video continuu.

2.4.5 Sinteză și diferențiere

Analiza de mai sus evidențiază o caracteristică comună a literaturii din cele patru domenii: fiecare sarcină de viziune artificială este, de regulă, studiată, optimizată și evaluată *în izolare*, pe seturi de date și în condiții experimentale dedicate. Modelele de referință adoptate în prezenta lucrare — ArcFace pentru recunoașterea facială, familia YOLO pentru detecția de obiecte, SlowFast pentru recunoașterea acțiunilor și arhitectura YOLO+OCR pentru plăcuțe —

reprezintă alegeri validate de cercetarea actuală, ceea ce asigură o fundamente solidă a soluției. Contribuția specifică a acestei lucrări nu constă, așadar, în propunerea unei arhitecturi neuronale noi pentru vreuna dintre aceste sarcini, ci în două direcții complementare: pe de o parte, adaptarea și extinderea acestor modele acolo unde literatura prezintă lacune (antrenarea unui detector de arme dedicat și constituirea unui set de date propriu pentru vandalism); pe de altă parte, și cel mai important, *integrarea simultană* a tuturor acestor sarcini, tratate separat în literatură, într-un singur flux de procesare concurent, capabil să funcționeze în timp real pe echipamente accesibile. Modul concret în care această integrare este proiectată și implementată constituie obiectul Capitolelor 3 și 4.

Capitolul 3:

Analiza Cerințelor și Proiectarea Sistemului

3.1 Analiza cerințelor

Înainte de prezentarea arhitecturii și a detaliilor de implementare, este important să fie stabilit în mod clar atât ceea ce trebuie să realizeze sistemul, cât și modul în care acesta trebuie să își îndeplinească funcțiile. În acest sens, secțiunea de față definește cerințele funcționale, care descriu capabilitățile oferite utilizatorului, precum și cerințele nefuncționale, care stabilesc constrângerile și criteriile de calitate ale soluției. Aceste cerințe vor constitui baza pe care se vor fundamenta deciziile de proiectare prezentate în capitolele următoare.

Distincția dintre cele două categorii este esențială: cerințele funcționale descriu comportamentul observabil al sistemului în timp ce cerințele nefuncționale fixează pragurile de performanță, securitate și uzabilitate sub care soluția nu mai este considerată viabilă pentru scenariul de supraveghere vizat. Ambele au fost derivate plecând de la analiza stadiului actual (Capitolul 2) și de la limitările identificate în soluțiile comerciale existente.

3.1.1 Cerințe funcționale

Cerințele funcționale sunt prezentate sub formă de listă numerotată, fiecare identificator **F_x** fiind referit ulterior în capitolele de proiectare și implementare. Lista nu detaliază toate operațiile CRUD, ci organizează funcționalitățile pe domenii coerente.

F1 – Streaming live multi-cameră. Sistemul trebuie să accepte simultan mai multe surse video (incluzând camera locală a stației, de tip webcam USB prin `cv2.VideoCapture`, precum și camere IP adăugate dinamic prin URL de flux RTSP/HTTP/MJPEG) pentru a livra fluxurile procesate către clienții web în timp real, folosind canalul WebSocket dedicat. Fiecare cameră are un identificator logic unic (**CAM-01**, **CAM-02**, ...) și poate fi pornită, oprită sau adăugată/eliminată fără a opri restul sistemului.

- F2 – Recunoaștere facială cu autorizare pe zone.** Pentru fiecare cadru, sistemul detectează fețele prezente, calculează reprezentări vectoriale faciale și le compară cu indexul FAISS construit din baza de persoane înregistrate. Pe lângă identificarea propriu-zisă, sistemul verifică dacă persoana are dreptul de acces în zona asociată camerei curente, marcând vizual indivizii neautorizați. Pentru a îmbunătăți robustețea recunoașterii, o persoană poate avea înregistrate mai multe imagini faciale. În plus, ștergerea unui utilizator declanșează reconstrucția indexului FAISS, iar modificarea metadatelor (nume, departament, zone autorizate) duce la reîmprospătarea cache-urilor de autorizare.
- F2bis – Estimarea atributelor demografice.** Pe lângă identificare, sistemul estimează attribute demografice ale fețelor detectate (vârstă, gen și emoție) folosite atât pentru statisticile agregate (F10), cât și pentru îmbogățirea metadatelor jurnalizate per detecție.
- F3 – Detecție și recunoaștere a plăcuțelor de înmatriculare.** Sistemul localizează plăcuțele de înmatriculare în cadrele video și extrage textul acestora prin OCR, cu stabilizarea citirii prin vot pe mai multe cadre. O plăcuță este considerată autorizată dacă numărul stabilizat este regăsit în baza de date de plăcuțe înregistrate.
- F4 – Detecție foc/fum cu confirmare temporală.** Sistemul detectează prezența focului și a fumului în cadre. Pentru a reduce numărul de fals pozitive provocate de surse legitime (lumini, reflexii, obiecte de culoare apropiată), o alarmă de incendiu se generează doar după confirmarea pe o fereastră temporală configurabilă, nu pe baza unui singur cadru.
- F5 – Detecție arme.** Sistemul identifică obiectele de tip armă și marchează cadrele și camerele afectate.
- F6 – Clasificare acțiuni umane.** Sistemul clasifică acțiunile observate în cadrele video în categorii relevante pentru supraveghere (activitate normală, luptă sau vandalism).
- F7 – Alarmare.** La declanșarea unei detecții critice (persoană necunoscută, persoană necunoscută sau neautorizată într-o zonă restricționată, foc/fum confirmat, armă, acțiune periculoasă, plăcuță de înmatriculare neautorizată), sistemul generează o alarmă persistentă în baza de date, afișată în interfața utilizator cu severitatea și contextul corespunzător. Pentru a preveni inundarea operatorului cu alarme identice, este aplicată o *deduplicare temporală* pe perechea (cameră, tip), cu un cooldown fix de 30 de secunde. Alarmerile au un ciclu de viață propriu: pot fi vizualizate individual, marcate ca rezolvate (păstrând evidența utilizatorului care a făcut rezolvarea) și actualizate în masă (*bulk-update*).
- F7bis – Configurarea la runtime a detectoarelor.** Fiecare modul AI (recunoaștere facială, demografie, plăcuțe, foc, arme, acțiuni umane) poate

fi activat sau dezactivat individual fără repornirea backend-ului, prin endpoint-uri dedicate de tip `toggle`. Acest lucru permite operatorului să ajusteze sarcina GPU în funcție de scenariu (ex. dezactivarea citire plăcuțe pe camere de interior).

- F8 – Gestiunea entităților (CRUD).** Sistemul asigură operații complete (creare, citire, actualizare și ștergere) pentru entitățile principale: persoane (organizate pe departamente, cu fotografii multiple și reprezentări vectoriale faciale), zone (care includ indicatorul `is_restricted`), camere (gestionate prin descoperire dinamică sau prin salvare persistentă în baza de date), plăcuțe de înmatriculare și utilizatori (cu roluri specifice). Actualizările aplicate persoanelor sau plăcuțelor se reflectă imediat și consistent în indexul FAISS, dar și în listele de autorizare. În plus, sistemul pune la dispoziție endpoint-uri dedicate pentru interogarea *istoricului accesărilor* (asociat unei persoane sau plăcuțe), precum și pentru schimbarea parolei contului curent.
- F9 – Jurnalizare și export.** Detectiile și alarmele sunt stocate în jurnale interogabile, prin intermediul tabelelor `detection_logs` și `alarms`. Aceste jurnale pot fi filtrate după interval de timp, cameră, tip de eveniment, status sau prin căutare după subiect. Pentru accesările persoanelor și ale vehiculelor există jurnale dedicate, `access_logs` și `vehicle_access_logs`. Jurnalurile de detecție pot fi exportate în format **CSV** pentru analiză externă, iar accesul vehiculelor este disponibil și prin endpoint-ul dedicat `/api/logs/vehicle`.
- F10 – Statistici și panou de activitate.** Sistemul agregă datele jurnalizate și pune la dispoziție rapoarte și grafice privind frecvența detecțiilor (`/api/logs/stats`, `/api/logs/timeseries`, `/api/logs/breakdown`), distribuția alarmelor pe camere și zone, demografia persoanelor identificate și alte metrice utile operatorilor. Pe pagina principală există un panou de *activitate recentă* actualizat în timp real prin WebSocket, care servește ca dashboard operațional.

Figura 3.1 sintetizează cele 10 cerințe funcționale sub forma unei diagrame use-case UML, plasând cei doi actori (Administrator, Operator) în raport cu funcționalitățile sistemului, grupate vizual pe domenii: detecție AI (F2–F6), interacțiune cu utilizatorul (F1, F7, F10) și administrare (F8, F9).

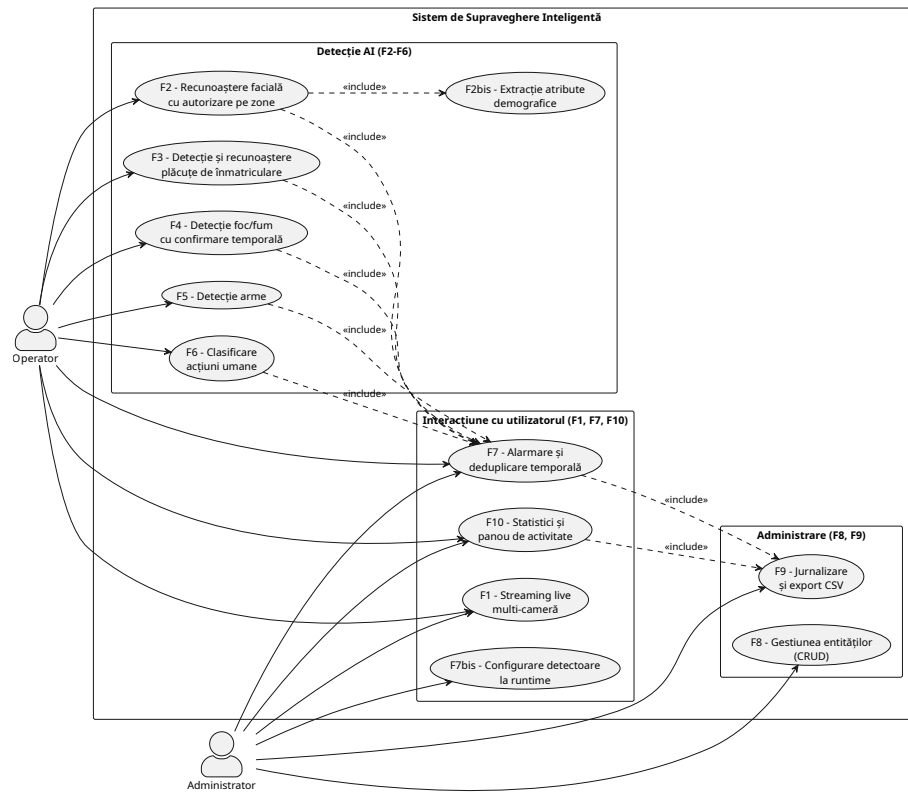


Figura 3.1: Diagrama use-case UML a sistemului de supraveghere inteligent, cu actorii Administrator și Operator în raport cu cele 10 cerințe funcționale, grupate pe domenii (detectie AI, interacțiune cu utilizatorul, administrare).

3.1.2 Cerințe non-funcționale

Cerințele nefuncționale stabilesc atributele de calitate sub care vor opera funcționalitățile sistemului. Acestea sunt marcate prin identificatori de forma **NF x** (paralel cu **F x** din §3.1.1), permițând referențierea lor în etapele ulterioare de proiectare, implementare și testare.

NF1 – Performanță (latență per cadru). Pipeline-ul principal de procesare a unui cadru, măsurat de la momentul achiziției până la difuzarea rezultatelor către clienți, trebuie să se încadreze sub **100 ms per cadru**, ceea ce permite atingerea unei rate de aproximativ **30 FPS** pe fluxul live al camerei principale. Această țintă este derivată din modul de operare paralel al firelor de execuție: fiecare detector rulează într-un fir dedicat, astfel încât latența percepută să fie dominată de cel mai lent detector de pe calea critică (recunoașterea facială), în timp ce detectorul cu cadență de clip (HAR/SlowFast, care procesează clipuri acumulate pe ~ 2 s) este suprapus asincron și nu se adună la latența per cadru.

NF2 – Scalabilitate. Arhitectura trebuie să permită extinderea la **mai multe camere simultan** fără modificări structurale ale codului, ci doar prin replicarea firelor de captură și ajustarea cozilor de cadre. Modelele AI sunt încărcate o singură dată în memorie și partajate între fluxuri, pentru a evita duplicarea costurilor de inițializare GPU.

NF3 – Securitate. Operațiile sensibile ale API-ului (precum gestiunea persoanelor, zonelor, plăcuțelor, utilizatorilor și configurarea modulelor AI) sunt protejate folosind o **autentificare bazată pe JWT** și restricționate printr-un **control de acces bazat pe roluri (RBAC)** cu nivelurile **admin** și **user**. Mai exact, acțiunile administrative sunt condiționate de dependența **require_admin**, regulă aplicată și endpoint-urilor de control al camerelor. Parolele sunt stocate în format hash (folosind bcrypt), iar token-urile au un timp de viață limitat la 8 ore. Cheia de semnare JWT este preluată din mediul de execuție prin variabila **JWT_SECRET**; în regim de producție (**APP_ENV=production**) absența sau lungimea insuficientă a acestei chei oprește explicit pornirea aplicației (*fail-fast*), fallback-ul cu o cheie aleatorie temporară fiind permis doar în dezvoltare. Sesiunile pot fi revocate instantaneu, înainte de expirarea celor 8 ore, printr-o **listă neagră** de token-uri (endpoint **/api/logout**). De asemenea, canalul WebSocket de streaming necesită autentificare. Token-ul JWT este transmis ca parametru de interogare (**/ws?token=...**) și este revalidat constant față de baza de date, prevenind astfel accesul unui cont suspendat sau șters la fluxul live. Comunicatia cu baza de date PostgreSQL se realizează prin credențiale dedicate, externalizate în variabile de mediu și izolate strict de datele utilizatorilor finali.

NF4 – Fiabilitate. Indexul FAISS este reconstruit deterministic din PostgreSQL la fiecare pornire, astfel încât pierderea fișierului de index nu duce la pierderea datelor faciale. Pipeline-ul tolerează căderea temporară a unei ca-

mere fără a opri procesarea celorlalte fluxuri. Cozile de cadre folosesc politici de tipul *drop-oldest* la saturare, astfel încât un detector lent să nu blocheze fluxul principal.

NF5 – Configurabilitate la runtime. Administratorul poate activa/dezactiva fiecare modul AI fără a necesita repornirea backend-ului și poate adăuga sau elimina dinamic camere IP. În ceea ce privește pragurile de încredere ale detectoarelor și ferestrele de *cooldown* pentru alarme și jurnale, acestea sunt definite ca parametri centralizați în cod. Deși pot fi ajustați fără a impune o reimplementare a logicii, acești parametri nu sunt expuși pentru modificare directă din interfața grafică.

NF6 – Mentenabilitate. Codul respectă o separare strictă pe module: fiecare detector (`facial_recognition_system`, `fire_detection_system`, `license_plate_recognition_system`, `weapon_detection_system`, `har_system`) este o unitate de sine stătătoare, importată de `app.py` și plasată într-un fir de execuție dedicat. Adăugarea unui nou detector se reduce la implementarea aceleiași interfețe și conectarea unui nou fir cu cozile sale.

NF7 – Observabilitate. Toate detecțiile și alarmele sunt înregistrate incluzând un timestamp, tipul evenimentului, camera, subiectul, nivelul de încredere, severitatea și metadatele JSON, oferind astfel un istoric complet și interogabil. La pornire, backend-ul raportează starea componentelor critice (resursele GPU/CUDA, modelele încărcate, conexiunea cu baza de date).

NF8 – Uzabilitate. Interfața de utilizare este dezvoltată sub forma unei aplicații web *responsive*, fiind accesibilă atât de pe stațiile de lucru fixe, cât și de pe dispozitive mobile pentru supravegherea de la distanță. Accesul în cadrul platformei este structurat pe roluri. Astfel, administratorul beneficiază de control complet asupra zonei de configurare și a gestiunii entităților, în timp ce operatorul utilizează interfața preponderent pentru vizualizarea fluxurilor video live și monitorizarea jurnalului de alarme.

NF9 – Portabilitate și mediu de execuție. Backend-ul este scris în Python (FastAPI), iar inferența AI necesită GPU compatibil CUDA pentru ca `InsightFace` și celelalte modele să atingă latențele țintă; în absența unui provider CUDA disponibil, `onnxruntime` revine automat pe `CPUExecutionProvider`, sistemul rămânând funcțional, dar cu latențe semnificativ mai mari. Frontend-ul este o aplicație React standard, distribuibilă ca artefact static.

3.2 Cazuri de utilizare

După formularea cerințelor în Secțiunea 3.1, pasul firesc este transpunerea acestora în interacțiuni concrete între actori și sistem. Această secțiune identifică

actorii, le delimitează responsabilitățile și descrie narativ cazurile principale de utilizare, oferind astfel puntea dintre *ce* oferă sistemul (cerințele $F\mathbf{x}$) și *cum* este folosit în practică.

Actori.

Sistemul distinge două roluri principale, mapate pe câmpul **role** al entității **user** din baza de date:

- **Administrator** – responsabil de configurarea sistemului. Are acces complet la gestiunea entităților (persoane, plăcuțe, zone, camere, utilizatori), la activarea/dezactivarea modulelor AI și activarea/dezactivarea camerelor de supraveghere. Tot ceea ce ține de *ce vede* sistemul și *pe cine îl recunoaște* este responsabilitatea acestui actor.
- **Utilizator obișnuit** – responsabil de supravegherea operațională. Vizualizează fluxurile live, monitorizează panoul de alarme, le triază și le marchează ca rezolvate și consultă jurnalele. Nu are dreptul să modifice configurația sistemului, să gestioneze entitățile sau să creeze conturi noi.

Ambii actori partajează un set comun de cazuri de utilizare (autentificare, schimbare parolă, vizualizare live și consultare jurnale), pe lângă care li se atribuie privilegii specifice fiecărui rol. Suplimentar, platforma interacționează cu o serie de *actori externi* pasivi, precum camerele IP, modelele AI și baza de date PostgreSQL. Deși nu reprezintă actori în sensul strict al modelării UML, aceste componente sunt esențiale și sunt tratate drept sisteme suport.

Cazuri de utilizare.

Cazurile sunt numerotate UC1...UC12 și descrise în formă narativă (pre-condiție, scenariu principal, post-condiție), cu trimitere la cerințele funcționale pe care le acoperă.

Figura 3.2 prezintă diagrama detaliată a cazurilor de utilizare. Cei doi actori (Administrator, Operator) sunt plasați la margini și conectați la elipsele cazurilor de utilizare. Cazurile UC1–UC3, UC10, UC11 și consultarea jurnalelor (UC12a) sunt partajate de ambii actori, în timp ce UC4–UC9 și vizualizarea statisticilor (UC12b) sunt accesibile doar Administratorului. Relațiile «include» marchează autentificarea (UC1) ca pre-condiție a tuturor celorlalte cazuri, iar relația «extend» leagă rezolvarea alarmei (UC11) de monitorizarea alarmelor (UC10).



- UC1 – Autentificare în sistem.** *Actori:* Administrator, Operator. *Pre-condiție:* utilizatorul are un cont creat. *Scenariu:* Utilizatorul introduce numele de utilizator și parola în pagina de login; backend-ul verifică hash-ul parolei, generează un token JWT semnat și îl returnează clientului, care îl stochează local și îl atașează tuturor cererilor ulterioare. *Post-condiție:* sesiunea este activă, iar UI-ul afișează doar componentele permise rolului utilizatorului. *Acoperă:* parte din NF3 (autentificare JWT).
- UC2 – Vizualizare fluxuri live multi-cameră.** *Actori:* Administrator, Operator. *Pre-condiție:* cel puțin o cameră este pornită. *Scenariu:* Utilizatorul deschide pagina principală; clientul stabilește o conexiune WebSocket cu backend-ul; backend-ul difuzează cadrele procesate (cu detecțiile suprapuse) către toți clienții conectați, la o rată țintă de aproximativ 30 FPS. Utilizatorul poate selecta camera afișată în prim-plan dintr-un grid. *Post-condiție:* fluxurile sunt afișate în timp real cu adnotările vizuale ale detectoarelor active. *Acoperă:* F1 (și indirect F2–F6 pentru adnotările suprapuse).
- UC3 – Schimbare parolă cont propriu.** *Actori:* Administrator, Operator. *Scenariu:* Utilizatorul autentificat introduce noua parolă; backend-ul o validează, îi calculează amprenta criptografică și o salvează în baza de date prin endpoint-ul `/api/users/me/password`. *Post-condiție:* la următoarea autentificare se folosește noua parolă. *Acoperă:* parte din NF3.
- UC4 – Înregistrare persoană nouă.** *Actor:* Administrator. *Scenariu:* Utilizând un singur formular, administratorul introduce metadatele persoanei (nume, `employee_id`, departament, zone autorizate) și încarcă una sau mai multe imagini faciale. La trimiterea datelor, interfața orchestrează transparent două apeluri REST succesive. Primul apel, `/api/persons/register`, stochează metadatele și returnează noul `person_id`. Al doilea apel, `/api/persons/{person_id}/faces`, declanșează la nivelul backend-ului extragerea vectorului de caracteristici InsightFace de 512 dimensiuni pentru fiecare imagine, salvarea acestuia în tabela `face_embeddings` și reconstrucția automată a indexului FAISS. *Excepție:* Dacă într-o imagine nu este detectată *exact* o singură față (situație care generează un vector de caracteristici de tip `None`), imaginea respectivă este ignorată și semnalată în lista de erori, fără a bloca procesarea celorlalte fișiere. *Post-condiție:* Imediat după finalizarea procesului, persoana poate fi identificată în fluxurile video live. *Acoperă:* F2, F8.
- UC5 – Gestiune zone.** *Actor:* Administrator. *Scenariu:* Administratorul creează, actualizează sau șterge o zonă, stabilind numele, descrierea și indicatorul `is_restricted`. Drepturile de acces nu sunt configurate la nivelul zonei, ci la nivelul individului, prin intermediul listei `authorized_zones` asociate fiecărei persoane (conform UC4), zona fiind referențiată prin nume. Orice modificare aplicată se propagă imediat în logica de verificare a accesului, odată cu reîmprospătarea memoriei cache aferente. *Post-condiție:* Detectarea unei persoane necunoscute sau neautorizate într-o

zonă cu acces restricționat va declanșa automat o alarmă (conform UC10).
Acoperă: F2, F8 și parțial F7.

UC6 – Gestiune plăcuțe. *Actor:* Administrator. *Scenariu:* Administratorul adaugă, modifică sau șterge datele aferente unei plăcuțe de înmatriculare (precizând numărul, tipul vehiculului, numele și ID-ul proprietarului, indicatorul `is_authorized`, data de expirare și diverse note). Spre deosebire de profilurile persoanelor, plăcuțele nu presupun o autorizare la nivel de zonă. În schimb, un vehicul este considerat autorizat dacă numărul său de înmatriculare figurează în baza de date în momentul procesării OCR. *Post-condiție:* Plăcuțele identificate sunt comparate cu înregistrările existente și *jurnalizate* cu statusul `authorized` sau `unauthorized`. Suplimentar, o citire OCR validă pentru o plăcuță neautorizată declanșează automat o alarmă cu nivel de severitate `high` (conform UC10 și F7). *Acoperă:* F3, F7, F8, F9.

UC7 – Adăugare/eliminare cameră. *Actor:* Administrator. *Scenariu:* Administratorul adaugă o cameră IP nouă prin specificarea URL-ului de streaming, a unui identificator unic și a locației, având totodată opțiunea de a activa o cameră persistentă din configurația stocată în baza de date (`/api/cameras`). Imediat după inițiere, backend-ul accesează fluxul video, lansează firul de execuție pentru captură și adaugă dispozitivul în lista camerelor active. Operația inversă, de eliminare a unei camere, va opri firul de execuție și va elibera resursele alocate. *Post-condiție:* Camera devine vizibilă sau este retrasă automat din interfața de vizualizare de tip grid. *Acoperă:* F1, F8.

UC8 – Configurarea modulelor AI la runtime. *Actor:* Administrator. *Scenariu:* Administratorul accesează panoul *Intelligence Settings* și activează sau dezactivează în mod individual modulele (recunoașterea facială, analiza demografică, citirea plăcuțelor, detecția focului, a armelor și a acțiunilor umane) utilizând endpoint-urile de tip `toggle`. Modificarea are efect imediat, fără repornirea backend-ului. Firul de execuție corespunzător începe sau încetează să mai pună cadre în coadă. *Post-condiție:* pipeline-ul rulează cu noul set de detectoare active. *Acoperă:* F7bis, NF5.

UC9 – Gestiune utilizatori. *Actor:* Administrator. *Scenariu:* Administratorul creează un cont nou, specificând numele, parola inițială și rolul asociat. Ulterior, acesta poate să modifice nivelul de acces atribuit sau să elimine conturi existente. *Post-condiție:* setul de utilizatori autorizați să acceseze sistemul este actualizat. *Acoperă:* F8, NF3.

UC10 – Monitorizare alarme. *Actori:* Administrator, Operator. *Pre-condiție:* pipeline-ul de procesare rulează și are detectoare active. *Scenariu:* ori de câte ori se declanșează o detecție critică, backend-ul, după aplicarea cooldown-ului de deduplicare, salvează o alarmă în baza de date împreună cu o captură (snapshot). Clientul interoghează periodic endpoint-ul

/api/alarms și afișează alarmele în panoul de alarme, cu severitatea, camera, timestamp-ul și captura asociată. *Post-condiție*: alarma este vizibilă tuturor operatorilor. *Acoperă*: F7, F2–F6, NF1.

UC11 – Rezolvare alarmă. *Actor*: Operator (sau Administrator). *Pre-condiție*: Există cel puțin o alarmă activă în sistem. *Scenariu*: Operatorul selectează o alarmă din panoul dedicat și o marchează drept rezolvată. Backend-ul înregistrează identitatea utilizatorului care a acționat, alături de timestamp-ul aferent rezolvării. Această operațiune poate fi aplicată și în masă (*bulk-update*) pentru a gestiona simultan un grup de alarme. *Extinde*: UC10. *Post-condiție*: Alarma vizată trece în starea **resolved** și este eliminată din vizualizarea alertelor active. *Acoperă*: F7.

UC12 – Consultare jurnale și statistici. *Scenariu*: Procesul este împărțit în două direcții principale de acțiune. Pentru *consultarea jurnalelor* (*actori*: Administrator și Operator), utilizatorul accesează pagina dedicată și aplică filtre în funcție de intervalul de timp, cameră, tipul evenimentului, status și subiect. Backend-ul returnează rezultatele sub formă paginată, oferind totodată opțiunea de export în format CSV. Pentru *vizualizarea statisticilor* (*actor*: exclusiv Administrator), interfața afișează grafice agregate precum evoluția detectiilor în timp, proporția fiecărui tip de eveniment și activitatea per cameră. *Post-condiție*: Utilizatorul obține o perspectivă de ansamblu detaliată asupra activității sistemului pentru perioada selectată. *Acoperă*: F9, F10.

Tabelul de trasabilitate cerință→caz de utilizare se regăsește implicit prin trimerile *Acoperă*: din fiecare descriere de mai sus și va fi folosit ca referință în Capitolul 6, unde fiecare caz de utilizare este verificat printr-un scenariu de test corespunzător.

3.3 Arhitectura generală a sistemului

După stabilirea cerințelor (§3.1) și a cazurilor de utilizare (§3.2), această secțiune prezintă arhitectura sistemului la nivel macro, evidențiind structura fundamentală, descompunerea în componente logice și distribuția acestora pe noduri fizice. Detaliile interne ale pipeline-ului multi-threaded și ale interfețelor REST/WebSocket sunt amânate pentru Secțiunile 3.4–3.6.

Sistemul este construit ca o aplicație web cu **trei straturi logice**: prezentare (clientul React), aplicație (backend-ul FastAPI cu pipeline-ul AI integrat) și persistență (baza de date PostgreSQL plus indexul FAISS reconstruit la pornire). Aceste straturi sunt utilizate de **cinci tipuri de actori tehnici**: utilizatorii umani prin browser, camerele video ca surse externe de date, modelele AI încărcate în memorie ca dependente în proces, baza de date ca sistem suport și GPU-ul ca resursă de calcul. Alegerea unei arhitecturi monolitice pentru backend, în care toate detectoarele rulează în același proces și partajează memoria GPU, este justificată în §3.4.3.

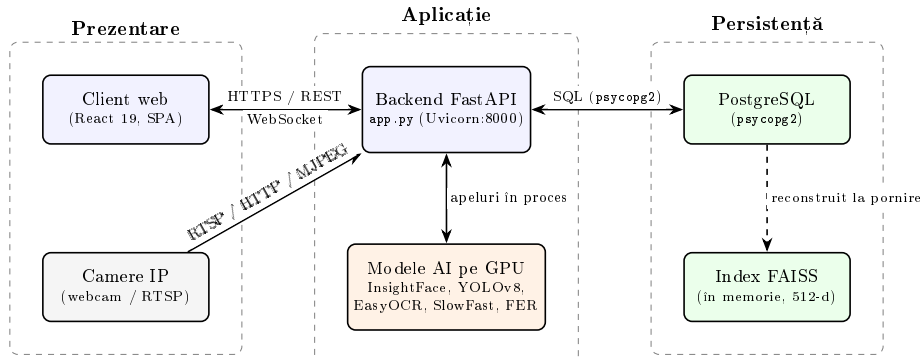


Figura 3.3: Diagramă bloc de ansamblu a sistemului, organizată pe cele trei straturi logice (Prezentare, Aplicație, Persistentă) și protocoalele de comunicare dintre componente: HTTPS/REST și WebSocket între client și backend, RTSP/HTTP/MJPEG pentru fluxurile camerelor, apeluri în proces către modelele AI de pe GPU și acces SQL la PostgreSQL prin driver-ul `psycopg2`. Indexul FAISS este menținut în memoria backend-ului și reconstruit din PostgreSQL la pornire.

3.3.1 Modelul client-server

Sistemul urmează un model client-server clasic, cu o singură instanță a serverului care deservește simultan mai mulți clienți web. Conexiunile sunt asimetrice: pentru operațiile tranzacționale (login, CRUD pe entități, interogări de jurnale, preluarea alarmelor) se folosește un canal **REST sincron** peste HTTP/HTTPS, iar pentru transmiterea continuă a cadrelor video procesate se folosește un canal **WebSocket** peste TCP, în care fluxul de date este predominant unidirecțional (server→client).

3.3.1.1 Clientul

Frontend-ul este o aplicație **React 19** compilată cu Create React App și stilizată cu Tailwind, livrabilă ca artefact static. Este o aplicație de tip *single-page*, organizată pe ecrane comutate printr-o stare internă (`activeTab`), fără un router dedicat: `CameraGrid`, `PersonManagement`, `PlateManagement`, `ZoneManagement`, `CameraManagement`, `UserManagement`, `AlarmManagement`, `Logs`, `Statistics`, `IntelligenceSettings`, `RecentActivity`. Clientul nu execută niciun cod AI, rolul său este de a afișa cadrele primite prin WebSocket, de a colecta input-ul utilizatorului și de a-l transmite ca apeluri REST.

3.3.1.2 Serverul

Backend-ul este o aplicație **FastAPI** pornită pe portul 8000 din `app.py`, încărcat ca o singură instanță Uvicorn (fără reload, pentru a evita dubla inițializare a modelelor pe GPU). Endpoint-urile sunt protejate prin token JWT și verificări

de rol (dependențele `require_admin` / `get_current_user`), aplicate înainte de invocarea handler-ului: operațiile administrative (controlul camerelor, gestiunea entităților, comutatoarele AI) necesită rolul de administrator, iar endpoint-urile operaționale (starea sistemului, listarea camerelor, jurnale) necesită doar autentificare. Canalul WebSocket este, la rândul lui, autentificat prin token JWT transmis ca parametru de interogare (aspect detaliat în §3.8). Pe lângă deservirea cererilor HTTP, serverul găzduiește pipeline-ul AI ca un set de fire de execuție de fundal, descrise în §3.4.

3.3.1.3 Sursele video

Camerele video funcționează ca actori externi care nu inițiază niciodată conexiuni către server. Serverul este responsabil cu deschiderea fluxurilor video prin intermediul bibliotecii OpenCV. Aceste surse pot include camera locală a stației și camere IP integrate prin URL-uri de streaming (RTSP/HTTP/MJPEG). Eșecul unei camere, cum ar fi un *timeout* sau o deconectare neprevăzută, nu propagă o eroare în cascadă, ci doar marchează camera ca inactivă și permite repornirea ulterioară.

3.3.1.4 Persistența

Stratul de persistență combină **PostgreSQL** utilizat pentru stocarea permanentă cu un **index FAISS** menținut în memoria backend-ului pentru căutarea vectorială rapidă a vectorilor de caracteristici faciale 512-d. Indexul nu reprezintă mecanismul principal de stocare a datelor, ci este reconstruit determinist din PostgreSQL la fiecare pornire prin modulul `faiss_index.py`, astfel încât pierderea fișierului de cache nu afectează integritatea informațiilor.

3.3.2 Diagrama de componente

La nivel intern, backend-ul este descompus în următoarele componente logice. Fiecare componentă este un modul Python sau un grup de fire de execuție cu responsabilitate clară, cuplate prin cozi `queue.Queue` sau prin apeluri directe.

Modul de captură. Implementat în `app.py` sub forma unui fir de execuție per cameră activă, care deschide fluxul prin OpenCV, citește cadrele în mod sincron și le distribuie (prin clonare `frame.copy()`) în cozile de procesare ale fiecărui detector activ și în coada `raw_frame_queue` folosită ca fallback pentru fuziune.

Modul AI. Containerul pentru toate detectoarele specializate, încărcate o singură dată la pornire și partajate între fluxurile camerelor. Conține următoarele sub-module independente, fiecare cu propriul fir de execuție și propria pereche de cozi (`*_processing_queue` de intrare, `*_results_queue` de ieșire):

- `facial_recognition_system` – detecție de fețe (**InsightFace**), vectori de caracteristici 512-d, căutare în FAISS, extragere atribute demografice (vârstă, gen prin InsightFace; emoție prin Mini-Xception/ FER).
- `license_plate_recognition_system` – detecție și OCR pe plăcuțe (**YOLOv8** pentru localizare + **EasyOCR** pentru text).
- `fire_detection_system` – detector foc/fum cu **confirmare temporală** pe fereastră multi-cadru, pentru reducerea fals pozitivelor.
- `weapon_detection_system` – detector **YOLOv8m** ajustat fin pe clasa unică **weapon**, antrenat pe Zenodo *Dangerous Items* + Roboflow *SOHAS*.
- `har_system` – recunoaștere acțiuni umane folosind **SlowFast-R50**, care clasifică fiecare clip în acțiunile **normal**, **fight** și **vandalism**.

Modul de fuziune. Firul *result merger* colectează rezultatele parțiale din toate cozile de ieșire ale detectoarelor active, le asociază pe baza câmpului `frame_id` și produce un cadru final adnotat (cu bounding box-uri, etichete, scoruri de încredere), plus un set agregat de detecții. Tot aici se aplică **logica de declanșare a alarmelor** (verificarea zonelor restricționate, deduplicarea pe cooldown) și **politica de jurnalizare** (un singur log per (cameră, tip, subiect) într-o fereastră scurtă).

Modul de control al accesului. Cache-urile `_person_zones_cache` și `_camera_zone_cache` și logica asociată din `app.py` verifică, pentru fiecare *persoană* identificată, dacă aceasta are dreptul de acces în zona asociată camerei (comparând zona camerei cu lista `authorized_zones` a persoanei). Cache-urile sunt reîmprospătate periodic (TTL) și forțat la modificarea entităților relevante. Plăcuțele nu participă la acest mecanism, autorizarea lor fiind globală (§3.1.1).

API REST. Stratul de controlere FastAPI definit în `app.py`, organizat pe domenii: `/api/login`, `/api/persons`, `/api/plates`, `/api/zones`, `/api/cameras`, `/api/users`, `/api/logs`, `/api/alarms`, `/api/stats`, plus endpoint-urile `toggle` pentru fiecare modul AI. Fiecare controler validează token-ul JWT, aplică verificarea de rol și delegă efectiv către modulele de business sau către `database_manager.py`.

Gestiunea WebSocket. Endpoint-ul `/ws` autentifică clientul (token JWT ca parametru de interogare) și îi alocă o **coadă proprie** (`queue.Queue`), înregistrată în dicționarul `client_queues`; modulul de fuziune nu publică într-o coadă partajată, ci difuzează fiecare cadru procesat către coada fiecărui client prin metoda `_broadcast_frame` (politică *drop-oldest* per client, astfel încât un client lent să-și piardă doar propriile cadre vechi). Mesajele difuzate (câmpul `type: video_frame`) conțin cadrul codificat (JPEG la calitate 75, redimensionat la maxim 800 px lățime, Base64), rezultatele detectoarelor pe câmpuri separate (`face_results`, `plate_results`, etc.) și starea modulelor (flag-urile `*_enabled`). Comunicarea este *strict*

unidirecțională (server→client): handler-ul doar trimite cadre, fără a citi mesaje de la client. Alarmerile nu sunt transmise prin acest canal, ci preluate separat prin REST (§3.6).

Nivelul de persistență. Modulul `database_manager.py` gestionează centralizat interacțiunea cu PostgreSQL printr-o conexiune `psycopg2` persistentă (cu `RealDictCursor`), oferind operații CRUD și interogări parametrizate pentru toate entitățile. Indexul FAISS, deși rulează tehnic în memoria procesului, este considerat parte a acestui nivel deoarece este reconstruit din baza de date. Datele binare sunt păstrate tot în baza de date: reprezentările vectoriale faciale ca BYTEA în `face_embeddings`, iar capturile de alarmă ca șiruri Base64 în coloana `snapshot` a tabelului `alarms`. Sistemul nu expune un serviciu de fișiere statice.

Cuplajul între componente este slab: fiecare detector poate fi activat/dezactivat independent (vezi UC8) fără a afecta restul, iar adăugarea unui nou detector se reduce la implementarea aceleiași interfețe (*intrare*: cadru, *ieșire*: listă de detecții) și conectarea unui nou fir cu cozile sale.

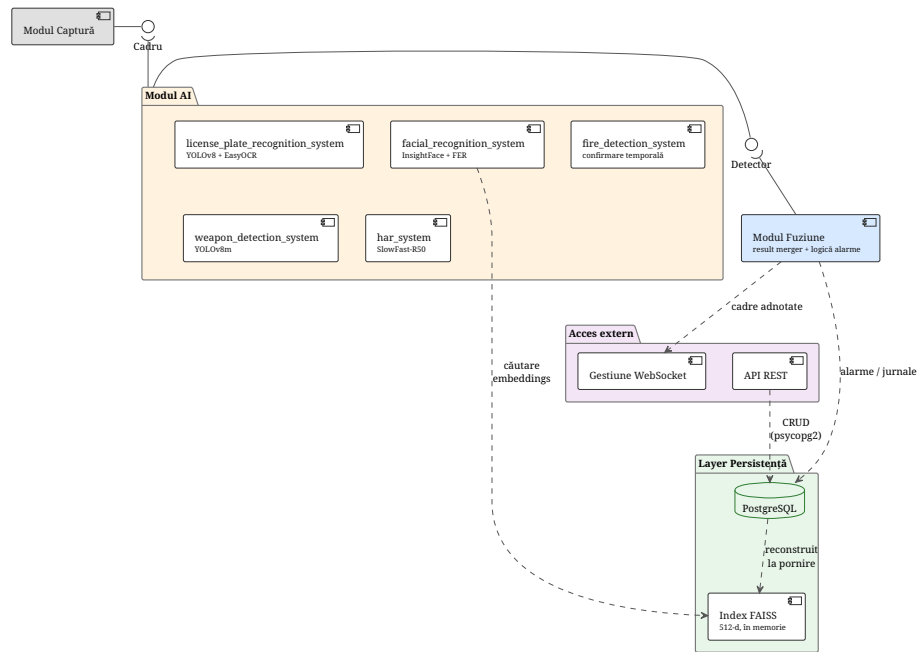


Figura 3.4: Diagrama de componente UML a backend-ului. Modulul Captură furnizează cadre (prin interfața *Cadru*) către Modulul AI. Acesta din urmă este o componentă complexă, alcătuită din cele cinci detectoare independente (recunoaștere facială, plăcuțe de înmatriculare, foc/fum, arme și acțiuni umane), care expune contractul comun *Detector* către Modulul Fuziune. Rolul Fuziunii este de a difuza cadrele adnotate către componenta de Gestiune WebSocket și de a salva alarmele și jurnalele generate. În paralel, API-ul REST realizează operațiile de tip CRUD. Ambele componente depind de Nivelul de Persistență, format din baza de date PostgreSQL și indexul FAISS (reconstruit la fiecare pornire). Contractele dintre componente sunt ilustrate vizual prin interfețe de tip lollipop.

3.3.3 Diagrama de implementare (deployment)

Distribuția fizică a sistemului este simplă în varianta de bază, constând într-un singur nod server pe care rulează toate componentele de procesare. Această arhitectură se poate extinde modular în cazul în care numărul de camere sau de clienți crește. Nodurile descrise mai jos sunt suficiente atât pentru scenariul demonstrativ al lucrării, cât și pentru un deployment de tip pilot.

3.3.3.1 Nodul client (browser)

Orice mașină cu un browser modern (Chrome, Firefox, Edge) care suportă WebSocket și ES2020. Nu există dependente instalate local: aplicația React este servită ca artefact static și se conectează la backend prin URL-ul configurat în `frontend/.env`. În scenariul de operare pot exista mai mulți clienți simultan.

3.3.3.2 Nodul server cu GPU

Mașina principală a sistemului, pe care rulează:

- procesul Python `app.py` (FastAPI + Uvicorn) cu toate firele AI;
- modelele AI încărcate în memoria GPU (InsightFace, YOLOv8, SlowFast, modelul de foc); modelul de emoție FER (Mini-Xception, TensorFlow) este ținut deliberat pe CPU, pentru a evita conflictele de runtime cu `onnxruntime-gpu`;
- indexul FAISS în memoria RAM;
- runtime-ul `onnxruntime-gpu` cu `CUDAExecutionProvider`, configurat la inițializarea InsightFace în `facial_recognition_system.py` (cu fallback pe `CPUExecutionProvider`).

Cerințele minime hardware sunt impuse de modelele AI: un GPU cu suport CUDA și cel puțin 8 GB VRAM pentru încărcarea simultană a tuturor detectoarelor; un CPU multi-core pentru cele ~ 7 fire de execuție; un SSD pentru log-uri și capturi.

3.3.3.3 Nodul bază de date

PostgreSQL poate coexista cu backend-ul pe același host (varianta implicită – conexiune prin `localhost`) sau poate fi externalizat pe un nod separat pentru scenarii de producție, accesat prin TCP cu credențiale dedicate.

3.3.3.4 Camere IP

Camerele sunt noduri independente, conectate la aceeași rețea locală cu serverul, care expun fluxuri RTSP, MJPEG sau HTTP. Identificarea lor este făcută prin URL plus un identificator logic (`CAM-01`, `CAM-02` etc.). Latența rețelei dintre camere și server contribuie direct la bugetul de 100 ms al cerinței NF1, motiv pentru care se recomandă rețea cablată sau Wi-Fi dedicat.

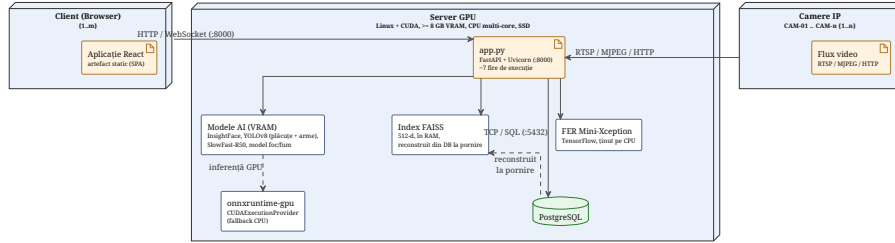


Figura 3.5: Diagrama de implementare (deployment) UML a sistemului. Nodul *Client (Browser)* – în una sau mai multe instanțe ($\{1..m\}$) – servește aplicația React ca artefact static și comunică cu serverul prin HTTP și WebSocket pe portul 8000. Nodul *Server GPU* găzduiește procesul `app.py` (FastAPI + Uvicorn, cu cele ~ 7 fire de execuție), modelele AI încărcate în VRAM (InsightFace, cele două instanțe YOLOv8 pentru plăcuțe și arme, SlowFast-R50 și modelul de foc/fum) prin `onnxruntime-gpu`, modelul de emoție FER ținut deliberat pe CPU, indexul FAISS în memoria RAM și baza de date **PostgreSQL** (acces prin TCP/SQL pe portul 5432, indexul FAISS fiind reconstruit din ea la pornire). *Camerele IP* (CAM-01..CAM-n, $\{1..n\}$) furnizează fluxuri RTSP/MJPEG/HTTP.

3.4 Proiectarea pipeline-ului multi-threaded

Pipeline-ul multi-threaded reprezintă decizia arhitecturală centrală a sistemului. Acesta definește modul în care backend-ul reușește să ruleze simultan cinci modele AI eterogene procesând fluxul aceleiași camere, cu scopul de a livra rezultatele agregate respectând limita de latență NF1. Aceasta secțiune descrie structura firelor de execuție, detaliază mecanismele de sincronizare alese și argumentează de ce o execuție paralelă in-proces este preferabilă comparativ cu o execuție secvențială sau cu o arhitectură bazată pe microservicii.

Decizia de bază este: **un fir de execuție per responsabilitate, cozi thread-safe cu capacitate limitată ca mecanism de cuplare slabă, un singur proces care găzduiește toate modelele pe GPU**. Această alegere derivă din trei constrângeri: (i) modelele AI au timpi de inferență neomogeni, care se întind pe mai bine de un ordin de mărime – de la ~ 4 ms pentru detecția YOLO a plăcuțelor până la ~ 83 ms pentru recunoașterea facială cu analiză demografică (latențe măsurate, vezi Tabelul 3.1 din §3.4.3), (ii) inițializarea GPU a fiecărui model este costisitoare și trebuie făcută o singură dată, (iii) operatorul trebuie să poată activa/dezactiva dinamic detectoarele (cerința F7bis, NF5).

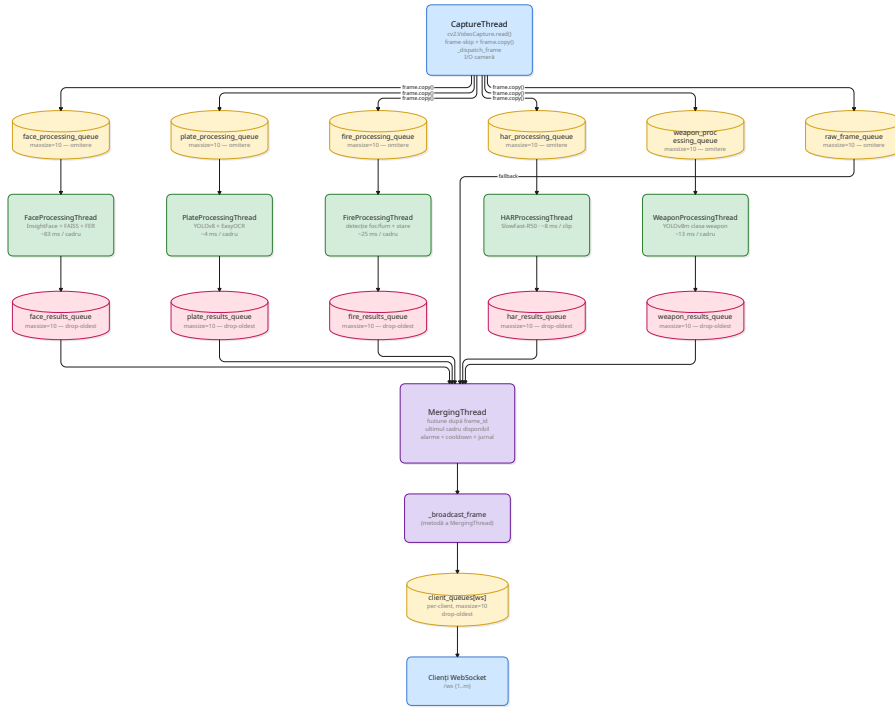


Figura 3.6: Diagrama de flux a pipeline-ului multi-threaded. Cele **sapte fire de execuție** sunt reprezentate ca dreptunghiuri, iar cele unsprezece cozi `queue.Queue(maxsize=10)` ca cilindri. Fluxul cadrelor pornește de sus, de la `CaptureThread` (care, prin `_dispatch_frame`, depune câte o copie `frame.copy()` în cozile de *intrare*), coboară prin cele cinci coloane paralele de fire AI – fiecare adnotat cu latența de inferență măsurată (Tabelul 3.1) – ale căror rezultate converg, prin cozile de *ieșire*, către `MergingThread`. Acesta aplică politica *ultimul cadru disponibil* (folosind `raw_frame_queue` ca *fallback*), declanșează alarmele și difuzează cadrul adnotat prin `_broadcast_frame` în cozile per-client `client_queues`, de unde este preluat de clienții `WebSocket`. Politica de saturare diferă în funcție de direcția cozii: *omitere* la intrare (cadrul nou este abandonat dacă firul detectorului are deja un backlog) și *drop-oldest* la ieșire (se scoate cel mai vechi rezultat pentru a-l insera pe cel nou).

3.4.1 Cele 7 fire de execuție

Backend-ul pornește, la inițializarea instanței `EnhancedSurveillanceSystem`, exact șapte fire de execuție daemon (vizibile în `app.py` prin numele `CaptureThread`, `FaceProcessingThread`, `PlateProcessingThread`, `FireProcessingThread`, `HARProcessingThread`, `WeaponProcessingThread`, `MergingThread`). Fiecare fir are o singură responsabilitate și interacționează cu restul exclusiv prin cozile partajate, evitând astfel sincronizarea prin variabile partajate la nivel fin.

3.4.1.1 Firul 1 – Captura video

Firul `CaptureThread` citește ciclic câte un cadru de la fiecare cameră activă prin `cv2.VideoCapture.read()`, cu reconectare automată a camerelor IP căzute. Fiecare cadru primește un `frame_id` monoton crescător și, sub rezerva unui mecanism de *frame-skip* (se procesează un cadru la fiecare `frame_skip`), este transmis metodei `_dispatch_frame`. Aceasta depune, pentru fiecare detector activ, o *copie* a cadrului (`frame.copy()`) în coada de procesare corespunzătoare (`face_processing_queue`, `plate_processing_queue`, `fire_processing_queue`, `har_processing_queue`, `weapon_processing_queue`) și, doar dacă toate cozile detectoarelor active au acceptat cadrul, o copie în `raw_frame_queue`, folosită de fuziune ca *fallback* pentru cadrele pentru care nu au sosit încă rezultatele detectoarelor.

3.4.1.2 Firul 2 – Recunoaștere facială și analiză demografică

Firul `FaceProcessingThread` consumă din `face_processing_queue`, rulează detecția fețelor cu `InsightFace`, calculează reprezentările vectoriale 512-d, le caută în indexul FAISS în memoria procesului și asociază fiecărei detecții vârsta și genul (din `InsightFace`), iar pentru fețele necunoscute și emoția (din modelul `Mini-Xception/FER`). Toate aceste operații sunt încapsulate într-un singur fir pentru că au aceeași intrare (regiunea facială) și aceeași localitate de cache GPU; separarea recunoașterii faciale de demografie ar dubla detecția fețelor fără beneficiu real.

3.4.1.3 Firul 3 – Plăcuțe de înmatriculare

Firul `PlateProcessingThread` rulează detecția plăcuțelor cu `YOLOv8` specializat, urmată de OCR-ul `EasyOCR` pe fiecare regiune detectată. Combinația detector+OCR este menținută în același fir deoarece OCR-ul operează doar pe ROI-urile produse de detector, iar separarea ar introduce o coadă inutilă pentru ROI-uri.

3.4.1.4 Firul 4 – Foc și fum

Firul `FireProcessingThread` rulează modelul de detecție foc/fum și menține starea temporală necesară confirmării pe fereastră multi-cadru (cerința F4). O

detecrie singulară nu produce alarmă. Doar o dectecție confirmată pe câteva cadre consecutive declanșează câmpul **alert** pe rezultatul publicat. Această stare este gestionată local, strict la nivelul firului de execuție, eliminând astfel necesitatea unor mecanisme de sincronizare concurentă.

3.4.1.5 Firul 5 – Recunoaștere acțiuni umane

Firul `HARProcessingThread` are un regim de funcționare distinct: acumulează cadre într-un buffer intern și, când are clipul complet de 64 cadre la `clip_duration_sec=2.0`, rulează `SlowFast-R50` cu pathway-urile `slow` (8 cadre) și `fast` (32 cadre) la `crop_size=224`, clasificând clipul în **normal**, **fight** sau **vandalism**. Forward-pass-ul `SlowFast` nu este cel mai scump dintre detectoare – măsurat, el costă în mediană ~ 8 ms pe GPU (Tabelul 3.1), mult sub recunoașterea facială. Particularitatea acestui fir este *cadența*, nu costul brut: el produce un rezultat o singură dată la ~ 2 s, pe măsură ce se umple clipul de 64 de cadre. Motivul principal pentru izolarea acestui modul nu este riscul de a bloca resursa GPU, ci necesitatea de a preveni ca ritmul său de acumulare a cadrelor să dicteze rata de actualizare a celorlalte detectoare din pipeline.

3.4.1.6 Firul 6 – Arme

Firul `WeaponProcessingThread` rulează modelul `YOLOv8m` ajustat fin pe clasa unică **weapon**. Detecria de armă este fără stare — fiecare cadru produce un rezultat independent printr-un singur forward-pass, fără buffer temporal (ca HAR) și fără procesare în aval pe ROI-uri (ca OCR-ul plăcuțelor). Tocmai această independență între cadre justifică firul dedicat: dectecția poate rula la rata maximă a camerei, izolată de latența celorlalte detectoare — esențial pentru o clasă critică pentru siguranță.

3.4.1.7 Firul 7 – Fuziune

Firul `MergingThread` este nucleul de agregare: consumă neblocaant din toate cozile de rezultate (`face_results_queue`, `plate_results_queue`, `fire_results_queue`, `har_results_queue`, `weapon_results_queue`) plus din `raw_frame_queue`, asociază rezultatele după `frame_id` și produce mesajul final pentru clienți (cadru adnotat + listă de dectecții + alarme noi). Tot aici se aplică logica de declanșare a alarmelor (verificarea zonelor restricționate via `_person_zones_cache`, deduplicarea pe baza dicționarului `alarm_cooldowns` cu un cooldown de 30 s, jurnalizarea cu deduplicare pe `log_cooldowns` de 5 s). Mesajul final este difuzat prin metoda `_broadcast_frame`, care îl depune în coada proprie a fiecărui client `WebSocket` conectat (dicționarul `client_queues`), fără a trece printr-o coadă de ieșire partajată.

3.4.1.8 Justificarea separării

Alocarea pe fire respectă două principii: **omogenitatea computațională** (operații care partajează intrarea și cache-ul GPU stau împreună – ex. recunoaște-

re facială cu demografie, detecție plăcută cu OCR) și **izolarea costului și a cadenței** (firul cel mai costisitor pe cadru – recunoașterea facială cu demografie, ~83 ms – și firul cu cadență de clip – HAR – rămân separate, ca să nu impună latența, respectiv ritmul lor, celorlalte detectoare). Numărul 7 nu este un obiectiv în sine, ci o consecință a celor cinci familii de detecții cerute plus capătul de pipeline (captură) și nucleul de agregare (fuziune).

3.4.2 Mecanisme de sincronizare

Sincronizarea este realizată prin **cozi thread-safe cu capacitate limitată** ca mecanism principal și prin **lock-uri fine de tip threading.Lock** pentru structurile de stare partajată (cache de zone, dicționare de cooldown, lista de camere). Această împărțire reduce semnificativ riscul de *deadlock* și de *race condition*, pentru că majoritatea fluxului de date trece prin cozi, unde sincronizarea este implementată odată în standard library.

3.4.2.1 Cozi sigure pentru concurență (*thread-safe*)

Toate cele unsprezece cozi utilizate în fluxul de procesare sunt instanțiate prin `queue.Queue(maxsize=10)` – cozi de tip FIFO cu mecanism intern de blocare, care oferă suport nativ pentru apeluri non-blocante (`block=False`) și pentru așteptare cu termen-limită (de exemplu, `get(timeout=0.1)`). Capacitatea limitată la 10 elemente reprezintă o decizie arhitecturală explicită: o memorie tampon (*buffer*) redusă garantează o latență minimă (cel mai vechi cadru aflat în procesare are o întârziere de maximum 10 cadre), dar impune și gestionarea activă a fenomenului de saturare. Politica de tratare a saturării diferă în funcție de rolul cozii, având însă un obiectiv unic: prevenirea creșterii necontrolate a dimensiunii acesteia. La **intrare** (cozile de procesare `*_processing_queue` și `raw_frame_queue`, alimentate de `CaptureThread` prin `_dispatch_frame`), dacă structura asociată unui detector este plină, noul cadru este pur și simplu *ignorat* (incrementând contorul `queue_skips`). Această decizie se justifică prin faptul că detectorul respectiv procesează deja un volum restant de date (*backlog*), nefiind oportună agravarea întârzierii. Coadă `raw_frame_queue` este alimentată exclusiv dacă toate cozile detectoarelor au acceptat cadrul curent; în caz contrar – sau dacă aceasta este deja plină – cadrul este contorizat separat în variabila `frames_dropped`. La **ieșire** (cozile `*_results_queue` și cozile individuale per-client din rețeaua WebSocket), producătorul aplică strategia **drop-oldest**: extrage explicit elementul cel mai vechi utilizând `get(block=False)` și introduce noul rezultat. În ambele scenarii, principiul fundamental rămâne același: *prioritatea o deține cel mai recent cadru sau rezultat, evitându-se acumularea unui istoric de date învechite*.

3.4.2.2 Partajarea cadrului curent

Cadrela citită de `CaptureThread` sunt *clonate* (`frame.copy()`) înainte de a fi distribuite în cozile detectoarelor. Această decizie elimină complet sincronizarea

pe conținutul cadrului: fiecare fir AI lucrează pe propria copie, prevenind astfel riscul modificării concurente a matricei de pixeli de către un alt fir de execuție. Costul memoriei este absorbit de capacitatea limitată a cozilor (cel mult $11 \times 10 = 110$ cadre simultan în RAM, în jur de 200 MB pentru rezoluții HD), iar costul copierii este sub 1 ms, neglijabil față de bugetul NF1.

3.4.2.3 Mecanismul *ultimul cadru disponibil*

Firul de fuziune nu așteaptă să aibă rezultatele *tuturor* detectoarelor pentru fiecare `frame_id`. Așteptarea ar reduce throughput-ul la minimum dintre detectoare (HAR, care livrează rezultate o dată la ~ 2 s, ar limita restul la 0.5 FPS). În schimb, fuziunea aplică o politică de tip *ultimul cadru disponibil*: consumă neblocaant din toate cozile de rezultate, păstrează în structuri locale ultimele rezultate per cameră primite de la fiecare detector și le suprapune pe cadrul cel mai recent. Astfel, detecțiile cu cadență continuă, produse pentru fiecare cadru (fețe, plăcuțe, foc, armă), sunt afișate la rata camerei, iar detecția cu cadență de clip (HAR), care livrează un rezultat o dată la ~ 2 s, este suprapusă cu valoarea ei cea mai recentă până la următoarea actualizare. Acest mecanism este cel care permite respectarea cerinței NF1 de 30 FPS *indiferent* de detectorul cel mai lent activ.

3.4.2.4 Stare partajată protejată prin lock-uri

Câteva structuri trebuie accesate concurent: lista camerelor active (`cameras_lock`), dicționarele cu intervalele de latență pentru alarme și jurnale (`alarm_lock`, `log_lock`), structurile de stocare temporară (*cache*) pentru autorizarea pe zone (`_zone_cache_lock`) și dicționarul cozilor per-client WebSocket (`client_lock`). Pentru toate acestea se folosesc mecanisme de sincronizare de tip `threading.Lock`, aplicate pe o secțiune de cod foarte restrânsă (câteva linii în interiorul blocului `with self.X_lock:`), pentru a minimiza timpul în care un fir de execuție rămâne blocat.

3.4.3 Justificarea procesării paralele

Alternativa naturală la pipeline-ul paralel este **procesarea secvențială** – pentru fiecare cadru, se aplică pe rând detector 1, detector 2, ..., detector 5, apoi se trimite rezultatul la client. Această variantă a fost respinsă din mai multe motive concrete, măsurabile pe propriul cod.

3.4.3.1 Utilizarea GPU-ului

Modelele AI partajează aceeași memorie GPU, dar nu același tip de calcul: InsightFace face inferență single-shot pe regiuni mici, YOLO face inferență pe imagine întreagă, SlowFast face inferență pe clip 3D temporal. Rularea lor în fire separate permite suprapunerea **copierii host→device** a unui detector cu **kernel execution** a altuia, prin streams CUDA, ceea ce într-o execuție secvențială ar rămâne timp mort. În plus, ONNX Runtime și PyTorch folosesc

thread pool-uri interne care exploatează concurența la nivel de operator – iar acestea sunt active doar dacă apelurile vin din fire diferite.

3.4.3.2 Măsurarea latențelor de inferență

Cifrele de latență folosite în această secțiune și în Figura 3.6 nu sunt estimări, ci valori **măsurate** pe sistemul real. Pentru aceasta a fost rulat un script de benchmark dedicat (`benchmark_inference.py`), care încarcă fiecare detector exact ca în `app.py` și îi cronometrează metoda `process_frame` pe un clip video reprezentativ – o filmare în care apar mai multe persoane (deci mai multe fețe simultan) și, ocazional, autovehicule cu plăcuțe de înmatriculare vizibile.

Metodologia urmărește izolarea timpului *de inferență* de costurile de pornire. Pentru fiecare detector se rulează întâi 10 cadre de încălzire (prima inferență pe GPU este sistematic mai lentă, din cauza alocării de memorie și a compilării kernel-urilor CUDA), apoi se măsoară 200 de cadre cu `time.perf_counter()`, forțând sincronizarea GPU (`torch.cuda.synchronize()`) înainte de oprirea cronometrului, astfel încât să se contorizeze execuția efectivă a kernel-urilor, nu doar lansarea lor asincronă. Se raportează mediana și percentila 95, mai robuste decât media în prezența cozilor lungi de latență. Inițializarea modelelor (încărcarea pe GPU) nu este inclusă în măsurători.

Măsurătorile au fost efectuate pe o stație cu **GPU NVIDIA GeForce RTX 4060 Laptop**, **CPU Intel Core Ultra 9 185H** și 30 GB RAM. Toate modelele rulează pe GPU, cu o singură excepție notabilă: clasificatorul de emoții (Mini-Xception/FER) rulează **pe CPU**, deoarece este suficient de mic încât latența să rămână acceptabilă, iar GPU-ul rămâne dedicat modelelor grele (vezi §4.3.4). Rezultatele sunt sintetizate în Tabelul 3.1.

Tabel 3.1: Latențe de inferență măsurate per detector, pe un clip video reprezentativ (200 de cadre măsurate per modul, după 10 cadre de încălzire). Hardware: GPU NVIDIA GeForce RTX 4060 Laptop, CPU Intel Core Ultra 9 185H.

Detector	Mediană (ms)	p95 (ms)	Debit (cadre/s)
Plăcuțe (YOLOv8 + EasyOCR)*	4.0	4.7	249
Foc/fum	24.6	25.6	41
Armă (YOLOv8m)	13.1	13.5	76
Acțiuni umane HAR (SlowFast-R50)	8.1	17.3	124
Recunoaștere facială (InsightFace + FAISS + FER)	83.4	161.4	12

* Mediana modulului de plăcuțe este dominată de cadrele fără plăcuță, în care rulează doar detecția YOLO; când EasyOCR se declanșează pe o plăcuță prezentă, latența crește la ~40–125 ms (vizibil în valorile maxime).

Trei precizări sunt necesare pentru interpretarea corectă a cifrelor. În primul rând, fiecare detector a fost măsurat *izolat*; în pipeline-ul real, cele cinci modele rulează concurent pe aceeași placă, astfel încât latențele sub sarcină completă, cu competiție pentru resursele GPU, sunt mai mari decât valorile per-modul din

tabel – care reprezintă deci un *plafon* optimist. În al doilea rând, mediana de ~ 4 ms a modulului de plăcuțe reflectă majoritatea cadrelor în care *nu* apare nicio plăcuță (rulează doar detecția YOLO); comportamentul real este dependent de date, latența urcând la ~ 40 – 125 ms atunci când OCR-ul se aplică efectiv pe o plăcuță. În al treilea rând, cifrele sunt specifice hardware-ului folosit; pe un alt GPU ele s-ar modifica proporțional, motiv pentru care configurația de test este raportată explicit.

3.4.3.3 Latența percepută per modul

Într-un pipeline secvențial, latența percepută de utilizator pentru *orice* detector este suma timpilor de inferență ai *tuturor* detectoarelor active. Cu latențele măsurate (Tabelul 3.1), suma medianelor celor cinci detectoare este de ~ 133 ms ($4 + 25 + 13 + 8 + 83$ ms), iar doar recunoașterea facială costă ~ 83 ms. Astfel, chiar și un client interesat exclusiv de fețe ar primi rezultate abia după ce *toate* detectoarele active au terminat, adică la ~ 133 ms după captură. Aceasta ar reduce rata de afișare la ~ 7 – 8 cadre/s, sub pragul NF1 de 30 FPS, și ar depăși bugetul de latență de 100 ms. În pipeline-ul paralel, fiecare modul își are propria latență individuală (de la ~ 4 ms la ~ 83 ms), iar mecanismul *ultimul cadru disponibil* face ca afișarea să fie limitată de rata camerei, nu de suma detectoarelor.

3.4.3.4 Dezactivarea selectivă

Cerința F7bis cere posibilitatea activării/dezactivării individuale a detectoarelor la runtime. Într-un pipeline secvențial, dezactivarea unui modul ar însemna fie un if-else în corpul ciclului principal (cod fragil, cu test la fiecare cadru), fie o reconfigurare a lanțului de procesare. În arhitectura paralelă, dezactivarea înseamnă pur și simplu **a nu mai depune cadre** în coada detectorului respectiv – firul rămâne pornit, blocat pe `get()`, fără cost CPU. Reactivarea este simetrică și instantanee.

3.4.3.5 Tolerare la eșec

Dacă un detector se confruntă cu o eroare recuperabilă (excepție tratată la nivel local sau model temporar indisponibil), pipeline-ul paralel funcționează independent de acesta. Firul de execuție corespunzător consumă în continuare cadrele din buffer și înregistrează eroarea, iar etapa de fuziune gestionează lipsa datelor în mod transparent. Într-un flux secvențial, orice excepție ar trebui interceptată individual, iar penalizarea de performanță generată de blocurile `try/except` s-ar propaga asupra latenței întregului sistem.

3.4.3.6 De ce nu microservicii separate

O variantă și mai distribuită ar fi un detector per proces (sau per container), cu IPC între ele. Această opțiune a fost respinsă pentru că ar fi multiplicat costul de inițializare GPU (fiecare proces și-ar fi încărcat propriile modele, depășind

rapid memoria VRAM disponibilă), ar fi introdus latență suplimentară prin serializare/deserializare cadrelor și ar fi complicat semnificativ deployment-ul (cerința NF9 de portabilitate). Multi-threading în-proces oferă paralelismul logic dorit cu cost zero la nivel de date partajate – toate firele văd aceeași memorie GPU și aceleași structuri de cache, ceea ce este exact ce trebuie pentru un singur server cu GPU dedicat (vezi §3.3.3).

3.5 Proiectarea API-ului REST

Toate operațiile tranzacționale ale sistemului (precum autentificarea, operațiile de tip CRUD asupra entităților, interogarea jurnalelor și a statisticilor, alături de controlul modulelor AI) sunt expuse prin intermediul unui API REST. Acesta este dezvoltat utilizând framework-ul FastAPI și este montat sub prefixul comun `/api`. Acest canal de comunicare este complementar protocolului WebSocket (detaliat în §3.6). Mai exact, API-ul REST deservește cereri punctuale, sincrone, bazate pe paradigma cerere-răspuns, în timp ce conexiunea WebSocket este dedicată transmiterii fluxului continuu de cadre video și alerte. Prezenta secțiune detaliază endpoint-urile disponibile (grupate pe categorii de funcționalități), convențiile de denumire adoptate, formatul standardizat al răspunsurilor și codurile HTTP de eroare utilizate.

3.5.1 Convenții de denumire

API-ul respectă consecvent convențiile REST: substantive la plural pentru colecții (`/api/persons`, `/api/zones`, `/api/plates`, `/api/cameras`, `/api/streams`, `/api/users`), identificatorul resursei ca segment de cale (`/api/persons/{person_id}`), iar verbul HTTP exprimă acțiunea (GET pentru citire, POST pentru creare, PUT pentru actualizare completă, PATCH pentru actualizare parțială, DELETE pentru ștergere). Sub-resursele și acțiunile derivate sunt exprimate ca segmente suplimentare (`/api/persons/{person_id}/faces`, `/api/persons/{person_id}/access-history`).

Cele două aspecte legate de camere sunt modelate ca resurse distincte, ambele denumite la plural. Fluxurile video procesate în timp real sunt expuse ca resursa `/api/streams`: un POST pornește un flux, un GET le listează pe cele active, iar un DELETE oprește un flux individual sau toate fluxurile. Configurațiile de camere persistate în baza de date sunt expuse separat, ca resursa `/api/cameras`. În același spirit, statisticile agregate folosesc consecvent sufixul `stats` – `/api/stats` la nivel de sistem, respectiv `/api/alarms/stats`, `/api/logs/stats`, `/api/har/stats` și `/api/weapon/stats` la nivel de modul – iar alarmele sunt grupate sub `/api/alarms`.

3.5.2 Catalogul endpoint-urilor pe categorii

Autentificare.

- POST /api/login – autentifică un utilizator (nume + parolă) și returnează un token JWT plus datele de profil.
- POST /api/logout – revocă token-ul curent (logout efectiv): jti-ul acestuia este înscris în lista neagră `revoked_tokens`, devenind invalid imediat, înainte de expirarea naturală.

Stare și control fluxuri video (runtime).

- GET /api/status – starea curentă a sistemului: streaming activ per cameră, ce module AI sunt pornite, statistici de performanță.
- POST /api/streams – pornește un flux video; corpul cererii selectează sursa: "local" pentru camera laptopului (id-ul rezervat CAM-01) sau "ip" împreună cu URL-ul, id-ul și locația unei camere IP.
- GET /api/streams – listează fluxurile active (camera locală + camerele IP).
- DELETE /api/streams/{camera_id} – oprește un flux individual; DELETE /api/streams – oprește toate fluxurile.

Control module AI (toggle).

- POST /api/face/toggle, /api/plate/toggle, /api/demographics/toggle, /api/fire/toggle, /api/har/toggle, /api/weapon/toggle – activează/dezactivează individual fiecare detector (cerința F7bis).
- GET /api/har/stats, GET /api/weapon/stats – statistici interne ale celor două detectoare.

Persoane.

- POST /api/persons/register – creează fișa unei persoane noi (meta-date) și returnează `person_id`-ul.
- GET /api/persons – listează persoanele; GET /api/persons/departments – departamentele distincte.
- GET | PUT | DELETE /api/persons/{person_id} – citire/actualizare/ștergere a unei persoane.
- POST /api/persons/{person_id}/faces – încarcă imagini faciale pentru persoană (atât la înregistrare, cât și ulterior).
- GET /api/persons/{person_id}/access-history – istoricul accesărilor persoanei.

Plăcuțe de înmatriculare.

- POST /api/plates/register; GET /api/plates; GET /api/plates/vehicle-types.
- GET | PUT | DELETE /api/plates/{plate_number}; GET /api/plates/{plate_number} /access-history.

Zone.

- GET /api/zones; POST /api/zones; PUT | DELETE /api/zones/{zone_id}.

Camere (persistate în baza de date).

- GET /api/cameras; POST /api/cameras; PUT | DELETE /api/cameras/{camera_id}.

Alarmer.

- GET /api/alarms; GET /api/alarms/stats; GET /api/alarms/{alarm_id}.
- PATCH /api/alarms/{alarm_id} – marchează o alarmă (ex. rezolvată); POST /api/alarms/bulk-update – actualizare în masă.

Jurnale.

- GET /api/logs – jurnalul de detecții, cu filtre; GET /api/logs/vehicle – jurnalul vehiculelor.
- GET /api/logs/stats, /api/logs/timeseries, /api/logs/breakdown – agregări pentru grafice.
- GET /api/logs/export – export CSV al jurnalului filtrat.

Statistici.

- GET /api/stats – metrice agregate la nivel de sistem.

Utilizatori.

- GET /api/users; POST /api/users; PUT | DELETE /api/users/{user_id}.
- PUT /api/users/me/password – schimbarea parolei contului propriu.

Tabelul 3.2 sintetizează catalogul complet, indicând pentru fiecare endpoint metoda HTTP, calea, rolul minim necesar – **Public** (fără autentificare), **Autentificat** (orice utilizator cu token JWT valid) sau **Admin** (rol privilegiat) – și cerința funcțională **Fx** pe care o deservește. Distincția de rol este impusă la nivel de backend prin dependențele FastAPI `get_current_user` (autentificare) și `require_admin` (autorizare de tip administrator).

Tabel 3.2: Catalogul endpoint-urilor REST: metodă HTTP, cale, rol minim necesar și cerința funcțională acoperită.

Metodă	Cale	Rol minim	Cerință
Autentificare			
POST	/api/login	Public	–
POST	/api/logout	Autentificat	–
Stare și control fluxuri video (runtime)			
GET	/api/status	Autentificat	F1
POST	/api/streams	Admin	F1
GET	/api/streams	Autentificat	F1
DELETE	/api/streams/{camera_id}	Admin	F1
DELETE	/api/streams	Admin	F1
Control module AI (toggle)			
POST	/api/face/toggle	Admin	F7bis
POST	/api/plate/toggle	Admin	F7bis
POST	/api/demographics/toggle	Admin	F7bis
POST	/api/fire/toggle	Admin	F7bis
POST	/api/har/toggle	Admin	F7bis
POST	/api/weapon/toggle	Admin	F7bis
GET	/api/har/stats	Autentificat	F6
GET	/api/weapon/stats	Autentificat	F5
Persoane			
POST	/api/persons/register	Admin	F8
GET	/api/persons	Admin	F8
GET	/api/persons/departments	Admin	F8
GET	/api/persons/{person_id}	Admin	F8
PUT	/api/persons/{person_id}	Admin	F8
DELETE	/api/persons/{person_id}	Admin	F8
POST	/api/persons/{person_id}/faces	Admin	F2
GET	/api/persons/{person_id}/access-history	Admin	F9
Plăcuțe de înmatriculare			
POST	/api/plates/register	Admin	F8
GET	/api/plates	Admin	F8
GET	/api/plates/vehicle-types	Admin	F8
GET	/api/plates/{plate_number}	Admin	F8
PUT	/api/plates/{plate_number}	Admin	F8
DELETE	/api/plates/{plate_number}	Admin	F8
GET	/api/plates/{plate_number}/access-history	Admin	F9
Zone			
GET	/api/zones	Autentificat	F8
POST	/api/zones	Admin	F8
PUT	/api/zones/{zone_id}	Admin	F8
DELETE	/api/zones/{zone_id}	Admin	F8
Camere (persistate în baza de date)			
GET	/api/cameras	Autentificat	F8
POST	/api/cameras	Admin	F8
PUT	/api/cameras/{camera_id}	Admin	F8
DELETE	/api/cameras/{camera_id}	Admin	F8
Alarmer			
GET	/api/alarms	Autentificat	F7
GET	/api/alarms/stats	Autentificat	F7, F10
GET	/api/alarms/{alarm_id}	Autentificat	F7
PATCH	/api/alarms/{alarm_id}	Autentificat	F7
POST	/api/alarms/bulk-update	Autentificat	F7
Jurnale			

continuă pe pagina următoare...

... continuare a Tabelului 3.2

Metodă	Cale	Rol minim	Cerință
GET	/api/logs	Autentificat	F9
GET	/api/logs/vehicle	Autentificat	F9
GET	/api/logs/stats	Autentificat	F10
GET	/api/logs/timeseries	Autentificat	F10
GET	/api/logs/breakdown	Autentificat	F10
GET	/api/logs/export	Autentificat	F9
Statistici			
GET	/api/stats	Autentificat	F10
Utilizatori			
GET	/api/users	Admin	F8
POST	/api/users	Admin	F8
PUT	/api/users/{user_id}	Admin	F8
DELETE	/api/users/{user_id}	Admin	F8
PUT	/api/users/me/password	Autentificat	F8

3.5.3 Structura răspunsurilor

Răspunsurile sunt codificate JSON. Pentru operațiile de comandă (POST, PUT, PATCH, DELETE), *payload*-ul utilizează o structură standardizată, compusă dintr-un câmp `status` (cu valorile "success", "info" sau "error"), însoțit de un câmp `message` explicativ:

```
{ "status": "success", "message": "Camera started" }
```

Răspunsul de autentificare adaugă token-ul și profilul utilizatorului:

```
{ "status": "success",
  "token": "<JWT>",
  "user": { "id": 1, "username": "admin",
            "role": "admin", "full_name": "..." } }
```

Pentru operațiile de citire (GET), răspunsul constă fie direct în obiectul sau colecția solicitată, fie în aceeași structură standardizată a *payload*-ului, care integrează datele cerute. Un detaliu relevant de implementare este faptul că toate răspunsurile sunt procesate prin intermediul funcției `convert_to_serializable`. Aceasta convertește tipurile de date NumPy (`np.integer`, `np.floating`, `np.ndarray`), des întâlnite în ieșirile modelelor AI, în tipuri native Python compatibile cu formatul JSON, prevenind astfel erorile de serializare.

3.5.4 Coduri de eroare

API-ul folosește codurile de stare HTTP standard, ridicate prin `HTTPException` din FastAPI:

- **200 OK** – succesul operației (implicit în FastAPI).
- **401 Unauthorized** – credențiale invalide la login ("Invalid username or password"), token expirat ("Token has expired") sau token invalid ("Invalid token").

- **403 Forbidden** – utilizator autentificat dar fără rolul necesar; ridicat de dependența `require_admin` cu mesajul "Admin access required" (vezi §3.8).
- **404 Not Found** – resursă inexistentă (persoană, plăcuță, zonă, alarmă cu ID inexistent).
- **422 Unprocessable Entity** – generat automat de FastAPI/Pydantic la validarea corpului cererii (câmpuri lipsă sau de tip greșit, conform modelelor `LoginRequest`, `BulkUpdateAlarmsRequest` etc.).
- **500 Internal Server Error** – excepții neașteptate (ex. eșecul deschiderii unei camere), capturate în handler și raportate cu `detail=str(e)`.

Corpul unei erori urmează formatul standard FastAPI, `{"detail": "<mesaj>"}`, ceea ce permite frontend-ului să afișeze direct mesajul în interfața.

3.6 Proiectarea canalului WebSocket

Dacă API-ul REST (§3.5) servește cereri punctuale, canalul WebSocket este responsabil de fluxul continuu de date în timp real: cadrele video procesate, adnotate cu rezultatele detectoarelor, sunt împinse de la server către toți clienții conectați. Această secțiune descrie endpoint-ul dedicat, formatul mesajelor (verificat direct în implementarea din `app.py`), cadența de transmitere și modul în care sunt tratate deconectarea și reconectarea.

3.6.1 Endpoint dedicat pentru streaming

Există un singur endpoint WebSocket, montat la `/ws`. La conectare, serverul autentifică mai întâi clientul – token-ul JWT este transmis ca parametru de interogare (`/ws?token=...`, deoarece browserele nu pot seta antete pe un WebSocket), este decodat și re-verificat față de baza de date, iar conexiunea este refuzată (cod de închidere 1008) dacă token-ul lipsește, este invalid/expirat sau aparține unui cont șters. Doar după validare serverul acceptă handshake-ul (`await websocket.accept()`) și alocă fiecărui client o **coadă proprie** (`queue.Queue`), înregistrată în dicționarul `client_queues`. Modelul este **broadcast unu-la-mulți**: firul de fuziune (§3.4.1) nu publică într-o coadă partajată, ci difuzează fiecare cadru, prin metoda `_broadcast_frame`, în coada fiecărui client; bucla fiecărui client consumă din coada sa și trimite mesajul mai departe. Comunicarea este predominant unidirecțională, server→client; clientul nu trebuie să trimită nimic pentru a primi fluxul.

3.6.2 Formatul mesajelor

Fiecare mesaj este un obiect JSON serializat cu `json.dumps` și trimis prin `send_text`. Mesajul de tip cadru video are următoarea structură (construită în metoda `_encode_and_queue_frame`):


```
{
  "type": "video_frame",
  "camera_id": "CAM-01",
  "frame": "<JPEG codificat Base64>",
  "face_results": [ ... ],
  "plate_results": [ ... ],
  "fire_results": [ ... ],
  "har_results": [ ... ],
  "weapon_results": [ ... ],
  "timestamp": "2026-05-20T12:34:56.789",
  "demographics_enabled": true,
  "fire_detection_enabled": true,
  "har_enabled": true,
  "weapon_detection_enabled": true
}
```

Câmpul `frame` conține cadrul adnotat (cu bounding box-uri și etichete deja desenate de modulul de fuziune), codificat **JPEG la calitate 75** și apoi **Base64**. Pentru a reduce volumul transmis, cadrul este redimensionat la o lățime maximă de 800 px înainte de codificare. Trebuie precizat că rezultatele detecțiilor nu sunt grupate într-un singur câmp `detections`, ci în **câmpuri separate per detector** (`face_results`, `plate_results`, `fire_results`, `har_results`, `weapon_results`); fiecare este o listă de obiecte cu structură specifică detectorului (de exemplu, un rezultat facial conține `name`, `confidence`, `bbox`, `age`, `gender`, `emotion`). Câmpurile booleene `*_enabled` comunică frontend-ului ce module sunt active, astfel încât interfața să poată ascunde/afișa straturile corespunzătoare. Toate valorile trec prin `convert_to_serializable` pentru a transforma tipurile NumPy în tipuri native serializabile JSON.

Alarmele nu sunt transmise printr-un tip de mesaj WebSocket separat; ele sunt stocate în baza de date și consultate de client prin endpoint-urile REST `/api/alarms` (§3.5). Captura asociată unei alarme (snapshot) este codificată tot JPEG+Base64, la calitate 60, în momentul declanșării.

Ciclul de viață complet al unei conexiuni este ilustrat în Figura 3.7: de la autentificare și handshake, prin alocarea cozii proprii și bucla de transmitere la ~30 FPS, până la deconectare și eliminarea cozii din dicționar.

3.6.3 Cadența de transmitere

Bucla de transmitere a fiecărui client consumă din *propria coadă* în mod ne-blocant (`get(block=False)`): dacă există un mesaj, îl trimite (`send_text(json.dumps(...))`); dacă nu, prinde `queue.Empty` și continuă. După fiecare iterație, bucla execută `await asyncio.sleep(0.03)`, ceea ce stabilește o cadență țintă de **aproximativ 30 FPS** (consistentă cu cerința NF1) și, în același timp, cedează controlul buclei de evenimente asincrone, permițând serverului să deservească în paralel ceilalți clienți și cererile REST.

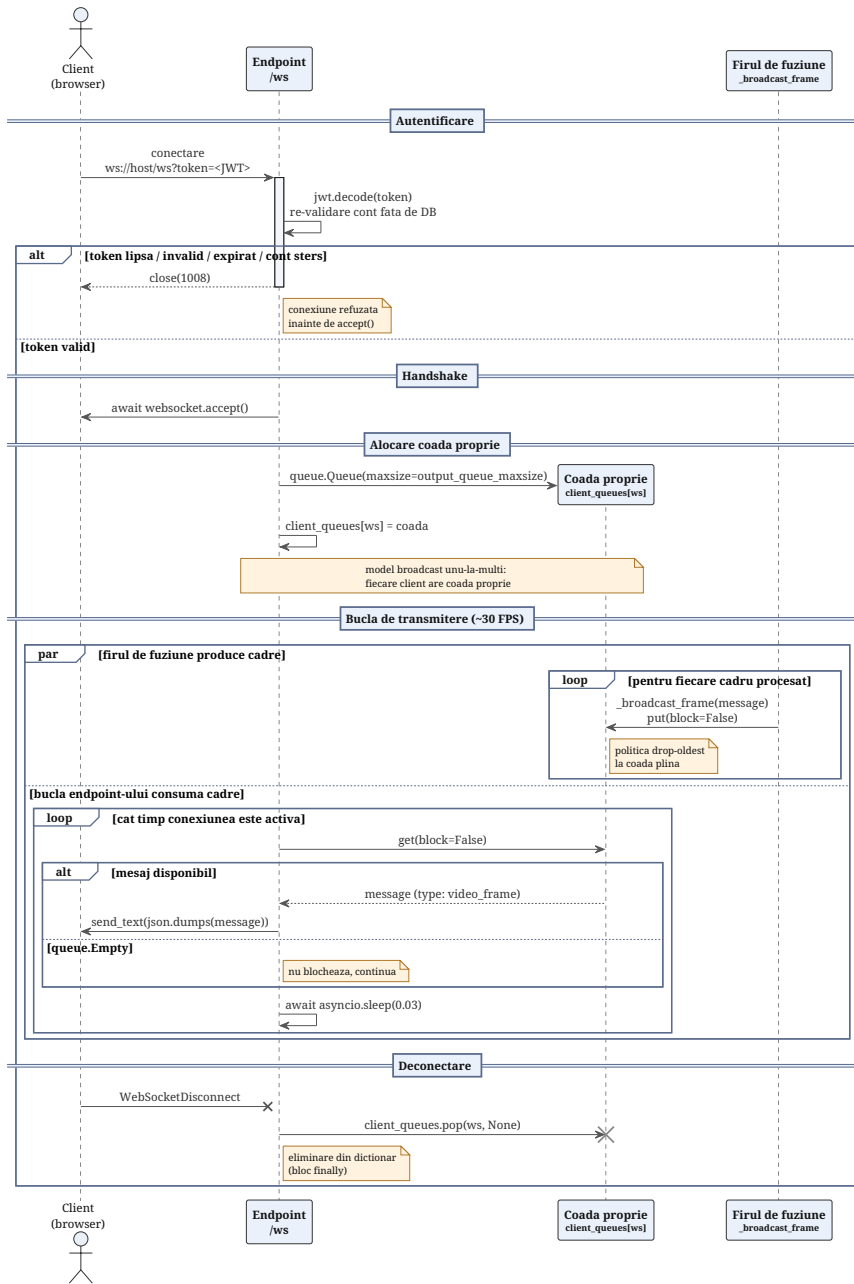


Figura 3.7: Diagramă de secvență UML a ciclului de viață al unei conexiuni WebSocket: autentificare (validarea token-ului JWT, cu închidere 1008 la eșec), handshake (accept()), alocarea cozii proprii în `client_queues`, bucla de transmitere la ~30 FPS (firul de fuziune produce prin `_broadcast_frame`, endpoint-ul consumă neblocaant și trimite mesajele `video_frame`) și deconectarea, cu eliminarea cozii din dicționar.

3.6.4 Heartbeat și detectarea deconectării

Implementarea curentă nu folosește un heartbeat la nivel de aplicație (ping/pong explicit cu mesaje dedicate). Detectarea deconectării se bazează pe două mecanisme: (i) excepția `WebSocketDisconnect`, ridicată de FastAPI/Starlette atunci când clientul închide conexiunea, prinsă explicit în handler; și (ii) fluxul continuu de cadre, care joacă rolul unui heartbeat implicit – absența mesajelor pentru o perioadă, combinată cu eroarea la `send_text`, declanșează ieșirea din buclă. La nivel de transport, protocolul WebSocket peste TCP dispune de propriul mecanism de ping/pong de control, gestionat de bibliotecă, pe care implementarea se bazează pentru menținerea conexiunii. Indiferent de cauză, blocul `finally` asigură eliminarea cozii clientului din `client_queues` (sub protecția `client_lock`), prevenind acumularea de conexiuni moarte.

3.6.5 Reconectare

Reconectarea este responsabilitatea clientului, nu a serverului – serverul tratează fiecare conexiune ca efemeră și nu păstrează stare per client între conexiuni (este *stateless* din acest punct de vedere). La pierderea conexiunii, clientul React reia handshake-ul către `/ws`; deoarece serverul difuzează mereu cel mai recent cadru (mecanismul *ultimul cadru disponibil* din §4.8.2), un client reconectat își reia fluxul imediat, fără a fi nevoie de resincronizare sau de recuperarea cadrelor pierdute. Această alegere – de a nu garanta livrarea fiecărui cadru – este corectă pentru un sistem de supraveghere live, unde un cadru pierdut este irelevant atâta timp cât cel curent este actual.

3.7 Proiectarea bazei de date

Baza de date reprezintă nucleul central de stocare al sistemului. Toate informațiile care trebuie să supraviețuiască unei reporniri (utilizatori, persoane, reprezentări vectoriale faciale, plăcuțe, zone, camere, alarme și jurnale) sunt stocate aici, în timp ce structurile volatile (precum indexul FAISS și *cache*-ul de zone) sunt reconstruite pe baza acestor date la fiecare inițializare. Această secțiune prezintă schema reală implementată în `database_manager.py`, detaliind relațiile dintre tabele, cheile, indecșii, precum și motivarea alegerii acestui sistem de gestiune a bazelor de date.

3.7.1 Alegerea sistemului de baze de date

Sistemul folosește **PostgreSQL**, nu SQLite. Decizia este motivată de mai multe trăsături folosite efectiv în schemă, indisponibile sau slabe în SQLite:

- tipul **tablou** (`TEXT[]`) pentru câmpul `authorized_zones` al persoanelor;
- tipul **JSONB** pentru metadatele de detecție (`detection_metadata, details`), cu interogare și indexare nativă;

- tipul `BYTEA` pentru stocarea reprezentărilor vectoriale faciale binare;
- **concurența reală** la scriere: aspect esențial, justificat de faptul că pipeline-ul multi-threaded (§3.4) scrie alarme și jurnale din firul de fuziune în paralel cu apelurile REST ale utilizatorilor. În această arhitectură, blocarea exclusivă a fișierului specifică SQLite ar deveni un factor limitativ critic pentru întregul sistem.

Conexiunea este configurată în `DB_CONFIG` către baza `facial_recognition` de pe `localhost`, cu credențiale dedicate (cerința NF3).

3.7.2 Diagrama entitate-relație

Figura 3.8 prezintă diagrama entitate-relație a schemei. Cheile primare surogat (`id`) sunt subliniate, iar cheile străine sunt marcate prin (FK) și legate de tabelul referit printr-o linie continuă, etichetată cu politica `ON DELETE` corespunzătoare. Cele patru relații impuse prin chei străine sunt de tip 1:N (notație *crow's-foot*): trei dintre acestea cu `ON DELETE CASCADE` (`face_embeddings` și `access_logs` către `persons`, respectiv `vehicle_access_logs` către `license_plates`), iar una cu `ON DELETE SET NULL` (`cameras` către `zones`). Liniile punctate marchează legăturile menținute exclusiv la nivel de aplicație, fără chei străine: cele patru câmpuri `camera_id` (de tip `VARCHAR`) către `cameras` și, cel mai important, relația dintre persoană și zonă. Aceasta din urmă *nu* trece printr-un tabel de joncțiune, ci este denormalizată sub forma unui atribut multi-valoare, `authorized_zones TEXT[]`, definit direct în tabelul `persons`, unde zonele sunt referite prin nume, nu prin constrângeri de tip FK.

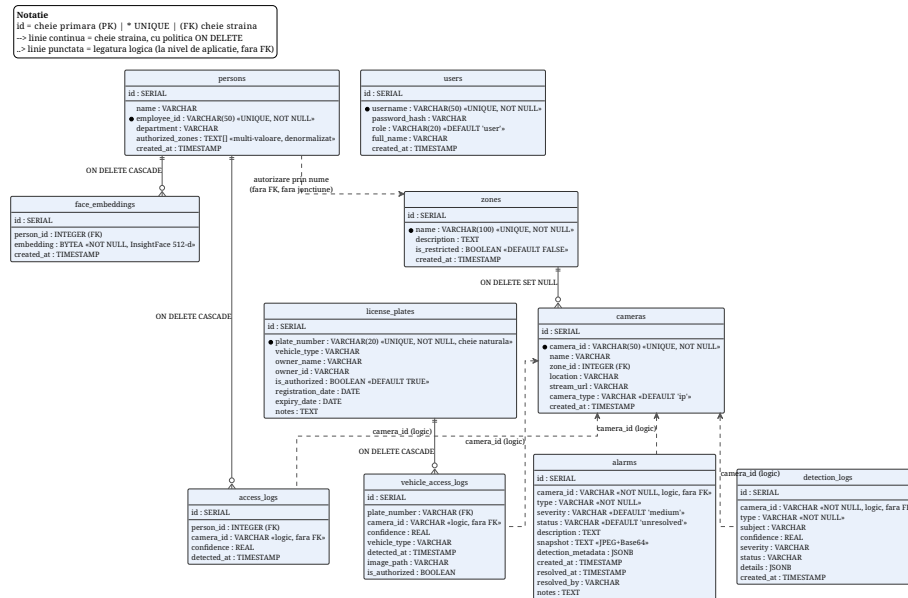


Figura 3.8: Diagrama entitate-relație (ERD) a schemei PostgreSQL `facial_recognition`, cu cele zece tabele implementate în `database_manager.py`. Cheile primare (id) sunt subliniate, iar cele cinci constrângeri UNIQUE sunt marcate cu asterisc. Liniile punctate evidențiază legăturile logice neimpuse prin FK: cele patru câmpuri `camera_id` către `cameras` și relația persoană–zonă, denormalizată ca atribut multi-valoare `authorized_zones TEXT[]` pe `persons` (nu ca entitate asociativă). Tabelul `users` este independent.

3.7.3 Tabelele principale

Schema reală conține zece tabele. Pentru a menține o concordanță cu implementarea, în continuare sunt utilizate denumirile exacte din modulul `database_manager.py`. Orice diferență între aceste denumiri tehnice și terminologia generică va fi semnalată explicit.

persons – persoanele înregistrate. `id` SERIAL PK; `name`; `employee_id` (UNIQUE NOT NULL); `department`; `authorized_zones` TEXT[]; `created_at`. *Observație de proiectare:* autorizarea pe zone este denormalizată ca tablou de nume de zone direct pe persoană, nu printr-un tabel de joncțiune `person_zone_authorization` separat. Compromisul: o citire foarte rapidă a zonelor alocate unei persoane (operațiune frecventă în , datele fiind reținute în memoria cache `_person_zones_cache`), în schimbul pierderii integrității referențiale către `zones` (zonele sunt referite prin nume, nu prin chei externe / FK).

face_embeddings – corespondentul *person_images* din cerință, dar stochează reprezentări vectoriale, nu imagini. `id` SERIAL PK; `person_id` INTEGER REFERENCES `persons(id)` ON DELETE CASCADE; `embedding` BYTEA NOT NULL (vectorul InsightFace 512-d serializat binar); `created_at`. Relația cu `persons` este 1:N (o persoană poate avea mai multe reprezentări vectoriale, câte una per imagine înregistrată), iar ON DELETE CASCADE asigură ștergerea reprezentărilor vectoriale odată cu persoana. Acest tabel este sursa din care se reconstruiește indexul FAISS la pornire.

access_logs – jurnalul accesărilor de persoane. `id` SERIAL PK; `person_id` REFERENCES `persons(id)` ON DELETE CASCADE; `camera_id`; `confidence`; `detected_at`.

license_plates – corespondentul *plates*. `id` SERIAL PK; `plate_number` VARCHAR(20) UNIQUE NOT NULL; `vehicle_type`; `owner_name`; `owner_id`; `is_authorized` BOOLEAN DEFAULT TRUE; `registration_date`; `expiry_date`; `notes`. Cheia naturală `plate_number` este folosită ca țintă de FK de către jurnalul vehiculelor.

vehicle_access_logs – jurnalul accesărilor de vehicule. `id` SERIAL PK; `plate_number` cu FOREIGN KEY (`plate_number`) REFERENCES `license_plates(plate_number)` ON DELETE CASCADE; `camera_id`; `confidence`; `vehicle_type`; `detected_at`; `image_path`; `is_authorized`.

users – conturile de utilizator. `id` SERIAL PK; `username` UNIQUE NOT NULL; `password_hash`; `role` VARCHAR(20) DEFAULT 'user'; `full_name`; `created_at`. Parola este stocată exclusiv ca hash (cerința NF3), iar role alimentează controlul de acces (§3.8).

alarms – corespondentul *alerts*. `id` SERIAL PK; `camera_id` NOT NULL; `type` NOT NULL; `severity` DEFAULT 'medium'; `status` DEFAULT 'unresolved'; `description`; `snapshot` (captura JPEG+Base64 la momentul declanșării);

detection_metadata JSONB; created_at; resolved_at; resolved_by; notes. Câmpurile resolved_* susțin ciclul de viață al alarmei (UC11).

zones – zonele de supraveghere. id SERIAL PK; name UNIQUE NOT NULL; description; is_restricted BOOLEAN DEFAULT FALSE; created_at.

cameras – registrul persistent de camere. id SERIAL PK; camera_id UNIQUE NOT NULL; name; zone_id INTEGER REFERENCES zones(id) ON DELETE SET NULL; location; stream_url; camera_type DEFAULT 'ip'; created_at. Politica ON DELETE SET NULL pe zone_id face ca ștergerea unei zone să lase camera fără zonă, nu să o șteargă.

detection_logs – corespondentul logs; istoricul complet al fiecărei detecții. id SERIAL PK; camera_id NOT NULL; type NOT NULL; subject; confidence; severity; status; details JSONB; created_at. Acesta este tabelul interogată de /api/logs și exportat ca CSV.

3.7.4 Chei primare și străine

Toate tabelele folosesc o cheie primară surogat id SERIAL (întreg auto-incrementat). Pe lângă cheie primare, schema definește următoarele **chei străine** și politici de ștergere:

- face_embeddings.person_id → persons.id, ON DELETE CASCADE;
- access_logs.person_id → persons.id, ON DELETE CASCADE;
- vehicle_access_logs.plate_number → license_plates.plate_number, ON DELETE CASCADE;
- cameras.zone_id → zones.id, ON DELETE SET NULL.

Constrângeri de unicitate (UNIQUE) protejează persons.employee_id, license_plates.plate_number, users.username, zones.name și cameras.camera_id. *Notă:* legătura logică persoană-zonă și cea alarmă/jurnal-cameră (camera_id este VARCHAR, nu FK către cameras) nu sunt impuse prin chei străine, ci la nivel de aplicație – o decizie de proiectare care privilegiază viteza de scriere a jurnalelor în detrimentul integrității referențiale stricte.

3.7.5 Indexarea

Pentru a susține interogările frecvente (filtrare de jurnale, listare de alarme, căutare de plăcuțe), schema definește următorii indecși secundari:

- idx_plate_number pe license_plates(plate_number);
- idx_detlogs_created_at pe detection_logs(created_at DESC), idx_detlogs_type pe (type), idx_detlogs_camera pe (camera_id) – acoperă filtrele paginii de jurnale (interval de timp, tip, cameră);

- `idx_vehicle_access_time` pe `vehicle_access_logs(detected_at)`, `idx_vehicle_camera` pe `(camera_id)`;
- `idx_alarms_status`, `idx_alarms_type`, `idx_alarms_severity`, `idx_alarms_created_at` pe `alarms` – acoperă filtrarea și sortarea panoului de alarme;
- `idx_cameras_zone` pe `cameras(zone_id)`.

Indexul pe `created_at` DESC al jurnalului de detectii este deosebit de relevant: cele mai multe interogări afișează evenimentele recente în ordine descrescătoare, iar indexul descendent elimină nevoia unei sortări la fiecare cerere.

3.8 Proiectarea sistemului de control al accesului

Controlul accesului protejează operațiile sensibile ale sistemului împotriva utilizatorilor neautorizați și implementează cerința NF3. El are trei componente: un model RBAC cu două roluri, un mecanism de sesiune fără stare bazat pe JWT (cu revocare prin listă neagră) și o stocare sigură a parolelor. Toate sunt prezentate mai jos exact așa cum sunt implementate în `app.py` și `database_manager.py`, inclusiv singura limitare reziduală rămasă.

3.8.1 Modelul RBAC

Sistemul folosește un model **Role-Based Access Control** minimal, cu două roluri stocate în coloana `users.role`:

- **admin** – acces complet: configurare, gestiunea tuturor entităților, activarea/dezactivarea modulelor AI;
- **user** – rolul implicit (DEFAULT 'user'): acces operațional la vizualizare, alarme, jurnale și statistici, fără drept de configurare.

Aplicarea rolurilor se face prin sistemul de dependențe FastAPI. Două funcții-dependență stau la baza protecției:

- `get_current_user` – decodează și validează token-ul JWT din antetul `Authorization: Bearer` și, suplimentar, *re-verifică utilizatorul față de baza de date* (prin `get_user_by_id`): un cont șters este respins cu 401 "User no longer exists", iar rolul curent din DB este preferat în locul celui din token (posibil învechit). Orice endpoint care o folosește cere doar ca utilizatorul să fie *autentificat* (oricare rol).
- `require_admin` – compusă peste `get_current_user`, verifică suplimentar `role == "admin"` și, în caz contrar, ridică `HTTPException 403` cu mesajul "Admin access required".

Astfel, nivelul de permisiune al unui endpoint este declarat la definirea lui, prin tipul dependenței injectate (`Depends(require_admin)` vs. `Depends(get_current_user)`).

3.8.2 Matricea de permisiuni

Tabelul 3.3 rezumă, pe module, ce poate face fiecare rol, conform dependențelor injectate efectiv în cod.

Tabel 3.3: Matricea de permisiuni pe module și roluri.

Modul / operație	admin	user	ne-autentificat
Autentificare (/api/login)	✓	✓	✓
Gestiune persoane (CRUD, imagini, istoric)	✓	—	—
Gestiune plăcuțe (CRUD, istoric)	✓	—	—
Gestiune utilizatori (creare/modificare/ștergere)	✓	—	—
Toggle module AI (face/plate/fire/HAR/weapon)	✓	—	—
Creare/modificare/ștergere zone	✓	—	—
Creare/modificare/ștergere camere (DB)	✓	—	—
Control fluxuri video runtime (pornire/oprire, /api/streams)	✓	—	—
Stare sistem (/api/status) / listare camere active	✓	✓	—
Listare zone / camere (citire)	✓	✓	—
Statistici interne detectoare (/api/har/stats, /api/weapon/stats)	✓	✓	—
Vizualizare jurnale + export CSV (inclusiv /api/logs/vehicle)	✓	✓	—
Vizualizare + gestiune alarme (rezolvare)	✓	✓	—
Statistici (/api/stats)	✓	✓	—
Schimbare parolă cont propriu	✓	✓	—
Logout / revocare token propriu (/api/logout)	✓	✓	—

Acoperirea completă a endpoint-urilor. Toate rutele /api (cu excepția firească a /api/login) au o dependență de securitate injectată: operațiile cu efect – controlul fluxurilor video în timp real (POST /api/streams, DELETE /api/streams, DELETE /api/streams/{camera_id}) – necesită Depends(require_admin), iar endpoint-urile de citire operațională (/api/status, GET /api/streams, /api/har/stats, /api/weapon/stats, /api/logs/vehicle) necesită Depends(get_current_user). Astfel, nicio operație nu poate fi efectuată fără autentificare prealabilă.

3.8.3 Hashing-ul parolelor

Parolele nu sunt stocate niciodată în format clar (*plaintext*). Mecanismul de securizare utilizează algoritmul **bcrypt** (prin intermediul bibliotecii **bcrypt** din

Python). La crearea sau actualizarea unui cont, parola este procesată prin funcția `bcrypt.hashpw(password.encode(), bcrypt.gensalt())`. Rezultatul generat, care include valoarea aleatoare de tip *salt* și factorul de cost încorporat în șir, este salvat în coloana `users.password_hash VARCHAR(255)`. La autentificare, validarea se realizează prin `bcrypt.checkpw(password.encode(), password_hash.encode())` în cadrul metodei `authenticate_user`, care returnează datele utilizatorului *fără* câmpul `password_hash`. Utilizarea funcției `gensalt()` garantează un *salt* unic pentru fiecare parolă, în timp ce costul adaptiv al algoritmului `bcrypt` asigură rezistența împotriva atacurilor de tip *brute-force* și *rainbow table*.

3.8.4 Gestiunea sesiunii (JWT)

Sesiunile sunt fără stare (*stateless*): după autentificarea reușită, serverul emite un JSON Web Token semnat, pe care clientul îl atașează fiecărei cereri ulterioare în antetul `Authorization: Bearer <token>`; serverul nu păstrează o stare de sesiune *pozitivă* în memorie sau în baza de date. Singura stare reținută este o listă neagră de token-uri revocate (tabela `revoked_tokens`, detaliată mai jos), consultată la fiecare cerere pentru a permite delogarea efectivă înainte de expirare.

Parametrii concreți ai token-ului, din `app.py`:

- **Algoritm:** HS256 (HMAC-SHA256, semnătură simetrică).
- **Durată de viață:** 8 ore (`JWT_EXPIRATION_HOURS = 8`), prin câmpul `exp`.
- **Payload:** sub (`username`), `user_id`, `role`, `full_name`, `jti` (identificator unic de token, generat cu `uuid4` și folosit pentru revocare), `exp`, `iat`. Includerea lui `role` în token permite verificarea rolului fără o interogare suplimentară în baza de date la fiecare cerere.
- **Validare:** la decodare, `jwt.ExpiredSignatureError` produce 401 "Token has expired", iar `jwt.InvalidTokenError` produce 401 "Invalid token".

Cheia de semnare. Cheia de semnare (`JWT_SECRET`) este externalizată în mediul de execuție (variabila de mediu `JWT_SECRET`). Comportamentul sistemului în cazul absenței acesteia depinde de mediul de rulare, definit de variabila `APP_ENV`: în **producție** (`APP_ENV=production`), o cheie absentă sau mai scurtă de 32 de caractere oprește explicit pornirea aplicației (`RuntimeError, fail-fast`), prevenind emiterea de token-uri cu o cheie efemeră; în **dezvoltare**, sistemul recurge la o cheie aleatorie per-proces și doar avertizează la pornire. Astfel, capcana silențioasă prin care un deployment real ar fi putut rula pe o cheie temporară (cu invalidarea token-urilor la fiecare repornire) este eliminată.

Revocarea sesiunilor. Deși token-ul în sine este *stateless*, sistemul oferă revocare instantanee printr-o listă neagră (tabela `revoked_tokens`, cu cheie `jti`). Fiecare token primește la emitere un identificator unic `jti` (`uuid4`);

endpoint-ul `POST /api/logout` înscrie `jti`-ul curent în listă, iar dependența `get_current_user` respinge cu 401 "Token has been revoked" orice token revocat – verificare aplicată și la deschiderea canalului WebSocket (închidere 1008). Pe lângă aceasta, `get_current_user` re-validează la fiecare cerere identitatea față de baza de date, astfel încât ștergerea unui cont și schimbarea de rol au efect imediat. Intrările expirate din listă nu mai au efect și sunt curățate periodic (`purge_expired_revocations`), astfel încât dimensiunea tabelului rămâne mărginită.

Limitare reziduală. Lista neagră revocă token-uri *individuale* (de exemplu sesiunea curentă, la logout, sau un token compromis). Ea nu invalidează automat *toate* sesiunile active ale unui utilizator la schimbarea parolei, întrucât serverul nu cunoaște `jti`-urile token-urilor încă neprezentate. Scenariul “delogare de pe toate dispozitivele” ar necesita o extindere suplimentară – de exemplu o coloană `users.tokens_valid_after` comparată cu `iat` la fiecare cerere. Această limitare reziduală este consemnată ca parte a analizei critice a cerinței NF3.

3.9 Proiectarea logicii de alarmare

Logica de alarmare este punctul în care detecțiile brute ale modulelor AI devin evenimente acționabile pentru operator. Ea este implementată centralizat în firul de fuziune (§3.4.1), după ce toate rezultatele unui cadru au fost agregate, și are trei responsabilități: (i) evaluarea regulilor de declanșare, (ii) atribuirea unei severități, (iii) persistarea și difuzarea alarmei cu deduplicare. Această secțiune descrie regulile exact așa cum sunt implementate în `app.py`.

3.9.1 Modelul evenimentelor: detecție, jurnal și alarmă

Înainte de a detalia regulile, este utilă o delimitare clară a noțiunilor cu care operează sistemul, întrucât ele sunt ușor de confundat, deși au roluri, niveluri de persistență și cicluri de viață diferite. Figura 3.9 sintetizează relațiile dintre ele.

Detecția este rezultatul brut produs de un modul AI pentru *un singur cadru*: o clasă, un nivel de certitudine (scor de încredere) și, după caz, o casetă de încadrare (*bounding box*). Ea este *volatilă* – există exclusiv pe cadrul adnotat difuzat în fluxul live și nu este stocată ca atare. Marea majoritate a detecțiilor (de exemplu, o față cunoscută sau un cadru fără evenimente) nu generează înregistrări persistente.

Jurnalul (tabelul `detection_logs`) reprezintă înregistrarea *pasivă* a unei detecții considerate relevante. Acesta constituie istoricul interogabil și exportabil în format CSV al sistemului – documentează *ce s-a întâmplat* și *când*, fără a impune vreo acțiune operatorului. Pentru a preveni încălcarea istoricului cu date redundante, jurnalizarea este deduplicată pe o fereastră scurtă, de 5 secunde, utilizând cheia (*cameră, tip, subiect*).

Alarma (tabelul `alarms`) constituie *escaladarea* unui subset de detecții considerate critice (persoană necunoscută, foc/fum confirmat, armă, acțiune periculoasă, vehicul neautorizat). Spre deosebire de jurnal, alarma solicită *atenția imediată* a operatorului: este însoțită de un nivel de severitate și de o captură de imagine a cadrului (*snapshot*), este afișată în panoul dedicat și este deduplicată pe o fereastră mai lungă, de 30 secunde (§3.9.5). Orice alarmă este inițializată implicit în starea `unresolved`.

Rezolvarea încheie ciclul de viață al alarmei. Operatorul o triază (UC11) și o marchează într-una dintre cele două stări terminale: `resolved` – *alarmă rezolvată*, corespunzând unui incident real care a fost tratat – sau `false_alarm` – *alarmă falsă*, atunci când evenimentul semnalat nu corespunde unei situații reale (un fals pozitiv al detectorului sau o declanșare lipsită de relevanță operațională). În ambele cazuri, se înregistrează utilizatorul care a acționat (`resolved_by`) și momentul deciziei (`resolved_at`), moment în care alarma este eliminată din lista alertelor active.

Distincția esențială este aceea că *jurnalul* și *alarma* funcționează ca două canale *paralele*, nu unul derivat din celălalt: orice detecție relevantă este jurnalizată, în timp ce doar subsetul critic generează, *în mod suplimentar*, o alarmă. Modulul de recunoaștere a plăcuțelor de înmatriculare ilustrează clar această dualitate – o citire OCR validă este întotdeauna jurnalizată cu starea `authorized` sau `unauthorized`, dar declanșează o alarmă *exclusiv* dacă plăcuța este neautorizată. Restul secțiunii detaliază regulile specifice prin care o detecție este promovată la stadiul de alarmă.

3.9.2 Reguli de declanșare

O alarmă este creată prin metoda `_create_alarm_if_needed(camera_id, alarm_type, severity, description, snapshot, metadata, dedup_subject)`. Regulile efective sunt următoarele:

Persoană necunoscută (tip face). Pentru orice rezultat facial cu `name == "Unknown"`, indiferent de zonă, se creează o alarmă de tip `face` cu severitate `critical` și descrierea `"Unauthorized person detected (...%)"`.

Persoană în zonă restricționată (tip `unauthorized_zone`). Verificarea suplimentară `_check_zone_authorization` se aplică doar dacă zona camerei are `is_restricted = TRUE`. În acest caz se creează o alarmă `unauthorized_zone` cu severitate `critical` în două situații: (a) *persoană necunoscută* în zona restricționată (`"Unknown person in restricted zone '...'"`), sau (b) *persoană cunoscută dar neautorizată* – recunoscută, dar al cărei `employee_id` nu apare în `authorized_zones` ale zonei (`"<nume> (<id>) in restricted zone '...' -- not authorized"`). O persoană recunoscută și autorizată nu generează nicio alarmă.

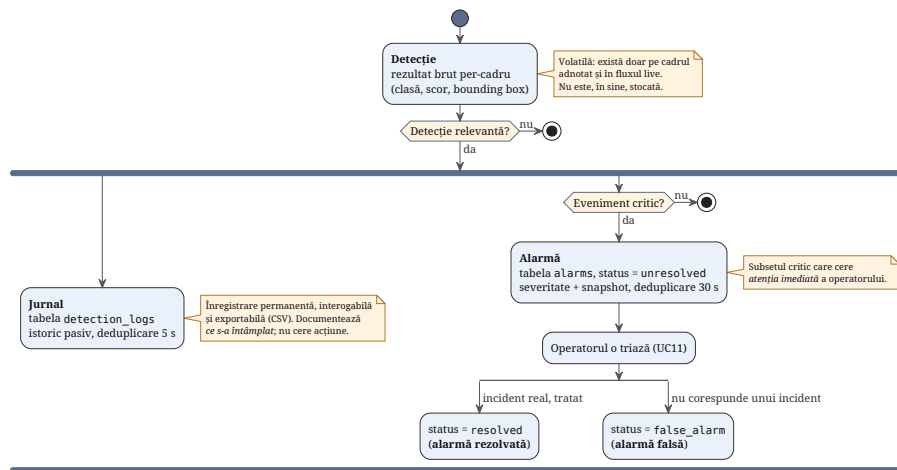


Figura 3.9: Modelul conceptual al evenimentelor și ciclul de viață al unei alarme. O *detecție* brută, volatilă, devine relevantă și este înregistrată ca *jurnal* (istoric pasiv, deduplicare 5 s); subsetul critic generează în plus o *alarmă* (stare **unresolved**, severitate, snapshot, deduplicare 30 s). Operatorul triază alarma (UC11) într-una din cele două stări terminale: **resolved** (alarmă rezolvată – incident real tratat) sau **false_alarm** (alarmă falsă). Jurnalul și alarma sunt canale *paralele*: o detecție relevantă este jurnalizată, iar dacă este și critică, este escaladată ca alarmă.

Armă detectată (tip `weapon`). Pentru orice rezultat din detectorul de arme se creează o alarmă `weapon`, cu severitatea preluată din rezultat (`r["severity"]`, implicit `critical`).

Foc/fum confirmat (tip `fire`). Se creează alarmă `fire` *doar* pentru rezultatele care au *ambele* câmpuri `confirmed` și `alert` adevărate – adică doar după confirmarea temporală pe fereastră multi-cadru (cerința F4). Severitatea este `critical` pentru clasa `fire` și `high` pentru `smoke`.

Acțiune periculoasă (tip `har`). Pentru orice rezultat HAR cu `class != "normal"` (fight, vandalism) se creează o alarmă `har`, cu severitatea preluată din rezultat (implicit `high`) și eticheta acțiunii în descriere.

Plăcuță neautorizată (tip `plate`). Pentru orice rezultat ANPR cu citire OCR validă (numărul plăcuței diferit de `None`/`""`/`"UNKNOWN"`) care *nu* este regăsit pe lista de plăcuțe autorizate, se creează o alarmă `plate` cu severitate `high` și descrierea `"Unauthorized vehicle <plate> detected (...%)"`, deduplicată pe numărul plăcuței. Rezultatele ANPR sunt în plus *jurnalizate* (prin `_log_detection_if_needed` cu tip `plate` și status `"authorized"` / `"unauthorized"`).

Listing-ul 3.1 ilustrează modul în care aceste reguli sunt evaluate în cadrul firului de fuziune. Cele cinci categorii de detecții sunt analizate independent pentru fiecare cadru, permițând astfel generarea simultană a mai multor alarme pe baza unei singure imagini. Fiecare ramură de execuție determină tipul, nivelul de severitate și subiectul de deduplicare (`dedup_subject`), delegând ulterior operațiunea de salvare persistentă către funcția comună `create_alarm_if_needed` (Listing-ul 3.2).

```

1 # Ruleaza dupa ce rezultatele unui cadru au fost agregate
2 for face in results.faces:
3     if face.name == "Unknown":
4         create_alarm_if_needed(camera_id, "face", "critical",
5                                 dedup_subject=bbox_bucket(face.bbox))
6     if camera.zone.is_restricted and not is_authorized(face, camera.zone):
7         subject = bbox_bucket(face.bbox) if face.is_unknown else
8             ↪ face.employee_id
9         create_alarm_if_needed(camera_id, "unauthorized_zone", "critical",
10                                dedup_subject=subject)
11
12 for weapon in results.weapons:
13     create_alarm_if_needed(camera_id, "weapon", weapon.severity,
14                             dedup_subject=weapon.cls)
15
16 for fire in results.fire:
17     # confirmare temporala multi-cadru
18     if fire.confirmed and fire.alert:
19         severity = "critical" if fire.cls == "fire" else "high"
20         create_alarm_if_needed(camera_id, "fire", severity,
21                                 dedup_subject=fire.cls)
22
23 for action in results.har:
24     # fight, vandalism, ...
25     if action.cls != "normal":
26         create_alarm_if_needed(camera_id, "har", action.severity,
27                                 dedup_subject=action.cls)
28
29 for plate in results.plates:
30     # fara citire OCR valida
31     if plate.text in (None, "", "UNKNOWN"):
32         continue
33     authorized = plate.text in authorized_plates
34     # placuta e si jurnalizata
35     log_detection_if_needed(camera_id, "plate",
36                             status="authorized" if authorized else
37                                 ↪ "unauthorized")
38
39 if not authorized:
40     create_alarm_if_needed(camera_id, "plate", "high",
41                             dedup_subject=plate.text)

```

Listing 3.1: Decizia de alarmare pentru un cadru, în firul de fuziune (app.py).

Notă. Spre deosebire de regula persoanelor, alarma de plăcuță este *globală* (nu depinde de zona camerei): o plăcuță este considerată neautorizată dacă numărul ei nu apare în baza de date, indiferent de cameră (vezi §3.1.1, F3 și F7).

3.9.3 Captura asociată (snapshot)

Când oricare regulă se declanșează pe un cadru, se captează un *snapshot*: cadrul anotat este redimensionat la o lățime maximă de 400 px, codificat JPEG la

calitate 60 și apoi Base64, fiind atașat alarmei (câmpul `snapshot` din tabela `alarms`). Calitatea redusă este o decizie deliberată de economisire a spațiului, întrucât snapshot-ul servește doar contextualizării vizuale a alarmei în panoul operatorului.

3.9.4 Niveluri de severitate și mapping pe tipuri de eveniment

Severitatea este un câmp text pe tabela `alarms`, cu valoarea implicită `medium` la nivel de schemă. Logica de alarmare produce două niveluri efective – `critical` și `high` –, valoarea `medium` rămânând doar ca implicit al coloanei. Tabelul 3.4 prezintă maparea tip-eveniment → severitate.

Tabel 3.4: Maparea tipurilor de eveniment pe severitate, conform implementării.

Tip alarmă	Condiție de declanșare	Severitate
<code>face</code>	persoană necunoscută (oriunde)	<code>critical</code>
<code>unauthorized_zone</code>	necunoscut/neautorizat în zonă restricționată	<code>critical</code>
<code>weapon</code>	armă detectată	<code>critical</code> (din rezultat)
<code>fire</code>	foc confirmat	<code>critical</code>
<code>fire</code>	fum confirmat	<code>high</code>
<code>har</code>	acțiune $\neq$ <code>normal</code> (fight, vandalism, ...)	<code>high</code> (din rezultat)
<code>plate</code>	plăcută neautorizată (OCR valid, negăsită)	<code>high</code>

3.9.5 Deduplicarea temporală

Pentru a nu inunda operatorul cu alarme repetate pentru același eveniment continuu, fiecare creare de alarmă trece printr-un mecanism de **cooldown**. Cheia de deduplicare este tripletul (`camera_id`, `alarm_type`, `dedup_subject`) – concatenat în forma "`camera_id:alarm_type:subiect`" – iar fereastra este `alarm_cooldown_seconds = 30`: dacă o alarmă cu aceeași cheie a fost creată în ultimele 30 de secunde, noua declanșare este ignorată. Verificarea și rezervarea slot-ului (actualizarea timestamp-ului) se fac sub `alarm_lock`, astfel încât două fire concurente să nu treacă ambele de poartă.

Componenta `dedup_subject` este cea care *distinge* detecțiile concurente de același tip, evitând să le colapseze într-o singură alarmă: persoanele necunoscute sunt diferențiate printr-un *bucket* de poziție al casetei (`_bbox_position_bucket`), persoanele recunoscute prin `employee_id`, acțiunile HAR prin clasa acțiunii, iar plăcuțele prin numărul lor. Armele folosesc drept subiect eticheta clasei returnate de detector; întrucât acesta este antrenat pe o singură clasă (`weapon`),

subiectul este în practică constant, astfel încât alarmele de armă se deduplicatează efectiv pe perechea (*cameră*, *tip*). Consecința – opusă unei deduplicări pe simpla pereche (*cameră*, *tip*) – este că două persoane necunoscute diferite, apărute simultan pe aceeași cameră, produc *două* alarme distincte, nu una. Dacă *dedup_subject* lipsește, mecanismul recade pe deduplicarea per (*cameră*, *tip*), ca mod implicit.

Pe lângă poarta din memorie (care se pierde la repornire), există un **backstop în baza de date**: înainte de a persista o alarmă, sistemul verifică prin *get_recent_alarm* dacă există deja o alarmă recentă, nerezolvată, cu aceeași cheie (inclusiv *dedup_subject*, păstrat în metadata), prevenind o rafală de duplicate imediat după ce serverul revine. Pentru comparație, jurnalizarea folosește propriul cooldown, mai scurt și pe subiect explicit (*log_cooldown_seconds* = 5).

Listing-ul 3.2 detaliază această componentă centralizată prin care este direcționat orice apel către funcția *create_alarm_if_needed* (introdusă în Listing-ul 3.1). Verificarea și rezervarea intervalului de latență (*cooldown*) se realizează în mod atomic, operațiunile fiind protejate de mecanismul de excludere mutuală *alarm_lock*. Această etapă este urmată de o verificare redundantă (*backstop*) la nivelul bazei de date, procesul finalizându-se cu înregistrarea persistentă și difuzarea alarmei către clienți.

```

1 def create_alarm_if_needed(camera_id, alarm_type, severity,
2                             description, snapshot, metadata, dedup_subject):
3     key = f"{camera_id}:{alarm_type}:{dedup_subject or ''}"
4     now = time.time()
5
6     # poarta atomica intre fire
7     with alarm_lock:
8         last = alarm_cooldowns.get(key, 0)
9         # 30 s
10        if now - last < ALARM_COOLDOWN_SECONDS:
11            # duplicat in fereastra de cooldown
12            return
13        # rezerva slotul
14        alarm_cooldowns[key] = now
15
16    # Backstop DB: cooldown-ul din memorie se pierde la repornire
17    if db.get_recent_alarm(camera_id, alarm_type,
18                          ALARM_COOLDOWN_SECONDS, dedup_subject):
19        # alarma recenta nerezolvata exista deja
20        return
21
22    metadata["dedup_subject"] = dedup_subject
23    db.create_alarm(camera_id, alarm_type, severity,
24                   description, snapshot, metadata)
25    # alarma persistata e difuzata in UI (panou alarme, REST)

```

Listing 3.2: Poarta comună de deduplicare a alarmelor (*_create_alarm_if_needed* din *app.py*).

Figura 3.10 sintetizează fluxul decizional complet de alarmare pentru un singur cadru, de la evaluarea rezultatelor agregate până la stocarea persistentă a evenimentului prin intermediul mecanismului centralizat de deduplicare.

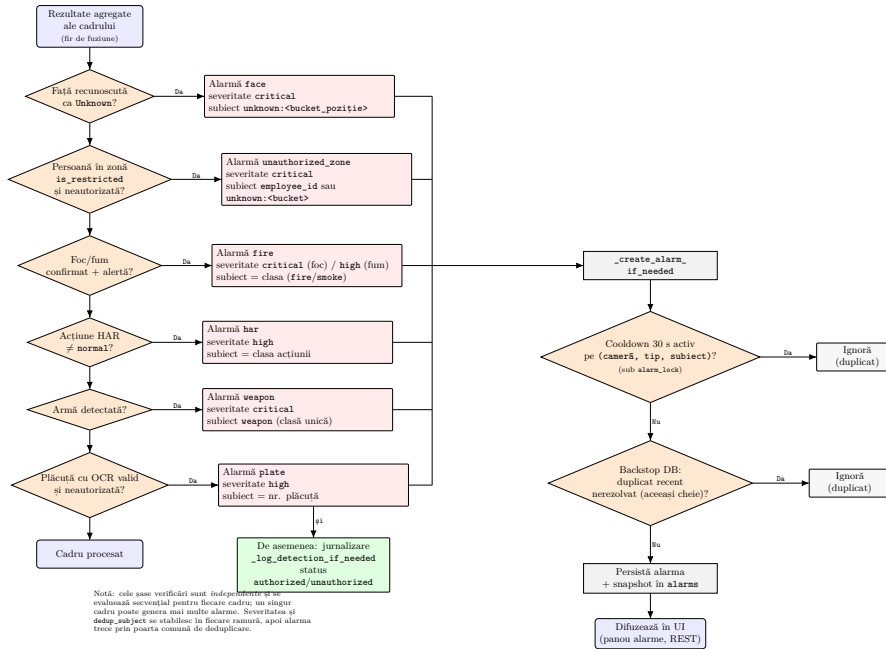


Figura 3.10: Diagrama de flux a deciziei de alarmare pentru un cadru. Pornind de la rezultatele agregate în firul de fuziune, cele șase verificări sunt evaluate *independent* și secvențial, astfel încât un singur cadru poate genera mai multe alarme. Fiecare ramură stabilește tipul, severitatea și `dedup_subject`, după care alarma trece printr-o poartă comună: cooldown de 30 s pe cheia (`cameră`, `tip`, `subiect`) (sub `alarm_lock`) și un *backstop* în baza de date împotriva duplicatelor recente nerezolvate, înainte de persistare și difuzare în interfață.

3.10 Proiectarea interfeței utilizator

Interfața utilizator este punctul de contact dintre operator și sistem; ea trebuie să facă vizibile, într-un mod imediat și neambiguu, fluxurile live, alarmele și starea entităților. Frontend-ul este o aplicație React (vezi §3.3.1), organizată ca un *single-page application* cu o bară laterală de navigare și un panou central de conținut care comută între ecranele descrise mai jos. Această secțiune prezintă structura de navigare, wireframe-urile ecranelor principale și principiile UX urmărite, raportate la implementarea efectivă din `frontend/src`.

3.10.1 Structura de navigare

Navigarea este controlată de componenta **Sidebar**, care definește nouă ecrane, fiecare cu un indicator `adminOnly` ce determină vizibilitatea în funcție de rol (mecanism aliniat cu RBAC-ul din §3.8):

- accesibile tuturor (`adminOnly: false`): *Dashboard, Alarms, Logs*;
- rezervate administratorului (`adminOnly: true`): *Person Management, Zone Management, Camera Management, Plate Management, Statistics, User Management*.

Filtrarea elementelor de meniu se face în client (`allNavItems.filter(item => !item.adminOnly || isAdmin)`), astfel încât un operator cu rol `user` nu vede deloc rutele de administrare. Aceasta este o protecție de *uzabilitate*, dublată pe server de verificările `require_admin` (protecția de securitate propriu-zisă).

3.10.2 Principii UX

Feedback vizual imediat. Fiecare acțiune produce un răspuns vizibil: stările `isLoading` pe butoane în timpul cererilor, mesaje de eroare/succes, indicatorul de conexiune WebSocket (`isConnected`) pe dashboard, și suprapunerea în timp real a detecțiilor peste fluxul video. Severitatea alarmelor este codată cromatic pentru identificare rapidă.

Confirmări pentru acțiuni distructive. Toate ștergerile cer o confirmare explicită prin dialog, cu mesaj care explică *consecințele*, nu doar acțiunea. De exemplu, ștergerea unei persoane avertizează că *toate datele faciale vor fi eliminate și persoana nu va mai fi recunoscută*, iar ștergerea unei zone avertizează că *camerele din ea vor rămâne neasignate, iar persoanele își vor pierde această zonă din lista autorizată*. Aceste confirmări sunt prezente în `PersonManagement`, `PlateManagement`, `ZoneManagement`, `CameraManagement` și `UserManagement`.

Filtrare consistentă. Tabelele cu volum mare de date oferă filtrare uniformă: pagina de *Alarme* filtrează pe status/tip/severitate, pagina de *Jurnale* pe tip/status/interval de timp/căutare, ambele cu paginare. Filtrele sunt transmise ca parametri de query către endpoint-urile REST corespunzătoare, deci filtrarea se face pe server, nu în client – important pentru seturi mari de date.

Layout responsive. Stilizarea cu Tailwind folosește clase responsive (`sm:`, `md:`, `lg:`, `grid-cols-*`) în toate componentele majore, astfel încât interfața să fie utilizabilă atât pe stația fixă a dispecerului, cât și pe dispozitive mobile pentru supraveghere de la distanță (cerința NF8).

3.11 Limitări ale proiectării

Deciziile de proiectare prezentate în acest capitol au fost luate în raport cu un scenariu țintă precis – un sistem de supraveghere inteligent pe un singur nod cu GPU dedicat, care deservește un număr moderat de camere și de operatori (§3.3.3). Multe dintre alegerile arhitecturale optime pentru acest scenariu se transformă însă în constrângeri atunci când contextul de utilizare se schimbă. Această secțiune adună, în mod onest și sistematic, limitările intrinseci ale proiectării – nu defecte de implementare, ci consecințe inevitabile ale compromisurilor asumate. Fiecare limitare este pusă în relație cu decizia de proiectare care o generează, întrucât majoritatea acestora reprezintă fața nefavorabilă a unui avantaj exploatat în altă parte.

3.11.1 Limitări arhitecturale și de scalabilitate

Arhitectura monolitică, mono-nod, nu este scalabilă pe orizontală. Toate cele cinci detectoare, serverul FastAPI și indexul FAISS rulează într-un *singur proces*, pe un *singur nod* cu un *singur GPU* (§3.3.3). Această alegere este justificată pentru scenariul țintă – partajarea modelelor în aceeași memorie GPU, cost zero de serializare, desfășurare (*deployment*) simplificată (§3.4.3) –, dar fixează un plafon: scalarea este posibilă doar *vertical* (un GPU mai puternic), nu *orizontal* (distribuirea camerelor pe mai multe noduri). La un număr mare de camere, cele cinci modele intră în competiție pentru aceeași placă, iar latențele sub sarcină completă depășesc valorile măsurate izolat în Tabelul 3.1, care constituie deci un *plafon optimist*. Cerința NF2 garantează extensia *structurală* (replicarea firelor de captură), nu și menținerea performanței la scară.

Punct unic de eșec, fără redundanță. Din aceeași opțiune mono-nod decurge absența oricărui mecanism de *înaltă disponibilitate*: căderea nodului server oprește întregul sistem de supraveghere, iar în varianta implicită baza de date PostgreSQL coexistă pe același echipament gazdă (§3.3.1). Nu există mecanisme de preluare a sarcinilor în caz de defecțiune (*failover*), replicare sau redundanță activă. Acest compromis este acceptabil pentru o implementare demonstrativă sau de tip pilot, dar reprezintă o limitare pentru un sistem de securitate critic, unde tocmai indisponibilitatea momentului supravegherii este inacceptabilă.

Transportul video prin WebSocket consumă lățime de bandă. Cadrele sunt redimensionate la maximum 800 px, codificate JPEG la calitate 75 și apoi **Base64** (§3.6), o codificare care adaugă un surplus de aproximativ 33% peste dimensiunea binară. În plus, modelul de difuzare este de tip *unu-la-multi*, cu o copie a cadrului trimisă în coada fiecărui client (§3.4.1), astfel încât traficul de rețea crește liniar cu numărul de operatori conectați. Nu se folosește un codec video cu compresie inter-cadru (de tip H.264/H.265 sau WebRTC). Alegerea privilegiază simplitatea – un brow-

ser poate decoda și afișa trivial un fișier JPEG Base64, fără o bibliotecă suplimentară –, în detrimentul eficienței la nivel de rețea.

3.11.2 Limitări de performanță și de procesare

Recunoașterea facială domină calea critică de latență. Cu o mediană de ~ 83 ms și o percentilă 95 de ~ 161 ms (Tabelul 3.1), recunoașterea facială cu analiză demografică este cel mai lent detector – de peste douăzeci de ori mai costisitor din punct de vedere computațional decât detecția plăcuțelor – și se află pe calea critică a cerinței NF1 (buget de 100 ms per cadru). Latența acestuia crește odată cu numărul de fețe simultane din imagine și sub competiția pentru resursele GPU, putând depăși bugetul țintă. Mecanismul bazat pe *cel mai recent cadru disponibil* (§4.8.2) maschează acest cost în interfață – fluxul rămâne fluid la rata camerei –, însă rata reală de actualizare a rezultatelor faciale rămâne limitată la aproximativ 12 actualizări pe secundă, indiferent de fluiditatea afișării.

Dependența de un GPU CUDA. Atingerea latențelor țintă presupune existența unui GPU compatibil CUDA cu cel puțin 8 GB VRAM (§3.3.3). În absența unui furnizor hardware CUDA, biblioteca `onnxruntime` comută automat pe `CPUExecutionProvider`, iar sistemul rămâne *funcțional, dar nu în timp real*, latențele crescând semnificativ (cerința NF9). Procesarea pe CPU reprezintă astfel o stare de funcționare degradată, nu o alternativă viabilă pentru exploatare, ceea ce restrânge gama de hardware pe care sistemul poate fi efectiv instalat.

Pierderea de cadre este asumată prin proiectare. Pentru a menține o latență minimă, cozile au o capacitate redusă (`maxsize=10`), iar politicile de tratare a saturării *ignoră* cadrele la intrare și aplică eliminarea elementului cel mai vechi (*drop-oldest*) la ieșire (§4.8.2); suplimentar, modulul de captură aplică un mecanism de omitere a cadrelor (*frame-skip*) (§3.4.1). Consecința directă este că sistemul *nu* procesează fiecare cadru: un eveniment foarte scurt, vizibil exclusiv pe imaginile omise sau abandonate, poate trece neînregistrat. Acesta este compromisul explicit dintre debit/latență (prioritizarea celui mai recent cadru) și completitudinea acoperirii temporale.

3.11.3 Limitări ale modulelor de detecție

Recunoașterea acțiunilor operează la nivel de clip, nu de cadru.

Modulul HAR nu poate clasifica un cadru izolat: el acumulează date într-o memorie tampon și produce un rezultat exclusiv periodic, o dată la ~ 2 s (la fiecare `clip_interval_frames`), pe baza unei secvențe de până la 64 de cadre (§3.4.1). Rezultă o *întârziere* și o *granularitate* de ordinul secundelor: o acțiune scurtă care începe și se încheie în interiorul unei ferestre de

procesare poate fi diluată în contextul clipului sau ratată complet, iar suprapunerea *ultimului rezultat* persistă eticheta anterioară pe cadrele dintre două inferențe.

3.11.4 Limitări ale modelului de date

Integritatea referențială este sacrificată în favoarea vitezei. Relația persoană–zonă este denormalizată ca atribut multivaloare `authorized_zones TEXT[]`, cu zonele referite prin *nume*, iar câmpurile `camera_id` din jurnale și alarme sunt de tip `VARCHAR`, nu chei externe către tabelul `cameras` (§3.7). Decizia favorizează citirea rapidă a autorizărilor (reținute în memoria *cache*) și scrierea accelerată a jurnalelor din firul de fuziune, dar baza de date nu mai poate garanta consistența strictă: redenumirea unei zone sau a unui identificator de cameră poate genera referințe logice orfane, fără ca sistemul de gestiune să semnaleze inconsistența.

Capturile sunt stocate ca text Base64 în baza de date. Capturile de imagine (*snapshots*) aferente alarmelor sunt salvate ca șiruri Base64 în coloana `snapshot` a tabelului `alarms` (§3.7, §3.9.1), iar sistemul nu expune un serviciu de fișiere statice sau un sistem de stocare a obiectelor (*object store*) dedicat. Alegerea simplifică arhitectura aplicației (o singură sursă de adevăr, fără un volum de fișiere fizice de gestionat), însă tabelul de alarme crește necontrolat odată cu volumul de evenimente, îngreunând realizarea copiilor de siguranță (*backups*) și interogările pe care, în mod normal, datele binare voluminoase nu ar trebui să le afecteze.

3.11.5 Limitări de confidențialitate și de gestiune a sesiunilor

Prelucrarea datelor biometrice ridică probleme de confidențialitate.

Sistemul detectează fețe, stochează reprezentări vectoriale faciale de 512 dimensiuni (date biometrice), estimează attribute demografice (vârstă, gen, emoție) și păstrează capturi ale persoanelor în alarme. Toate acestea constituie categorii de date personale sensibile, vizate de reglementările de protecție a datelor (precum GDPR). Proiectarea actuală tratează riguros *accesul* la aceste informații (control bazat pe roluri, autentificare JWT, parole securizate prin hashing bcrypt – §3.8), dar nu include, la nivel de arhitectură, un mecanism explicit de *consimțământ*, o politică de *retenție* și *expirare* a reprezentărilor faciale sau o procedură de anonimizare.

Aceste limitări nu invalidează soluția: cele mai multe sunt compromisuri, asumate pentru a satisface scenariul țintă cu resurse hardware realiste, iar fiecare dintre ele constituie un punct de plecare concret pentru direcțiile de dezvoltare viitoare (Capitolul 7). Recunoașterea lor explicită face parte din analiza critică a proiectării și delimitează clar domeniul în care sistemul propus reprezintă o soluție adecvată.

Capitolul 4: Implementarea Modulelor de Inteligență Artificială

4.1 Privire de ansamblu și justificarea alegerii modelelor

Acest capitol descrie cele șase module de inteligență artificială ale sistemului – recunoașterea facială, analiza demografică și emoțională, recunoașterea plăcuțelor de înmatriculare, detecția focului și a fumului, detecția armelor și recunoașterea acțiunilor umane –, precum și fuziunea rezultatelor acestora la nivel de cadru. Înainte de a detalia fiecare modul în parte, această secțiune oferă o imagine de ansamblu a sarcinilor de inteligență artificială rezolvate și justifică, pentru fiecare dintre ele, modelul (sau biblioteca) ales în raport cu alternativele relevante din literatură și din practica curentă.

Figura 4.1 oferă o imagine de ansamblu a fluxului de procesare, înainte de descrierea individuală a modulelor. Fiecare cadru capturat din fluxul video este transmis, în paralel, către cele cinci module de inteligență artificială, care rulează *concurrent* pe aceeași placă grafică; rezultatele lor sunt apoi recombinate într-o etapă de fuziune, care generează alarmele și jurnalele și difuzează fluxul video adnotat către operatori. Mecanismele care fac posibilă această arhitectură (sincronizarea rezultatelor, generarea și deduplicarea alarmelor, difuzarea către clienți) sunt tratate în detaliu în Secțiunea 4.8, iar arhitectura multi-threaded completă a fost prezentată în Capitolul 3 (Figura 3.6).

Selecția fiecărui model nu a urmărit exclusiv acuratețea brută pe seturi de date de referință, ci a fost ghidată de un set de patru criterii corelate cu specificul aplicației (un sistem de supraveghere care procesează un flux video în timp real):

1. **Inferență în timp real în regim concurrent** – toate modelele rulează simultan, pe fire de execuție separate, utilizând *aceeași* placă grafică (a se vedea Secțiunea 4.8); un model lent ar deveni factorul limitativ pentru întregul flux de procesare;
2. **Robustețe în condiții reale de supraveghere** – rezoluții reduse, iluminare neuniformă, unghiuri oblice și ocluzii parțiale;

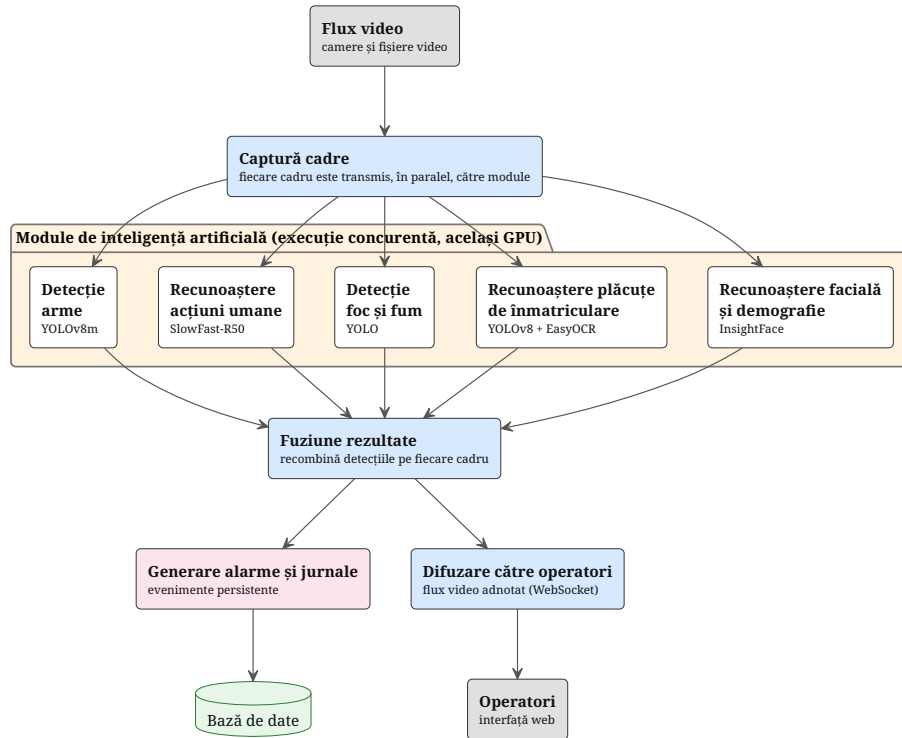


Figura 4.1: Imagine de ansamblu a fluxului de procesare al modulelor de inteligență artificială. Cadrele capturate din fluxul video sunt transmise, în paralel, către cele cinci module de inteligență artificială; sub fiecare modul este indicat modelul (sau biblioteca) pe care se bazează. Rezultatele converg în etapa de fuziune, care le recombina pe fiecare cadru, generează alarmele și jurnalele și difuzează cadrul adnotat către operatori. Pentru claritate, diagrama rămâne la nivel de ansamblu; detalierea firelor de execuție, a coșilor și a mecanismului de fuziune se găsește în Figura 3.6 și în Secțiunea 4.8.

3. **Disponibilitatea unor ponderi pre-antrenate mature** – care reduc sau elimină costul de antrenare și oferă un punct de pornire solid pentru optimizarea suplimentară (*fine-tuning*), acolo unde aceasta a fost necesară;
4. **Ușurința integrării** în arhitectura Python concurentă a aplicației (compatibilitate cu ecosistemele PyTorch / ONNX Runtime, API stabil, întreținere activă).

Tabelul 4.1 sintetizează, pentru fiecare sarcină de inteligență artificială, alternativele luate în considerare, modelul ales și argumentele principale care au stat la baza deciziei. Detaliile de implementare sunt prezentate în secțiunile următoare ale prezentului capitol.

Tabel 4.1: Justificarea alegerii modelelor de inteligență artificială, pe sarcini.

Sarcină AI	Alternative considerate	Model ales	Justificarea alegerii
Detecția și recunoașterea facială	FaceNet, Dlib (ResNet), DeepFace, ArcFace de sine stătător	InsightFace (buffalo_s)	Vectori de trăsături (<i>embeddings</i>) de 512 dimensiuni, normalizați L2, antrenați utilizând funcția de cost ArcFace pe setul Glint360K, robusti la variații de postură și iluminare; detecția, recunoașterea și datele demografice se obțin într-o singură etapă de procesare; execuție eficientă pe GPU prin ONNX Runtime; bibliotecă întreținută activ.
Estimarea vârstei și a genului	Modele dedicate (DEX, FairFace), clasificatoare Caffe age/gender	InsightFace genderage	Modul inclus deja în pachetul buffalo_s , ceea ce elimină costul suplimentar de inferență (rezultatele fiind extrase în aceeași iterație cu recunoașterea facială); previne încărcarea în memorie a unui model adițional.
Recunoașterea emoțiilor	DeepFace, VGG-Face, ResMaskNet (AffectNet)	Mini-Xception (pachetul fer)	Rețea convoluțională compactă, cu latență neglijabilă la execuția pe CPU, eliberând astfel resursele GPU pentru modelele cu cerințe computaționale ridicate; pachet matur, antrenat pe setul standard FER-2013; se aplică direct pe zona facială decupată furnizată de InsightFace.

continuă pe pagina următoare...

... continuare a Tabelului 4.1

Sarcină AI	Alternative considerate	Model ales	Justificarea alegerii
Detecția obiectelor (plăcuțe, foc/fum, arme)	Faster R-CNN, SSD, RetinaNet, DETR, EfficientDet	YOLOv8 / YOLOv10	Detectoare într-o singură etapă (<i>single-stage</i>) capabile de inferență în timp real, aspect esențial pentru rularea concurentă a mai multor modele; ecosistem Ultralytics matur, cu posibilități facile de optimizare suplimentară (<i>fine-tuning</i>) și ponderi pre-antrenate disponibile; precizie competitivă în raport cu detectoarele <i>two-stage</i> .
OCR pe plăcuțe de înmatriculare	Tesseract, PaddleOCR, Keras-OCR, TroOCR	EasyOCR (CRAFT + CRNN)	Performanță superioară pe imagini degradate sau cu rezoluție redusă (cazul tipic al plăcuțelor în cadru, situație în care depășește net performanțele Tesseract); model bazat pe învățare profundă (CRNN) care operează pe imagini în tonuri de gri; dispune de suport GPU și permite definirea unei liste de permisiuni (<i>allowlist</i>) pentru restrângerea alfabetului.
Recunoașterea acțiunilor umane	I3D, C3D, X3D, MoViNet, TimeSformer / VideoMAE	SlowFast R50	Arhitectură matură care separă explicit ramura spațială (<i>Slow</i>) de ramura temporală (<i>Fast</i>), fiind optimă pentru distingerea acțiunilor pe baza mișcării; beneficiază de ponderi pre-antrenate pe setul Kinetics-400 prin intermediul PyTorchVideo; varianta R50 oferă un echilibru solid între capacitatea de predicție și costul computațional (inferență în timp real), evitând complexitatea pătratică și necesitatea pre-antrenării masive specifice modelelor bazate pe arhitecturi Transformer.

continuă pe pagina următoare. . .

... continuare a Tabelului 4.1

Sarcină AI	Alternative considerate	Model ales	Justificarea alegerii
Căutarea vectorială (similaritate)	Brute-force NumPy, Annoy, HNSWlib, ScaNN, pgvector	FAISS (IndexFlatL2)	Căutare exactă, care garantează identificarea celui mai apropiat vecin și elimină etapa antrenării indecșilor pentru căutări aproximative; operațiunea rămâne instantanee pentru dimensiunea tipică a unei baze de date de angajați; bibliotecă matură (susținută de Meta), cu integrare directă pentru vectori de trăsături de 512 dimensiuni.

Se remarcă faptul că majoritatea modulelor reutilizează modele pre-antrenate în forma lor originală (InsightFace, EasyOCR, detectoarele YOLO pentru plăcuțe și foc/fum, Mini-Xception), două componente fiind ajustate fin în cadrul proiectului pe seturi de date construite special: detectorul de arme (YOLOv8m, Secțiunea 4.6) și recunoașterea acțiunilor (SlowFast R50, Secțiunea 4.7). Această strategie mixtă – modele de-a gata acolo unde există soluții robuste, ajustare fină acolo unde nu există – reflectă exact criteriile de selecție enumerate mai sus: minimizarea costului de dezvoltare fără a sacrifica robustețea în scenariile specifice de supraveghere.

4.2 Modulul de recunoaștere facială

Modulul de recunoaștere facială constituie componenta centrală a sistemului de control al accesului și este implementat în clasa `FacialRecognitionSystem` din fișierul `facial_recognition_system.py`. Rolul său este de a identifica, în fiecare cadru video, persoanele înregistrate anterior în baza de date și de a distinge între persoanele cunoscute (angajați) și cele necunoscute. Spre deosebire de pipeline-urile clasice de recunoaștere facială, care înlanțuie un detector de fețe, un model separat de extragere a vectorilor de caracteristici și încă unul pentru estimarea atributelor demografice, implementarea de față consolidează toate aceste sarcini într-o singură trecere prin pachetul de modele *buffalo_s* al framework-ului InsightFace.

Pipeline-ul intern al modulului are patru etape, descrise în subcapitolele care urmează: (1) detecția fețelor și extragerea reprezentărilor vectoriale cu InsightFace, (2) indexarea și căutarea vectorilor cu FAISS, (3) strategia de stabilire a identității pe baza distanței și (4) optimizarea prin omiterea analizei demografice pentru fețele deja recunoscute. La nivelul aplicației, modulul rulează într-un fir de execuție dedicat (*Thread 2*), care preia cadre dintr-o coadă de tip `queue.Queue`, apelează metoda `process_frame()` și publică rezultatele în coada de fuziune (a se vedea secțiunea 4.8).

Figura 4.2 sintetizează întregul flux de procesare al modulului, de la cadrul de intrare până la rezultatul transmis firului de fuziune, reunind cele patru etape enumerate mai sus împreună cu analiza demografică și emoțională (secțiunea 4.3). Se remarcă punctul central de decizie – pragul dublu aplicat distanței și nivelului de încredere – care determină tratarea diferențiată a fețelor cunoscute față de cele necunoscute. Subcapitolele care urmează detaliază, pe rând, fiecare etapă a acestui flux.

4.2.1 Detecția fețelor și extragerea reprezentărilor vectoriale

Detecția fețelor și extragerea vectorilor de caracteristici sunt realizate de obiectul `FaceAnalysis` din `InsightFace`, inițializat cu pachetul de modele `buffalo_s`. Acest pachet reunește trei submodele care, în abordarea tradițională, ar fi necesitate încărcarea și apelarea separată:

- **det_500m** – detectorul de fețe, care returnează caseta de încadrare (*bounding box*) și cele cinci puncte caracteristice (landmark-uri) pentru fiecare față din cadru;
- **w600k_mbf** – rețeaua de recunoaștere, o variantă `MobileFaceNet` antrenată pe setul de date `Glint360K` cu funcția de pierdere `ArcFace`, care produce vectorul de caracteristici facial de 512 dimensiuni;
- capul **genderage** – care estimează vârsta și genul persoanei.

Alegerea variantei `buffalo_s` (în locul, de exemplu, a variantei `buffalo_sc`) este motivată tehnic: rețeaua de recunoaștere `w600k_mbf` este identică în ambele pachete, astfel încât reprezentările vectoriale de 512 dimensiuni rămân compatibile, însă `buffalo_s` include modulul suplimentar `genderage`, indispensabil pentru analiza demografică (secțiunea 4.3). În lipsa acestui modul de predicție, estimările privind vârsta și genul ar fi indisponibile, sistemul returnând valoarea `None`.

Pentru fiecare cadru, metoda `process_frame()` apelează `self.face_app.get(frame)`, care efectuează într-o singură trecere detecția tuturor fețelor și, pentru fiecare, calculează `bounding box`-ul, landmark-urile, vectorul de caracteristici L2-normalizat (`normed_embedding`, vector de 512 de elemente de tip `float32`) și estimările de vârstă și gen. `Bounding box`-ul de tip `[x1, y1, x2, y2]` returnat de `InsightFace` este convertit în formatul `(x, y, w, h)` și ancorat în limitele cadrului de metoda statică `_bbox_xyxy_to_xywh()`, pentru a evita decupaje în afara imaginii.

4.2.1.1 Configurarea execuției pe GPU

Inferența `InsightFace` rulează pe GPU prin runtime-ul `ONNX`, providerul `CUDAExecutionProvider` fiind configurat explicit cu opțiunea `cudnn_conv_algo_`

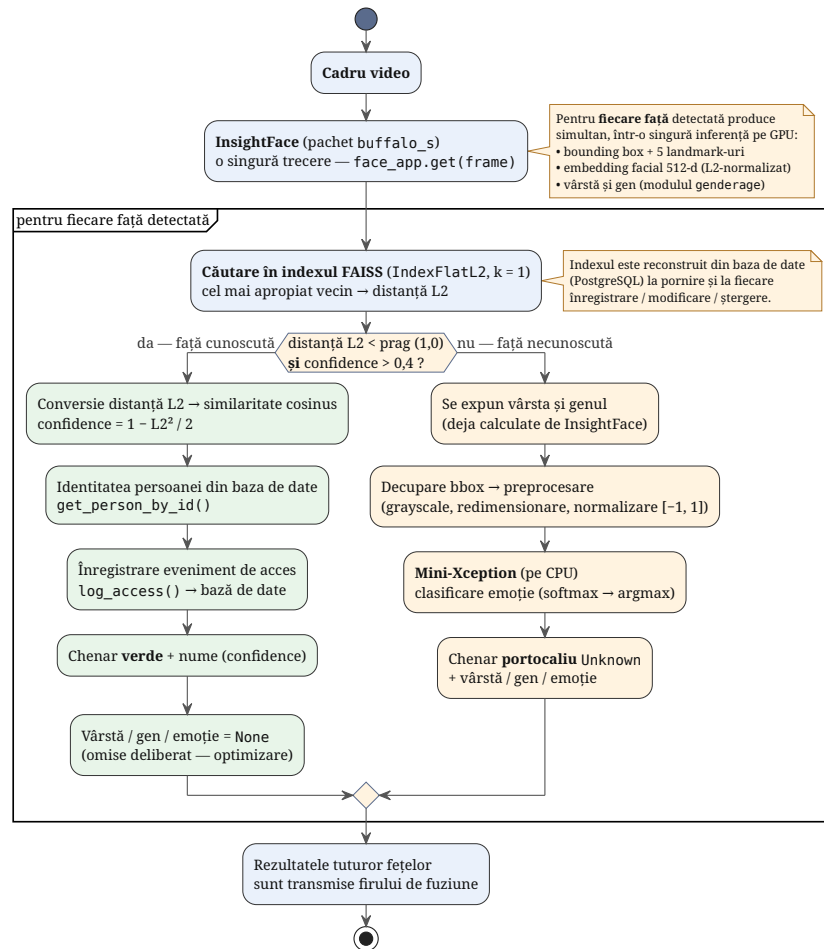


Figura 4.2: Fluxul de procesare al modulului de recunoaștere facială. Fiecare cadru este analizat de InsightFace într-o singură trecere, care produce simultan, pentru toate fețele, caseta de încadrare, embedding-ul de 512 dimensiuni și estimările demografice. Pentru fiecare față, embedding-ul este căutat în indexul FAISS, iar identitatea se stabilește pe baza unui prag dublu (distanță $L_2 < 1,0$ și încredere $> 0,4$). Fețele cunoscute (ramura verde) sunt jurnalizate ca evenimente de acces, iar atributele demografice sunt omise deliberat; fețele necunoscute (ramura portocalie) sunt caracterizate demografic, iar emoția este clasificată suplimentar de rețeaua Mini-Xception pe CPU. Indexul FAISS este reconstruit determinist din baza de date.

`search="HEURISTIC"`. Această opțiune este o decizie de implementare importantă în contextul arhitecturii multi-threaded a aplicației: strategia implicită, `EXHAUSTIVE`, măsoară empiric timpul fiecărui algoritm de convoluție folosind mecanismul de *stream capture* al CUDA. Cum InsightFace rulează concurrent cu modelele PyTorch (YOLO pentru plăcuțe, foc și arme, respectiv SlowFast pentru acțiuni) pe același GPU, o astfel de captură în desfășurare ar întrerupe orice kernel PyTorch paralel cu eroarea *“operation not permitted when stream is capturing”*. Varianta `HEURISTIC` selectează algoritmul pe baza unor euristici, fără benchmarking și deci fără captură, eliminând coliziunile între firele de execuție.

4.2.1.2 Gestionarea bibliotecilor CUDA și a TensorFlow

La importul modului, funcția `_preload_nvidia_pypi_libs()` preîncarcă explicit, prin `ctypes.CDLL(..., RTLD_GLOBAL)`, bibliotecile partajate CUDA, cuDNN și cuBLAS livrate în pachetele pip `nvidia-*-cu12`, astfel încât `onnxruntime` să le poată găsi fără a necesita setarea manuală a variabilei `LD_LIBRARY_PATH`. În plus, TensorFlow (folosit de modelul de emoții) este forțat să nu folosească GPU-ul prin `set_visible_devices([], "GPU")`, deoarece menținerea lui pe GPU provoacă conflicte de context CUDA cu `onnxruntime-gpu` și `segmentation fault` la inițializare.

4.2.1.3 Înregistrarea persoanelor

Reprezentările vectoriale ale persoanelor cunoscute sunt generate în momentul înregistrării acestora în sistem. Metoda `register_person()` primește o listă de imagini faciale și execută modelul InsightFace pe fiecare dintre ele. În situația în care sunt detectate mai multe fețe într-o imagine, sistemul o selectează pe cea cu aria maximă a casetei de încadrare (*bounding box*), prin intermediul funcției `_extract_single_face()`. Ulterior, este extras *vectorul de caracteristici* de 512 dimensiuni, care va fi stocat atât în baza de date, cât și în indexul FAISS. Pentru încărcările de imagini efectuate prin interfața web, metoda `extract_embedding_from_image()` acceptă exclusiv imaginile care conțin o singură față detectabilă, prevenind astfel riscul asocierii ambigue a unui *vector de caracteristici* cu o persoană.

4.2.2 Indexarea cu FAISS FlatL2

Căutarea identității unei fețe presupune compararea vectorului său de caracteristici cu toți descriptorii persoanelor înregistrate. Pentru a face această operație eficientă, sistemul folosește biblioteca FAISS, încapsulată în clasa `FaissIndex` din fișierul `faiss_index.py`. Tipul de index utilizat este ***IndexFlatL2***, care realizează o căutare *exactă* (*brute-force*) folosind distanța euclidiană L2 între vectorul de interogare și fiecare vector din index. Spre deosebire de indecșii aproximativi (precum IVFFlat), căutarea exactă nu necesită o etapă de antrenare și garantează găsirea celui mai apropiat vecin; aceasta este alegerea

potrivită pentru dimensiunea tipică a unei baze de angajați, unde numărul de vectori de caracteristici este suficient de mic încât căutarea liniară să rămână instantanee.

4.2.2.1 Maparea identităților

FAISS operează cu indici poziționali interni (0, 1, 2, ...), nu cu identificatorii persoanelor din baza de date. De aceea, clasa `FaissIndex` menține două dicționare de mapare: `id_map`, care asociază fiecărui index FAISS un `person_id`, și `reverse_id_map`, care leagă fiecare `person_id` de lista de indici corespunzători (o persoană poate avea mai multe reprezentări vectoriale, câte una pentru fiecare imagine înregistrată).

4.2.2.2 Persistența și reconstrucția indexului

Indexul FAISS este o structură de date alocată în memorie, în timp ce baza de date PostgreSQL constituie punctul central de stocare și referință pentru toți vectorii de caracteristici. În tabelul `face_embeddings`, fiecare *vector de caracteristici* este salvat în coloana `embedding` de tip `BYTEA`, fiind serializat cu ajutorul bibliotecii `pickle`. La pornirea sistemului, metoda `_load_embeddings_from_db()` preia toate datele prin `get_all_embeddings()` și reconstruiește complet indexul prin apelarea funcției `rebuild_index()`. Această procedură de reconstrucție este declanșată automat ori de câte ori o persoană este înregistrată, modificată sau ștearsă, garantând astfel consistența strictă între stocarea persistentă și indexul volatil.

4.2.2.3 Siguranța accesului concurent

Întrucât firul de recunoaștere facială interoghează indexul în fiecare cadru, în timp ce firele care deserveșc cererile HTTP îl pot reconstrui (la înregistrarea sau ștergerea persoanelor), iar operațiile `add/search` ale FAISS nu sunt sigure pentru execuția concurentă, toate accesările indexului și ale dicționarelor de mapare sunt serializate cu un lock reentrant (`threading.RLock`). Caracterul reentrant este necesar deoarece `rebuild_index()` apelează intern `add_embeddings_batch()`, care preia același lock.

4.2.3 Strategia de matching

Stabilirea identității unei fețe pe baza distanței euclidiene este implementată în metoda `_match_embedding()`. Aceasta interoghează mai întâi indexul pentru cel mai apropiat vecin ($k=1$) cu un prag “infinit”, astfel încât distanța reală până la cel mai apropiat vector să poată fi înregistrată pentru calibrare (activabilă prin variabila de mediu `FR_DEBUG`), după care aplică manual pragul de decizie configurat.

4.2.3.1 De la distanța L2 la similaritatea cosinus

Deoarece vectorii de caracteristici sunt L2-normalizați, între distanța euclidiană la pătrat și similaritatea cosinus a doi vectori există relația exactă $L2^2 = 2 - 2 \cos \theta$, echivalentă cu $\cos \theta = 1 - L2^2/2$. Această echivalență permite raportarea identității prin două mărimi interpretabile:

- `recognition_threshold = 1.0` – pragul aplicat distanței L2. Un prag $L2 = 1,0$ corespunde unei similarități cosinus de 0,5. Empiric, distribuția distanțelor pentru aceeași persoană se situează în intervalul $[0,6; 0,85]$ pe camera web folosită, deci un candidat cu $L2 \geq 1,0$ este respins ca necunoscut;
- `min_confidence = 0.4` – pragul aplicat încrederii, exprimată ca similaritate cosinus. Valoarea 0,4 este punctul de operare clasic “aceeași persoană” pentru ArcFace.

Dacă distanța celui mai apropiat vecin depășește `recognition_threshold`, metoda returnează `None` (față necunoscută). În caz contrar, distanța este convertită în similaritate cosinus, mărginită la intervalul $[0; 1]$ și raportată ca nivel de încredere; identificatorul persoanei este apoi rezolvat în datele complete (nume, ID angajat, departament) prin `get_person_by_id()`. În metoda `process_frame()`, o față este considerată cunoscută doar dacă există o potrivire și încrederea ei depășește `min_confidence`. Pentru fiecare potrivire validă se înregistrează un eveniment de acces în baza de date prin `log_access()`.

Vizual, fețele cunoscute sunt marcate cu un chenar verde și eticheta `nume (încredere)`, iar cele necunoscute cu un chenar portocaliu și eticheta `Unknown`, prin metoda `_draw_face()`.

Listing-ul 4.1 prezintă strategia de asociere a identității, exact așa cum este implementată în metoda `_match_embedding()`: identificarea celui mai apropiat vecin în indexul FAISS, respingerea candidaților care depășesc pragul distanței L2, conversia distanței în similaritate cosinus și stabilirea identității. Această secvență este urmată de un filtru suplimentar bazat pe parametrul `min_confidence`, aplicat la nivelul funcției `process_frame()`.

```

1 # vector L2-normalizat, 512-d
2 def match_embedding(embedding):
3     # Cautare nemarginita: inregistram distanta reala pentru calibrare,
4     # apoi aplicam noi pragul de decizie configurat
5     raw = faiss_index.search(embedding, k=1, threshold=float("inf"))
6     if not raw:
7         # index gol, niciun candidat
8         return None
9
10    # cel mai apropiat vecin (distanța L2)
11    person_id, distance = raw[0]
12
13    # 1.0 (echivalent cos_sim = 0.5)
14    if distance >= recognition_threshold:
15        # fata necunoscuta
16        return None
17
18    # Vectori L2-normalizati: L2^2 = 2 - 2*cos_sim => cos_sim = 1 - L2^2/2
19    cos_sim = 1.0 - distance ** 2 / 2.0
20    # marginit la [0, 1]
21    confidence = max(0.0, min(1.0, cos_sim))
22
23    person = db.get_person_by_id(person_id)
24    if not person:
25        return None
26    return {"person_id": person_id, "name": person["name"],
27            "confidence": confidence, "distance": distance}
28
29 # In process_frame() o fata e considerata cunoscuta doar daca exista o
30 # potrivire SI confidence > min_confidence (0.4); altfel ramane "Unknown".

```

Listing 4.1: Strategia de matching facial (`_match_embedding` din `facial_recognition_system.py`).

4.2.4 Optimizarea: omiterea analizei demografice pentru fețele cunoscute

O optimizare deliberată la nivelul *pipeline*-ului de procesare facială constă în condiționarea analizei demografice de rezultatul etapei de recunoaștere. Vârsta și genul sunt generate de modulul `genderage` al arhitecturii InsightFace pentru toate fețele dintr-un cadru, în aceeași trecere care produce și vectorul de caracteristici — acest cost nu poate fi evitat per-față. În cadrul funcției `process_frame()`, aceste atribute sunt însă reținute și expuse *exclusiv pentru fețele necunoscute*, iar emoția prezisă de rețeaua Mini-Xception (detaliată în Secțiunea 4.3) este calculată tot *numai pentru fețele necunoscute*, printr-o trecere suplimentară. În cazul unei persoane identificate cu succes, câmpurile `age`, `gender`, `emotion` și `emotion_confidence` sunt inițializate direct cu valoarea `None`.

Raționamentul are atât o componentă de proiectare, cât și una de performanță. Din perspectivă semantică, calcularea informațiilor demografice pentru o per-

soană deja recunoscută reprezintă o operațiune redundantă, întrucât identitatea, vârsta exactă și departamentul sunt deja disponibile în baza de date. Din punct de vedere al performanței, această decizie elimină necesitatea unei treceri suplimentare (*forward-pass*) prin rețeaua de clasificare a emoțiilor pentru fețele cunoscute, la fiecare cadru. Economia de resurse este considerabilă, în special pentru că inferența rețelei Mini-Xception se execută pe unitatea CPU. Prin urmare, costul computațional aferent analizei demografice este asumat doar atunci când generează informație utilă, adică în cazul persoanelor neidentificate, pentru care caracterizarea demografică și emoțională devine relevantă în scenariile de supraveghere.

4.3 Analiza demografică și emoțională

Pe lângă identificarea persoanelor cunoscute, sistemul caracterizează demografic și emoțional persoanele *necunoscute*, adică acele fețe care nu pot fi asociate niciunei intrări din baza de date. Această analiză produce trei atribute pentru fiecare astfel de față: vârsta estimată, genul și emoția. Implementarea este integrată în aceeași clasă `FacialRecognitionSystem` și în aceeași metodă `process_frame()` ca recunoașterea facială, evitând o trecere separată de detecție a fețelor. Vârsta și genul sunt produse de modulul `genderage` al Insight-Face pentru *toate* fețele detectate, în aceeași trecere care generează și vectorul de caracteristici, fiind însă reținute și afișate exclusiv pentru fețele necunoscute; în schimb, estimarea emoției, care presupune o trecere suplimentară prin rețeaua Mini-Xception, este efectuată doar pentru fețele necunoscute. După cum a fost detaliat în secțiunea 4.2.4, pentru persoanele deja identificate câmpurile `age`, `gender`, `emotion` și `emotion_confidence` sunt lăsate la valoarea `None`, întrucât caracterizarea lor demografică ar fi redundantă.

Extragerea atributelor se bazează pe două modele distincte: vârsta și genul sunt furnizate de modulul `genderage` din cadrul arhitecturii InsightFace (deja evaluat implicit prin execuția pachetului `buffalo_s`), în timp ce starea emoțională este estimată de o rețea convoluțională independentă, Mini-Xception, aplicată direct pe regiunea facială decupată (*face crop*).

4.3.1 Estimarea vârstei și genului

Vârsta și genul nu necesită un model dedicat: ambele sunt generate de modulul `genderage` al pachetului `buffalo_s`, în aceeași trecere InsightFace care furnizează bounding box-ul, landmark-urile și vectorul de caracteristici. Pentru fiecare obiect `Face` returnat de detector, atributele sunt extrase astfel:

- **Vârsta** – câmpul `face.age` (valoare reală) este rotunjit la cel mai apropiat întreg. Dacă atributul lipsește, vârsta este raportată ca `None`;
- **Genul** – câmpul `face.gender` este o valoare întreagă în convenția Insight-Face (1 = masculin, 0 = feminin), care este tradusă în etichetele textuale "Man", respectiv "Woman".

Această abordare nu adaugă cost de inferență suplimentar: estimările sunt deja calculate de InsightFace, fiind doar citite și formate. (secțiunea 4.2.1).

4.3.2 Recunoașterea emoțiilor cu Mini-Xception

Spre deosebire de vârstă și gen, emoția este clasificată de un model separat: Mini-Xception, o rețea neuronală convoluțională compactă furnizată de pachetul Python `fer` sub forma fișierului de ponderi `emotion_model.hdf5`. Modelul este antrenat pe setul de date FER-2013 și clasifică expresia facială într-una din șapte categorii, ordonate conform ieșirilor softmax ale modelului în tuplul `_EMOTION_LABELS`: `angry`, `disgust`, `fear`, `happy`, `sad`, `surprise` și `neutral`.

Un aspect important al implementării este faptul că API-ul implicit al pachetului `fer` este *ocolit complet*. Pachetul `fer` efectuează în mod normal propria sa detecție de fețe cu un clasificator Haar cascade înainte de a clasifica emoția – o trecere redundantă, întrucât InsightFace a detectat deja toate fețele din cadru. În locul acestui flux, modelul Mini-Xception este încărcat direct prin `tf.keras.models.load_model()` (cu `compile=False`) și este alimentat cu crop-ul facial deja delimitat de bounding box-ul InsightFace. Astfel se elimină o detecție de fețe duplicată și se garantează că emoția este clasificată exact pe regiunea identificată de pipeline-ul principal.

Predicția propriu-zisă, implementată în metoda `_classify_emotion()`, constă într-o etapă de propagare directă (*forward pass*) prin rețea, urmată de aplicarea operației `argmax` pe cele șapte probabilități furnizate de funcția *softmax*. Rezultatul returnat este un tuplu format din eticheta emoției și scorul de încredere asociat. Dimensiunea de intrare necesară modelului este extrasă automat din proprietatea `input_shape`, asigurând astfel adaptarea dinamică a etapei de preprocesare (detaliată în continuare) la arhitectura specifică rețelei.

4.3.3 Preprocesarea crop-ului facial

Înainte de a fi evaluat de rețeaua de clasificare a emoțiilor, decupajul facial (extras din imaginea de ansamblu pe baza casetei de încadrare furnizate de InsightFace, în format BGR) parcurge o etapă riguroasă de preprocesare. Acest proces reproduce fidel fluxul intern de prelucrare al pachetului `fer`, garantând astfel faptul că ponderile antrenate ale modelului primesc datele de intrare exact în distribuția statistică necesară. Pașii de transformare, implementați în cadrul metodei `_classify_emotion()`, sunt următorii:

1. **Conversia în tonuri de gri:** regiunea facială în format BGR este convertită în scară de gri utilizând funcția `cv2.cvtColor`, deoarece modelul Mini-Xception procesează exclusiv imagini cu un singur canal de culoare;
2. **Redimensionarea:** imaginea rezultată este adusă la dimensiunile de intrare impuse de model (la valoarea `_emotion_target_size`, dedusă dinamic din atributul `input_shape`);

3. **Normalizarea:** intensitatea pixelilor este scalată din intervalul $[0, 255]$ în intervalul $[0, 1]$ (prin împărțire la 255), fiind ulterior mapată în intervalul $[-1, 1]$ aplicând transformarea matematică $(x - 0,5) \cdot 2$;
4. **Extinderea dimensionalității tensorului:** matricea bidimensională este remodelată la forma $(1, H, W, 1)$ (adăugându-se dimensiunea lotului de prelucrare, *batch size*, și axa canalului), acesta fiind formatul structural așteptat de modelele Keras.

Întregul lanț este protejat de un bloc `try/except`: dacă, din orice motiv, crop-ul este gol sau preprocesarea eșuează, metoda returnează `(None, None)`, iar fața este raportată fără emoție, fără a întrerupe procesarea celorlalte fețe din cadru.

4.3.4 Inferența pe CPU

Spre deosebire de InsightFace, care rulează pe GPU, rețeaua Mini-Xception este rulată deliberat pe CPU. Această decizie este aplicată chiar la importul modulului, unde TensorFlow este forțat să nu vadă niciun dispozitiv GPU prin `tf.config.set_visible_devices([], "GPU")`.

Motivația este dublă. În primul rând, menținerea TensorFlow pe GPU în paralel cu `onnxruntime-gpu` (folosit de InsightFace) provoacă conflicte de context CUDA și `segmentation fault` la initializare; izolarea TensorFlow pe CPU elimină complet această sursă de instabilitate. În al doilea rând, Mini-Xception este o rețea suficient de mică încât latența inferenței pe CPU să fie neglijabilă, iar emoția se calculează doar pentru fețele necunoscute (secțiunea 4.2.4); prin urmare, costul pe CPU este minor, iar GPU-ul rămâne dedicat integral modelelor grele.

4.4 Modulul de recunoaștere a plăcuțelor de înmatriculare

Modulul de recunoaștere a plăcuțelor de înmatriculare (ANPR – *Automatic Number Plate Recognition*) identifică și citește numerele de înmatriculare ale vehiculelor din fluxul video, le validează formatul, le caută în baza de date și decide dacă vehiculul este autorizat. Modulul este implementat în clasa `LicensePlateRecognitionSystem` din fișierul `license_plate_recognition_system.py` și rulează într-un fir de execuție dedicat (*Thread 3*).

Pipeline-ul combină un detector de obiecte (YOLOv8) cu un motor de recunoaștere optică a caracterelor (EasyOCR), între care se interpun mai mulți pași de preprocesare a imaginii și, ulterior, o etapă de stabilizare temporală a rezultatului. O provocare majoră, care a influențat semnificativ implementarea, este faptul că plăcuțele apar în cadru la rezoluții reduse (tipic 116–151 px lățime), insuficiente pentru o recunoaștere OCR fiabilă. Din acest motiv, *pipeline*-ul include un pas explicit de mărire a imaginii (*up-scaling*) și o etapă avansată de

filtrare și accentuare a detaliilor. Cele opt subcapitole care urmează descriu, în ordinea execuției, etapele acestui flux de prelucrare.

Figura 4.3 ilustrează întregul flux de prelucrare, de la detecția plăcuței până la decizia de autorizare, evidențiind cei patru pași de preprocesare a imaginii și cele patru puncte de decizie ale modulului: declanșarea condiționată a măririi imaginii, mecanismul de *fallback* pe decupajul color, validarea formatului și votarea temporală, respectiv verificarea autorizării. Subcapitolele care urmează detaliază, în ordinea execuției, fiecare etapă a acestui flux.

4.4.1 Detecția plăcuțelor cu YOLOv8

Localizarea plăcuțelor în cadru este realizată cu ajutorul unui model YOLOv8, instanțiat prin intermediul clasei YOLO din pachetul `ultralytics`, pe baza fișierului de ponderi `models/license_plate/best.pt`. Modelul de bază utilizat este `harshitsingh09/yolov8-license-plate-detector`, o variantă a arhitecturii YOLOv8 publicată pe platforma Kaggle și dedicată exclusiv detecției plăcuțelor de înmatriculare (având definită o singură clasă, `license_plate`). Modelul a fost integrat în sistem în forma sa originală, fără a necesita o etapă suplimentară de antrenare (ajustare fină) în cadrul acestui proiect. La momentul inițializării, pentru a optimiza viteza de inferență, sistemul verifică disponibilitatea mediului CUDA și, în caz afirmativ, transferă execuția modelului pe unitatea GPU.

Pentru fiecare cadru procesat, metoda `process_frame()` apelează funcția `predict()` utilizând parametrul `imgsz=640` și un prag de confidență `conf=0.25`, extrăgând astfel casetele de încadrare candidate (*bounding boxes*). Fiecare casetă, definită prin coordonatele `[x1, y1, x2, y2]`, este ulterior *extinsă* cu 20% în toate direcțiile prin intermediul funcției `expand_bbox()` (utilizând un coeficient `pad_ratio` de 0.20). Această margine de siguranță mai generoasă garantează includerea limitelor fizice ale plăcuței alături de un context vizual minim, prevenind astfel decuparea accidentală a caracterelor aflate la extremități înaintea evaluării OCR. Regiunea decupată rezultată este transmisă etapelor ulterioare de prelucrare.

4.4.2 Up-scaling-ul plăcuțelor mici

Deoarece plăcuțele detectate prezintă frecvent dimensiuni insuficiente pentru o recunoaștere OCR precisă, regiunea decupată este procesată de funcția `upscale_if_small()`, având ca scop redimensionarea la o lățime țintă de 250 px. Algoritmul aplică următoarea regulă: dacă lățimea curentă se situează sub acest prag, se calculează un factor de scalare $scale = 250/w$. Acesta este limitat superior la valoarea 3.0 pentru a preveni amplificarea excesivă a zgomotului de imagine. Redimensionarea propriu-zisă utilizează interpolarea bicubică (`cv2.INTER_CUBIC`), selectată datorită calității superioare a reconstrucției detaliilor în procesul de suprascalară. Casetele care îndeplinesc deja criteriul de lățime minimă rămân nemodificate. Printr-o astfel de operațiune, o regiune inițială de 116×33 px este transformată într-o imagine de aproximativ

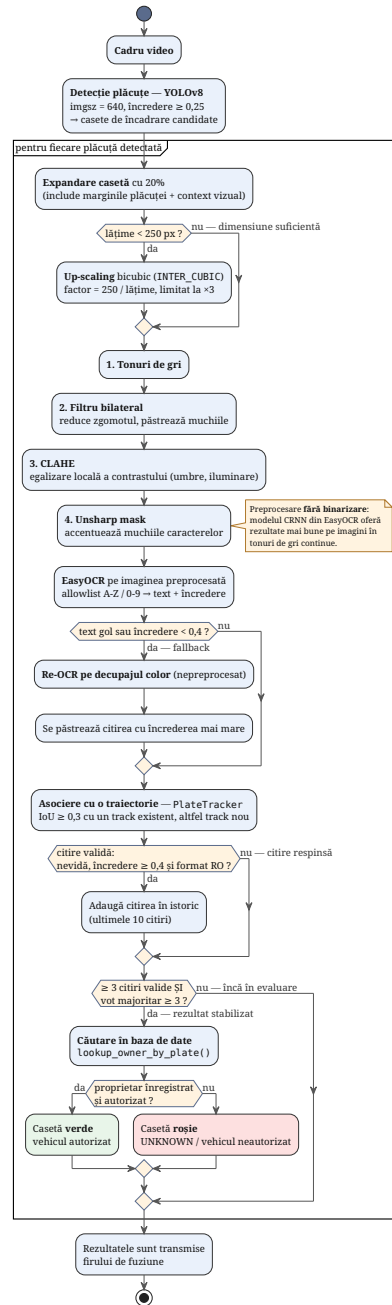


Figura 4.3: Fluxul de procesare al modulului de recunoaștere a plăcuțelor. Fiecare plăcuță detectată de YOLOv8 este decupată cu o margine de context, mărită dacă este prea mică și supusă unui lanț de patru operații de preprocesare. Doar după consensul pe minimum trei cadre se interoghează baza de date și se decide autorizarea (casetă verde) sau respingerea (casetă roșie).

250 × 70 px, rezoluție la care performanța algoritmului OCR atinge un nivel optim de fiabilitate.

4.4.3 Pipeline-ul de preprocesare pentru OCR

Imaginea mărită este ulterior pregătită pentru etapa de recunoaștere optică a textului (OCR) prin intermediul funcției `preprocess_plate()`. Aceasta aplică o secvență de patru operații, optimizată special pentru a maximiza performanța algoritmului EasyOCR în identificarea plăcuțelor de înmatriculare:

1. **Conversie în tonuri de gri** – imaginea color este transformată în grayscale;
2. **Filtru bilateral** (`cv2.bilateralFilter` cu $d = 11$ și sigma de culoare și de spațiu de 17) – reduce zgomotul, păstrând în același timp muchiile caracterelor;
3. **Egalizare locală de histogramă CLAHE** (`clipLimit=2.0, tileGridSize=(8, 8)`) – compensează umbrele și iluminarea neuniformă;
4. **Unsharp mask** – accentuează muchiile caracterelor după mărire, construit ca diferență ponderată între imaginea originală (coeficient 1,5) și o versiune blurată Gaussian cu $\sigma = 1,0$ (coeficient -0,5).

O decizie deliberată este aceea de *a nu binariza* imaginea: modelul CRNN folosit de EasyOCR oferă rezultate mai bune pe imagini continue în tonuri de gri decât pe imagini cu prag aplicat. Pipeline-ul se oprește, prin urmare, la o imagine grayscale curățată și accentuată, nu la o imagine alb-negru.

4.4.4 Apelul EasyOCR

Citirea propriu-zisă a caracterelor este realizată de EasyOCR în metoda `_run_ocr()`. Apelul `reader.readtext()` este parametrizat specific pentru plăcuțe:

- `allowlist` restrânge alfabetul recunoscut la literele mari A-Z și cifrele 0-9, eliminând caracterele imposibile pe o plăcuță;
- `text_threshold=0.6`, `low_text=0.3` și `mag_ratio=1.5` reglează sensibilitatea detecției de text și mărirea internă a EasyOCR;
- `paragraph=False` dezactivează gruparea în paragrafe, nepotrivită pentru un singur rând de caractere.

Întrucât EasyOCR poate returna mai multe fragmente de text (de exemplu, grupul de litere și cel de cifre separat), fragmentele sunt sortate de la stânga la dreapta după coordonata x a colțului stânga-sus, apoi sunt curățate (păstrând doar caractere alfanumerice, transformate în majuscule) și concatenate într-un singur șir. Încrederea raportată este media aritmetică a încrederilor fragmentelor componente.

4.4.5 Fallback pe imaginea color

Preprocesarea agresivă (grayscale, filtrare, CLAHE, unsharp) îmbunătățește citirea în majoritatea cazurilor, dar în anumite condiții de iluminare poate degrada rezultatul. Pentru a acoperi aceste situații, pipeline-ul include un mecanism de *fallback*: dacă citirea de pe imaginea preprocesată este goală sau are o încredere sub 0.4, OCR-ul este reluat pe decupajul color original (nepreprocesat). Dacă această a doua citire are o încredere mai mare, ea o înlocuiește pe prima. Astfel, sistemul nu rămâne blocat pe o preprocesare care, ocazional, dăunează în loc să ajute.

4.4.6 Tracking-ul plăcuțelor între cadre

Extragerea textului dintr-un singur cadru este o abordare predispusă la erori, deoarece algoritmul OCR poate confunda cu ușurință caractere similare din punct de vedere vizual (precum 0 și O, sau B și 8) de-a lungul cadrelor succesive. Pentru a garanta stabilitatea rezultatului, fiecare detecție este monitorizată temporal prin intermediul clasei `PlateTracker`. Asocierea unei noi detecții cu o traiectorie existentă (*track*) se realizează evaluând suprapunerea spațială: se calculează metrica *IoU* (*Intersection over Union*, prin apelul funcției `_iou()`) între caseta de încadrare curentă și casetele traiectoriilor active. Detecția este atribuită traiectoriei care prezintă valoarea maximă a metricei *IoU*, cu condiția ca aceasta să depășească pragul minim definit de `iou_threshold=0.3`. În situația în care nu se identifică nicio corespondență validă, sistemul inițializează o nouă traiectorie, alocându-i un identificator unic.

Fiecare traiectorie menține un istoric al citirilor recente, stocat într-o structură de date liniară cu capacitate limitată (un obiect `deque` configurat cu parametrul `history=10`). Suplimentar, entitatea folosește parametrul de stare `age`, care contorizează numărul de cadre consecutive în care respectiva traiectorie nu a primit actualizări. La procesarea fiecărui cadru, metoda `age_tracks()` incrementează acest contor de inactivitate pentru toate traiectoriile monitorizate, eliminându-le automat pe cele care depășesc pragul `max_age=15` cadre. Acest mecanism asigură oprirea urmăririi pentru vehiculele care au părăsit câmpul vizual al camerei.

4.4.7 Validarea formatului

Înainte de a fi inclusă în procesul decizional de agregare, fiecare citire este validată în raport cu formatul standard al plăcuțelor de înmatriculare din România, prin intermediul expresiei regulate `ROMANIAN_PLATE_RE`. Această regulă acceptă configurații formate din una sau două litere pentru indicativul județului, urmate de două sau trei cifre și un grup final de trei litere (de exemplu, `CJ12ABC` sau, specific pentru București, `B123ABC`). În cadrul metodei `PlateTracker.update()`, un rezultat OCR este adăugat în istoricul instanței de urmărire exclusiv dacă îndeplinește cumulativ trei condiții: este un șir nevid, prezintă un scor de confidență de minimum 0.4 și respectă strict formatul impus. Rezultatele

incorect formatate (provenite, în general, din erori ale modulului OCR) sunt astfel filtrate anterior etapei de evaluare, contribuind la o creștere semnificativă a robusteții rezultatului final.

4.4.8 Votarea temporală

Determinarea rezultatului final pentru o traiectorie se realizează printr-un proces de *selecție majoritară* aplicat pe istoricul citirilor valide. Această logică de agregare, integrată în metoda `PlateTracker.update()`, se declanșează exclusiv după ce instanța de urmărire a înregistrat un minim de `min_votes=3` citiri. Datele candidate sunt centralizate utilizând o structură de tip `Counter`, în cadrul căreia fiecare text cumulează suma scorurilor de confidență aferente. Șirul de caractere cu scorul total maxim este desemnat câștigător; dacă numărul recunoașterilor individuale care îl susțin atinge pragul `min_votes`, acesta devine rezultatul consolidat (starea `emitted`) al traiectoriei, având un scor de confidență final calculat ca media aritmetică a scorurilor individuale.

Listing-ul 4.2 prezintă metoda `PlateTracker.update()`, care reunește cele trei etape descrise mai sus: asocierea prin IoU cu o traiectorie, filtrarea citirilor după formatul românesc și votul majoritar pe istoric.

```

1 def update(bbox, text, conf):
2     # Asociaza detectia cu un track existent prin IoU (prag 0.3)
3     best_id, best_iou = None, IOU_THRESHOLD
4     for tid, t in tracks.items():
5         i = iou(bbox, t["bbox"])
6         if i > best_iou:
7             best_id, best_iou = tid, i
8
9     # niciun track: creeaza unul nou
10    if best_id is None:
11        best_id = next_id()
12        tracks[best_id] = {"bbox": bbox, "age": 0,
13                           # ultimele 10 citiri
14                           "readings": deque(maxlen=HISTORY),
15                           "emitted": None, "emitted_conf": 0.0}
16
17    t = tracks[best_id]
18    # reseteaza contorul de inactivitate
19    t["bbox"], t["age"] = bbox, 0
20
21    # Pastreaza citirea doar daca e nevida, conf >= 0.4 SI respecta formatul
22    ↪ RO
23    if text and conf >= 0.4 and ROMANIAN_PLATE_RE.match(text):
24        t["readings"].append((text, conf))
25
26    # Vot majoritar dupa cel putin min_votes (3) citiri valide
27    if len(t["readings"]) >= MIN_VOTES:
28        scores = Counter()
29        for txt, c in t["readings"]:
30            # cumuleaza scorurile de confidenta
31            scores[txt] += c
32        best_text, total_conf = scores.most_common(1)[0]
33        votes = sum(1 for txt, _ in t["readings"] if txt == best_text)
34        if votes >= MIN_VOTES:
35            # rezultat consolidat (stabilizat)
36            t["emitted"] = best_text
37            # media confidentei sustinatoare
38            t["emitted_conf"] = total_conf / votes
39
40    return t["emitted"], t["emitted_conf"]

```

Listing 4.2: Votarea temporală a plăcuțelor (PlateTracker.update din license_plate_recognition_system.py).

Interogarea bazei de date este inițiată strict după ce o traiectorie generează o citire stabilizată: funcția `lookup_owner_by_plate()` verifică existența unui proprietar, iar validarea unei înregistrări atestă statutul de vehicul autorizat. Plăcuțele aflate în stadiu de evaluare sau care nu figurează în sistem sunt etichetate drept UNKNOWN (sau "Unknown car"). La nivelul interfeței vizuale, vehiculele autorizate sunt delimitate printr-o casetă verde, în timp ce vehiculele neautorizate sau necunoscute sunt marcate cu roșu. Acest mecanism de agregare temporală garantează că o plăcuță este validată și supusă verificării doar

atunci când există un consens pe mai multe cadre consecutive asupra aceluiași text, eliminând astfel cu succes erorile de citire tranzitorii.

4.5 Modulul de detecție foc și fum

Modulul de detecție a focului și a fumului identifică în fluxul video apariția flăcărilor și a emisiilor de fum, facilitând declanșarea unor alerte timpurii în scenariile de incendiu. Acesta este implementat în clasa `FireDetectionSystem` din fișierul `fire_detection_system.py` și operează într-un fir de execuție dedicat (*Thread 4*).

Provocarea principală a acestui modul nu constă în detecția propriu-zisă, ci în controlul strict al rezultatelor fals pozitive: focul și, în special, fumul, reprezintă categorii vizuale cu un grad ridicat de ambiguitate (fiind predispuse la confuzii cu pereți gri, formațiuni noroase, abur, reflexii calde sau lumina apusului). O detecție primară obținută de la un model YOLO, dacă ar fi utilizată direct pentru declanșarea unei alarme, ar genera un volum inacceptabil de avertizări false. Din acest motiv, implementarea suprapune peste modelul de bază un *mecanism de filtrare structurat pe cinci niveluri*, care transformă detecțiile candidate în alerte finale exclusiv după ce acestea îndeplinesc, în ordine succesivă, condițiile tuturor filtrelor. Subcapitolele următoare descriu modelul de bază, cele cinci etape de filtrare și, în final, semnificația indicatorilor de stare `confirmed` și `alert` asociați fiecărei detecții.

Figura 4.4 ilustrează acest mecanism sub forma unei cascade de filtrare: fiecare detecție candidată furnizată de YOLOv10 traversează, în ordine, cele cinci niveluri, fiind eliminată de îndată ce nu satisface un criteriu. Diagrama evidențiază ce categorie de detecții este respinsă la fiecare nivel, precum și derivarea finală a celor doi indicatori de stare – `confirmed` (eveniment real, afișat pe cadru) și `alert` (moment de declanșare a alarmei). Subcapitolele care urmează descriu modelul de bază și fiecare dintre cele cinci filtre.

4.5.1 Modelul YOLOv10 pre-antrenat

Modelul de bază utilizat este `TommyNgx/YOLOv10-Fire-and-Smoke-Detection`, o variantă a arhitecturii YOLOv10 publicată pe platforma HuggingFace și antrenată specific pentru cele două clase de interes: `Fire` și `Smoke`. Modelul este integrat în sistem în forma sa originală, nefiind necesară o etapă suplimentară de reantrenare (ajustare fină), și este instanțiat prin intermediul clasei `YOLO` (din pachetul `ultralytics`) pe baza fișierului de ponderi `models/fire_and_smoke/best.pt`. Imediat după încărcare, denumirile claselor sunt normalizate prin convertirea literelor în minuscule (`fire`, `smoke`), asigurând astfel o utilizare consecventă în restul fluxului de prelucrare.

La nivelul fiecărui cadru, metoda `process_frame()` execută funcția `model.predict()` utilizând un prag de confidentă egal cu cea mai mică valoare dintre pragurile definite per clasă, alături de un prag IoU de 0.45 pentru algoritmul de suprimare a non-maximelor (NMS). Această abordare a fost aleasă

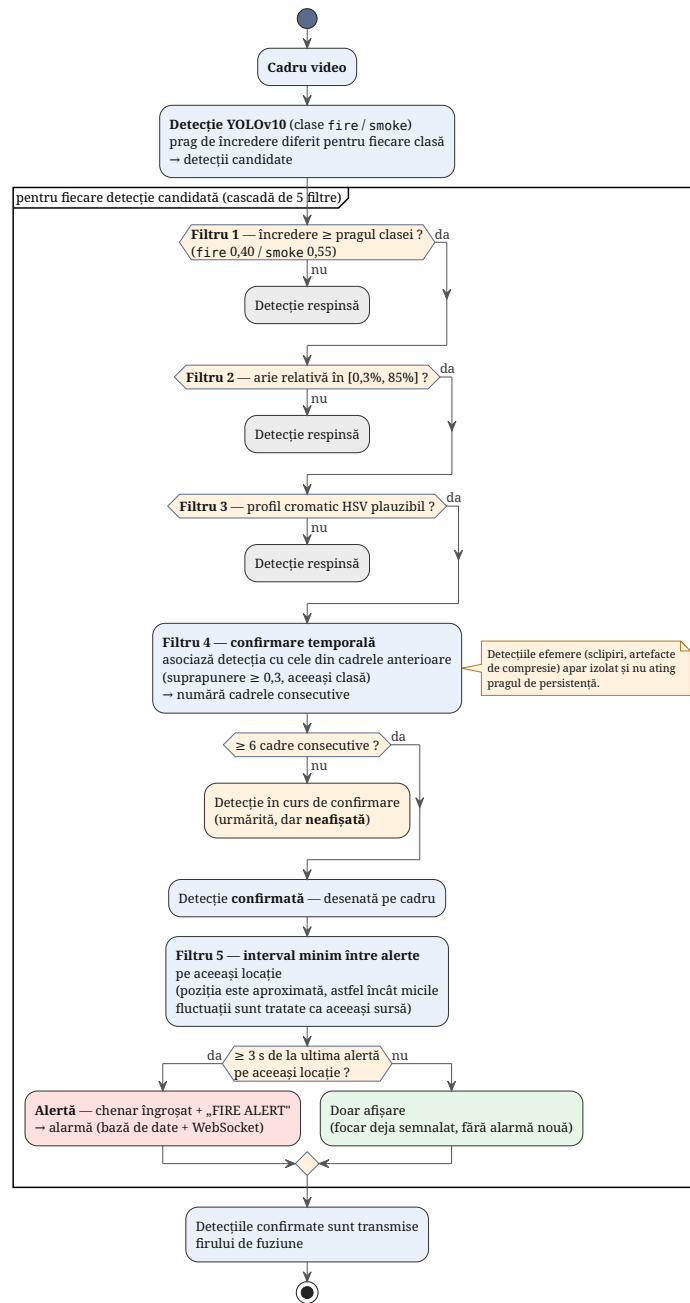


Figura 4.4: **Cascada de filtrare foc/fum.** Detecțiile YOLOv10 parcurg cinci etape: încredere, dimensiune, analiză HSV, agregare temporală IoU și *cooldown*. Rezultatele sunt progresiv respinse (gri), în așteptare (portocaliu), confirmate vizual (verde) sau marcate ca alertă (roșu). Indicatorii **confirmed** și **alert** separă starea validată de evenimentul declanșat.

în mod deliberat: modelului i se permite să returneze toate detecțiile candidate care depășesc pragul minim de confidentă, urmând ca filtrarea riguroasă la nivel de clasă să fie aplicată ulterior, în mod explicit, de către mecanismul de procesare. Această arhitectură oferă un control deplin asupra procesului decizional de declanșare a alertelor.

4.5.2 Pipeline-ul de filtrare în cinci niveluri

Fiecare rezultat candidat furnizat de modelul YOLOv10 parcurge, în mod secvențial, cinci niveluri de filtrare. Pentru ca o detecție să fie validată și transformată în alertă, aceasta trebuie să îndeplinească cerințele tuturor celor cinci etape; neîndeplinirea criteriilor la oricare dintre aceste niveluri atrage respingerea automată a detecției (sau, în cazul specific al filtrului temporal, reținerea acesteia în starea de detecție aflată în curs de confirmare). Sistemul menține o serie de statistici interne (precum `rejected_by_size`, `rejected_by_color` etc.) pentru a contoriza cu precizie numărul detecțiilor invalidate la nivelul fiecărui filtru.

Listing-ul 4.3 sintetizează cascada celor cinci filtre aplicată fiecărui candidat YOLOv10 în `process_frame()`. Etapele 3, 4 și 5 sunt detaliate în subsecțiunile următoare, împreună cu Listing-urile 4.4 și 4.5.

```

1 # candidati YOLOv10
2 for box in yolo_results.bboxes:
3     x1, y1, x2, y2 = clamp_to_frame(box.xyxy)
4     class_name, conf = box.class_name, box.conf
5
6     # Filtru 1: prag de încredere diferențiat pe clasă (fire 0.40 / smoke
7     ↪ 0.55)
8     if conf < CONF_THRESHOLDS[class_name]:
9         continue
10
11     # Filtru 2: plauzibilitate dimensională (între 0.3% și 85% din cadru)
12     area_ratio = (x2 - x1) * (y2 - y1) / frame_area
13     if not (MIN_AREA_RATIO <= area_ratio <= MAX_AREA_RATIO):
14         stats["rejected_by_size"] += 1
15         continue
16
17     # Filtru 3: verificare cromatică euristică în HSV
18     if not color_plausible(frame[y1:y2, x1:x2], class_name):
19         stats["rejected_by_color"] += 1
20         continue
21
22     # Filtru 4: confirmare temporală prin tracking IoU
23     consecutive = update_frame_history(class_name, (x1, y1, x2, y2),
24     ↪ matched_keys)
25     # 6 cadre consecutive
26     confirmed = consecutive >= MIN_CONSECUTIVE_FRAMES
27
28     # Filtru 5: cooldown de alertă pe locație (3 s)
29     should_alert = confirmed and alert_cooldown_expired(class_name, x1, y1)
30
31     detections.append({"bbox": (x1, y1, x2, y2), "class": class_name,
32     ↪ "confirmed": confirmed, "alert": should_alert})

```

Listing 4.3: Cascada celor cinci filtre aplicată fiecărui candidat (`process_frame` din `fire_detection_system.py`).

4.5.2.1 Praguri de încredere per clasă

Primul nivel aplică un prag de încredere *diferențiat pe clasă*: 0.40 pentru `fire` și 0.55 pentru `smoke`. Pragul mai sever pentru fum reflectă faptul că acesta este semnificativ mai predispus la false pozitive (suprafețe gri, nori, abur) decât focul, care are o semnătură vizuală mai distinctivă. O detecție a cărei încredere este sub pragul clasei sale este eliminată imediat.

4.5.2.2 Filtre de dimensiune

Cel de-al doilea nivel de filtrare evaluează plauzibilitatea dimensională a casetei de încadrare, raportând suprafața acesteia la aria totală a cadrului. Sunt respinse atât detecțiile cu o suprafață neglijabilă (sub 0.3% din imagine, care reprezintă de obicei zgomot vizual sau reflexii punctuale), cât și cele disproporționate de mari (peste 85% din cadru, indicând tipic o clasificare eronată care se extinde

pe aproape întreaga suprafață vizuală). Astfel, sunt validate și transmise etapei următoare exclusiv detecțiile a căror arie relativă se încadrează în intervalul matematic $[0,003, 0,85]$.

4.5.2.3 Verificarea cromatică HSV

Cel de-al treilea nivel de filtrare constă într-o verificare cromatică euristică, implementată prin intermediul metodei `_color_plausible()`. Aceasta operează în spațiul de culoare HSV, exploatând faptul că focul și fumul prezintă semnături cromatice specifice:

- **Foc:** regiunea analizată trebuie să conțină o pondere semnificativă de pixeli cu nuanțe calde, un grad înalt de saturație și o intensitate luminoasă ridicată. Mai exact, se caută valori ale nuanței (*hue*) din spectrul roșu-portocaliu-galben (situate în intervalele $h \leq 25$ sau $h \geq 160$, compensând astfel dispunerea circulară a valorilor de nuanță în jurul originii), asociate cu o saturație $s \geq 90$ și o luminozitate $v \geq 120$. Condiția de validare impune ca minimum 12% din pixelii regiunii să îndeplinească aceste criterii;
- **Fum:** regiunea trebuie să se caracterizeze printr-o saturație redusă și o luminozitate moderată. Prin urmare, cel puțin 55% din pixeli trebuie să aparțină spectrului cenușiu (având o saturație $s \leq 60$ și o luminozitate $v \geq 40$, limită inferioară impusă pentru a exclude zonele de umbră absolută). Suplimentar, abaterea standard a canalului de luminozitate trebuie să depășească valoarea de 6, o constrângere menită să respingă suprafețele cu o cromatică perfect uniformă (precum cerul senin sau un perete neted), în cadrul cărora un nor de fum real ar induce, în mod inerent, o variație a intensității.

Detecțiile care nu satisfac profilul cromatic specific clasei prezise sunt respinse automat. Această etapă elimină o proporție considerabilă a rezultatelor fals pozitive care ar fi putut valida primele două niveluri de filtrare.

4.5.2.4 Confirmarea temporală prin tracking IoU

Cel de-al patrulea nivel de filtrare impune o condiție de persistentă temporală: o detecție devine *confirmată* exclusiv dacă este identificată în mod constant pe parcursul a cel puțin 6 cadre consecutive. Pentru a asigura trasabilitatea unei detecții între cadre succesive, metoda `_update_frame_history()` corelează caseta de încadrare curentă cu înregistrările din istoricul temporal, utilizând ca metrică suprapunerea spațială ($\text{IoU} \geq 0,3$, calculată prin funcția `_compute_iou()`), cu obligativitatea apartenenței la aceeași clasă. În cazul identificării unei corespondențe, contorul de apariții consecutive al înregistrării este incrementat; în caz contrar, sistemul inițializează o intrare nouă, având contorul setat la valoarea 1. Instanțele care nu mai sunt detectate în cadrul analizat sunt invalidate și eliminate din istoric, întrerupând astfel continuitatea

secvenței. Acest mecanism exclude cu succes detecțiile efemere (cum ar fi scîlpirile luminoase de scurtă durată sau artefactele de compresie video), care apar izolat și nu pot indica prezența unui incendiu real.

Listing-ul 4.4 detaliază mecanismul de corelare temporală implementat în metoda `_update_frame_history()`: caseta de încadrare curentă este asociată cu o detecție din istoric pe baza metricii IoU (*Intersection over Union*), fiind condiționată de apartenența la aceeași clasă. În cazul unei asocieri valide, contorul de apariții consecutive este incrementat; în caz contrar, sistemul inițializează o nouă intrare, având contorul setat la valoarea 1.

```

1 def update_frame_history(class_name, bbox, matched_keys):
2     # Cauta in istoric caseta cu cea mai mare suprapunere (IoU), aceeași
    ↪ clasa
3     best_key, best_iou = None, 0.0
4     for key, entry in frame_history.items():
5         if key in matched_keys or not key.startswith(class_name + "_"):
6             continue
7         iou = compute_iou(bbox, entry["bbox"])
8         if iou > best_iou:
9             best_iou, best_key = iou, key
10
11     # 0.3
12     if best_key is not None and best_iou >= IOU_MATCH_THRESHOLD:
13         # streak++
14         frame_history[best_key]["count"] += 1
15         frame_history[best_key]["bbox"] = bbox
16         frame_history[best_key]["last_seen_frame"] = current_frame_number
17         matched_keys.add(best_key)
18         return frame_history[best_key]["count"]
19
20     # Nicio corespondenta: intrare noua, contor initializat la 1
21     new_key = f"{class_name}_{current_frame_number}_{id(bbox)}"
22     frame_history[new_key] = {"bbox": bbox, "count": 1,
23                               "last_seen_frame": current_frame_number}
24     matched_keys.add(new_key)
25     return 1

```

Listing 4.4: Corelarea temporală a detecțiilor prin IoU (`_update_frame_history` din `fire_detection_system.py`).

4.5.2.5 Cooldown de alertă pe locație

Cel de-al cincilea nivel de filtrare previne saturarea sistemului cu alerte redundante generate de același eveniment. Odată ce o detecție dobândește starea de *confirmată*, o nouă alertă este emisă exclusiv dacă a expirat un interval de timp prestabilit de la ultima notificare asociată *aceleiași locații*. Identificarea spațială este aproximată printr-o cheie de indexare simplificată, construită din eticheta clasei și coordonatele punctului de origine, divizate printr-o împărțire întreagă la un factor de 40 (`class_name_x1//40_y1//40`). Această cuantificare a spațiului asigură faptul că detecțiile care prezintă fluctuații minore de poziție

sunt atribuite aceleași surse. Intervalul minim de latență între două alerte consecutive pentru o locație identică este setat la 3 secunde. Prin această metodă, un eveniment persistent declanșează o alertă inițială, urmată de notificări recurente controlate, evitându-se astfel o suprasarcină de avertizări la procesarea fiecărui cadru.

4.5.3 Flag-urile *confirmed* și *alert*

Fiecare predicție returnată de metoda `process_frame()` este însoțită de doi indicatori de stare (valori booleene), care codifică rezultatul fluxului de filtrare și care îndeplinesc roluri funcționale distincte:

- **confirmed**: capătă valoarea de adevăr atunci când detecția a validat primele patru niveluri și a atins pragul de persistentă temporală (minimum 6 cadre consecutive). O astfel de predicție este tratată drept un eveniment real și este *reprezentată grafic* pe cadru; rezultatele candidate neconfirmate sunt ascunse, prevenind astfel fluctuația vizuală (afișarea intermitentă) a casetelor incerte;
- **alert**: este evaluat ca adevărat exclusiv în cadrul în care o predicție confirmată îndeplinește și condiția intervalului de latență, semnalând momentul exact pentru declanșarea unei *alarme* (spre exemplu, înregistrarea în baza de date și transmiterea unei notificări prin protocolul WebSocket). La nivelul interfeței, o detecție marcată prin indicatorul **alert** este evidențiată printr-un chenar îngroșat, fiind însoțită de un mesaj de avertizare de tipul **FIRE ALERT** suprapus pe cadru.

Această decuplare a indicatorilor separă *starea* sistemului (prezența validată a focului sau a fumului în cadrul curent) de *evenimentul* de notificare (necesitatea declanșării imediate a unei alarme). Abordarea permite afișarea continuă a focurilor confirmate, eliminând totodată riscul generării unor alerte redundante.

Listing-ul 4.5 ilustrează modul în care sunt evaluați cei doi indicatori în cadrul metodei `process_frame()`, pentru o detecție care a trecut deja de cele trei filtre preliminare (nivel de încredere, dimensiune și validare HSV). Indicatorul **confirmed** este activat odată cu atingerea pragului de persistentă temporală, în timp ce parametrul **alert** impune, suplimentar, respectarea unui interval de latență (*cooldown*) asociat locației respective. La finalul prelucrării fiecărui cadru, intrările din istoric care nu au fost reconfirmate prin asocieri noi sunt eliminate, întrerupând astfel secvența aparițiilor consecutive.

```

1 # Detectie care a trecut filtrele 1-3 (incredere, dimensiune, HSV):
2 consecutive = update_frame_history(class_name, (x1, y1, x2, y2),
  ↳ matched_keys)
3 # 6 cadre consecutive
4 confirmed = consecutive >= MIN_CONSECUTIVE_FRAMES
5
6 should_alert = False
7 if confirmed:
8     # Cheie de locatie cuantizata, ca fluctuatiile mici sa fie aceeasi sursa
9     alert_key = f"{class_name}_{x1 // 40}_{y1 // 40}"
10    last = last_alert_time.get(alert_key)
11    # 3 s
12    if last is None or (now - last).total_seconds() > ALERT_COOLDOWN:
13        should_alert = True
14        last_alert_time[alert_key] = now
15
16 # eveniment real: se deseneaza pe cadru
17 detection["confirmed"] = confirmed
18 # momentul de declansare a alarmei
19 detection["alert"] = should_alert
20
21 # La finalul cadrului: intrarile nevazute acum sunt sterse (streak intrerupt)
22 for k in [k for k, v in frame_history.items()
23          if v["last_seen_frame"] < current_frame_number]:
24     del frame_history[k]

```

Listing 4.5: Derivarea indicatorilor confirmed și alert (process_frame din fire_detection_system.py).

4.6 Modulul de detecție a armelor (model ajustat fin propriu)

Spre deosebire de componentele anterioare, care integrează modele pre-antrenate în forma lor originală, modulul dedicat detecției de arme se fundamentează pe o arhitectură YOLOv8 reantrenată (ajustată fin) integral în cadrul acestui proiect, utilizând un set de date construit special pentru această sarcină. Această decizie a fost motivată de absența unui model public suficient de robust pentru identificarea generică a armelor în scenariile de supraveghere video. Prezenta secțiune detaliază întregul flux de dezvoltare, pornind de la deciziile de proiectare (precum definirea unei singure clase de interes și selecția arhitecturii de bază), continuând cu etapa de colectare și uniformizare a setului de date, și finalizându-se cu parametrizarea procesului de antrenare și evaluarea rezultatelor obținute. Fluxul de antrenare este structurat modular în patru scripturi, executate secvențial: 01_download_datasets.py, 02_relabel_and_merge.py, 03_train_yolo.py și 04_evaluate_test.py.

Figura 4.5 oferă o imagine de ansamblu a întregului flux, de la cele două seturi de date sursă până la modelul antrenat, evaluarea lui pe partiția de test și utilizarea la inferență, structurat pe cele patru scripturi menționate. Subca-

pitolele care urmează detaliază fiecare etapă: justificarea clasei unice, selecția modelului de bază, seturile de date, preprocesarea și unificarea lor, parametrii de antrenare și rezultatele obținute.

4.6.1 Justificarea unei singure clase *weapon*

Modelul este antrenat să recunoască o singură clasă generică, **weapon**, care acoperă deopotrivă arme de foc (pistoale, puști), arme albe (cuțite, macete) și obiecte contondente (bâte de baseball). Decizia de a grupa toate tipurile de arme într-o singură clasă este motivată practic: în contextul unui sistem de supraveghere, ceea ce contează este *prezența* unei arme în cadru, nu categoria ei exactă. Această abordare reduce problema de clasificare la una de detecție binară (armă / non-armă), permite modelului să generalizeze mai bine pe date noi și elimină confuziile între subtipuri de arme, care oricum ar declanșa același tip de alertă.

4.6.2 Selectarea modelului de bază

Modelul de bază ales pentru antrenarea suplimentară (ajustare fină) este **YOLOv8m** (varianta *medium*), pre-antrenat pe setul de date COCO. Această variantă oferă un echilibru bun între acuratețe și viteză de inferență: este suficient de complexă pentru a învăța formele variate ale armelor, dar suficient de ușoară pentru a rula în timp real, alături de celelalte modele, pe același GPU. Procesul pornește de la ponderile COCO (`pretrained=True`), beneficiind astfel de trăsăturile vizuale generice deja învățate, pe baza cărora detectorul se adaptează la noua clasă.

4.6.3 Seturile de date utilizate

Deoarece niciun set de date public nu oferea, în mod individual, o acoperire suficientă pentru diversitatea tipurilor de arme, a condițiilor de iluminare și a perspectivelor necesare, s-a decis combinarea a două seturi de date independente. Acestea au fost preluate automat prin intermediul scriptului `01_download_datasets.py`:

- **Dangerous Items** (Zenodo) – 8.478 de imagini, cu cinci clase originale, toate arme: **machete**, **knife**, **baseball_bat**, **rifle** și **gun**. Imaginile acoperă scene diverse, de la arme prezentate izolat până la arme ținute în mână;
- **SOHAS Weapon Detection** (Roboflow) – 5.858 de imagini, cu șase clase originale: **pistol**, **knife**, **smartphone**, **billete** (bancnote), **monedero** (portofel) și **tarjeta** (card). Ultimele patru clase sunt obiecte non-armă care pot fi ușor confundate cu arme; ele nu sunt păstrate ca o clasă distinctă, ci sunt eliminate din etichete la preprocesare, iar imaginile respective sunt reutilizate ca exemple negative (vezi Secțiunea 4.6.4).

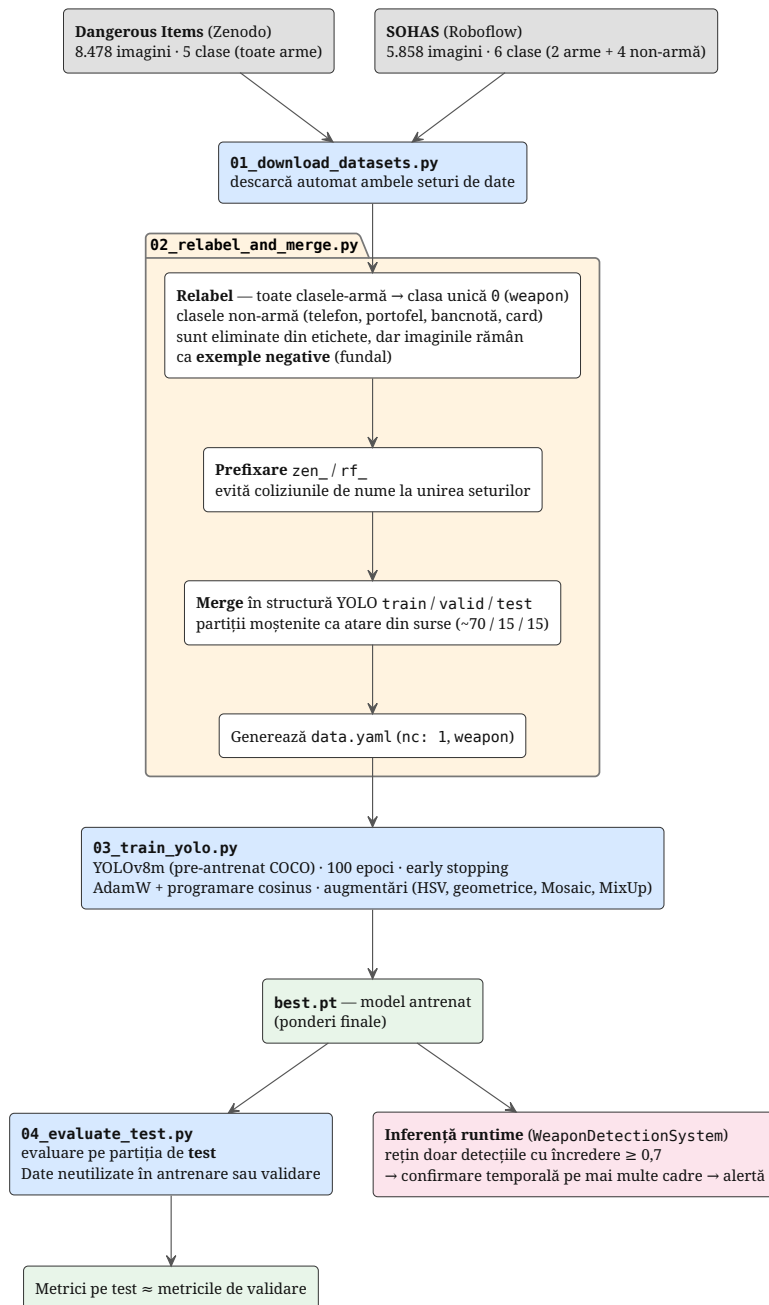


Figura 4.5: Fluxul de date și antrenare a detectorului de arme. Cele patru scripturi execută secvențial: (1) descărcarea seturilor Zenodo și Roboflow; (2) unificarea lor (remapare pe o clasă, reținerea exemplelor negative și formatare YOLO); (3) antrenarea modelului YOLOv8m (**best.pt**); (4) evaluarea pe datele de test. Ponderile validate sunt ulterior integrate în aplicația de supraveghere.

După combinare, setul de date final conține 14.336 de imagini și 12.592 de bounding box-uri, oferind o diversitate suficientă pentru antrenarea unui detector robust.

Figura 4.6 prezintă distribuția adnotărilor (numărul de bounding box-uri) pe categoriile originale ale celor două surse, înainte de remaparea la clasa unică **weapon**. Deoarece clasa **knife** apare în ambele seturi, ea este reprezentată prin două bare distincte (**knife** (Zenodo) și **knife** (SOHAS)), pentru a evidenția contribuția fiecărei surse.

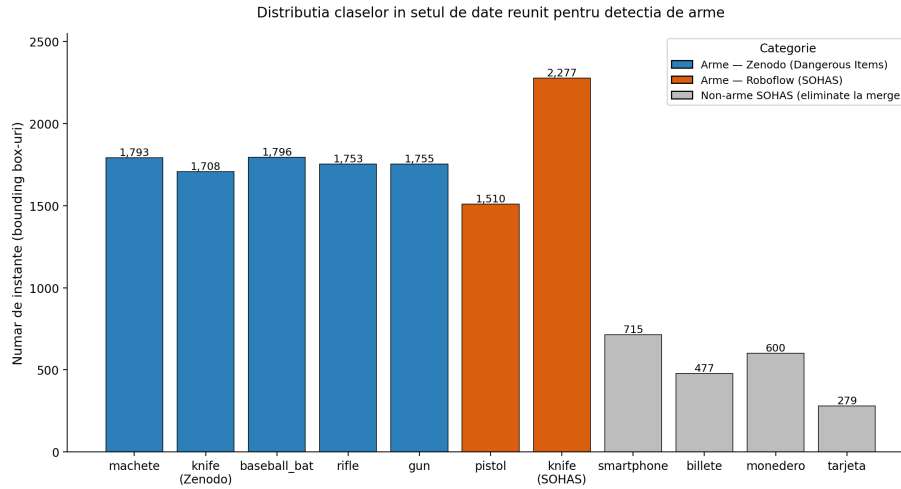


Figura 4.6: Distribuția casetelor de încadrare (*bounding boxes*) pe clasele originale ale celor două seturi de date, anterior remapării la categoria unică **weapon**. Cromatica indică proveniența și tipologia datelor: albastru pentru cele cinci clase de arme din setul Zenodo, portocaliu pentru clasele de arme reținute din SOHAS (**pistol** și **knife**), respectiv gri pentru clasele non-armă din SOHAS (excluse la unificare și reutilizate drept exemple negative). Categoria **knife**, comună ambelor surse, este reprezentată distinct. Valorile afișate deasupra barelor indică volumul total de adnotări, cumulat pentru partițiile de antrenare, validare și testare.

Din analiza distribuției datelor se desprind două observații relevante privind calitatea setului de antrenare. În primul rând, setul Zenodo prezintă o distribuție echilibrată: cele cinci clase de arme beneficiază de un volum de adnotări foarte apropiat (între 1.708 pentru **knife** și 1.796 pentru **baseball_bat**), prevenind astfel dominanța unei categorii specifice în timpul antrenării. La acestea se adaugă, din setul SOHAS, încă 1.510 adnotări pentru clasa **pistol** și 2.277 pentru **knife**. Prin cumularea celor două surse, armele albe (**knife**, totalizând 3.985 de instanțe) și armele de foc (**pistol**, **rifle**, **gun**, totalizând 5.018 instanțe) devin cele mai bine reprezentate categorii, o pondere care reflectă frecvența acestor amenințări în scenariile reale de supraveghere. Însurate, cele

șapte categorii de arme totalizează **12.592** de adnotări – valoare ce reprezintă numărul exact de casete de încadrare (*bounding boxes*) din setul unificat, întrucât adnotările pentru obiectele inofensive nu sunt păstrate ca ținte pozitive.

În al doilea rând, cele patru clase non-armă din SOHAS (**smartphone** – 715, **billete** – 477, **monedero** – 600 și **tarjeta** – 279, însumând 2.071 de obiecte) nu sunt eliminate, ci sunt valorificate strategic drept exemple negative. Imaginile aferente sunt reținute în setul de date fără adnotări (cu fișiere de etichetare vide), instruind astfel modelul să evite clasificarea eronată a obiectelor de mână cu geometrie similară (precum telefoanele sau portofelele), conform mecanismului de preprocesare detaliat în Secțiunea 4.6.4.

4.6.4 Preprocesarea și unificarea

Cele două seturi de date folosesc scheme de etichetare diferite, astfel încât a fost necesară o etapă de preprocesare pentru unificarea lor într-o structură YOLO coerentă, realizată de scriptul `02_relabel_and_merge.py`. Operațiile principale sunt:

- **Relabel** – toate clasele de arme din ambele seturi de date sunt remapate la clasa unică 0 (**weapon**). În cazul Zenodo, toate cele cinci clase sunt arme, deci toate sunt păstrate; în cazul Roboflow, doar **pistol** și **knife** sunt păstrate, iar clasele non-armă (**smartphone**, **billete**, **monedero**, **tarjeta**) sunt *eliminate* din etichete;
- **Merge** – cele două seturi de date sunt unite într-o singură structură YOLO cu trei partiții (**train**, **valid**, **test**); partiția **val** a sursei Zenodo este redenumită **valid** pentru consistență. Important de subliniat: împărțirea în antrenare/validare/testare nu este efectuată în cadrul acestui proiect, ci este *moștenită ca atare* din cele două seturi sursă, ambele livrate deja pre-partiționate (Zenodo cu **train/val/test**, iar exportul YOLOv8 al SOHAS cu **train/valid/test**). Scriptul nu aplică nicio re-partiționare aleatorie – fiecare imagine rămâne în partiția pe care o avea în setul de date de origine, iar partițiile omonime din cele două surse sunt concatenate;
- **Negative samples** – exemplele negative nu necesită niciun tratament special în scriptul de antrenare. O imagine este copiată în setul de date final chiar dacă fișierul ei de etichetă rămâne gol, fie pentru că nu conținea nicio adnotare, fie pentru că singurele adnotări (telefon, portofel, bancnotă, card) au fost eliminate la pasul de *relabel* de mai sus. YOLOv8 interpretează automat orice imagine cu fișier de etichetă gol drept fundal (*background*): la antrenare aceasta nu contribuie cu nicio țintă pozitivă, iar orice detecție de armă pe ea este penalizată ca fals pozitiv. Astfel, aceste imagini învață modelul ce *nu* este o armă și reduc rata de false pozitive la inferență, mai ales pentru obiectele care seamănă cu armele (cazul imaginilor cu telefoane sau portofele din Roboflow, ale căror etichete au fost golite).

Întrucât partițiile sunt preluate din sursele originale, proporțiile finale rezultă din modul în care erau deja împărțite acestea, nu dintr-un raport impus de proiect. Pe cele 14.336 de imagini ale setului de date combinat, distribuția este de aproximativ **70 % antrenare** (10.041 imagini), **15 % validare** (2.104 imagini) și **15 % testare** (2.191 imagini), adică un raport de aproximativ 70/15/15.

La final, scriptul generează automat un fișier `data.yaml` cu structura standard YOLO, declarând o singură clasă (`nc: 1, names: {0: weapon}`).

4.6.5 Strategia de denumire (`zen_` / `rf_`)

O problemă subtilă la unirea celor două seturi de date este coliziunea numelor de fișiere: ambele surse pot conține, de exemplu, o imagine numită `0001.jpg`, care s-ar suprascrie reciproc la copierea în directorul comun. Pentru a o evita, fiecărei imagini și etichete i se adaugă un prefix în funcție de sursă: `zen_` pentru imaginile din Zenodo și `rf_` pentru cele din Roboflow. Astfel, perechile imagine-etichetă rămân corelate (același prefix și același nume de bază), iar fișierele celor două surse coexistă fără pierderi în arborele `train/valid/test` comun.

4.6.6 Parametrii de antrenare

Antrenarea propriu-zisă, realizată de scriptul `03_train_yolo.py`, a folosit următorii parametri principali:

- **Epoci:** 100 de epoci planificate, cu *early stopping* (`patience=20`) – oprire automată dacă metrica de validare nu se mai îmbunătățește timp de 20 de epoci;
- **Batch size:** 16 imagini per batch;
- **Rezoluția de intrare:** 640×640 pixeli, valoarea standard pentru YOLOv8;
- **Optimizator:** AdamW, cu learning rate inițial $lr_0 = 0,001$ și factor final $lr_f = 0,01$, conform unei programări cosinus a ratei de învățare (`cos_lr=True`), și `weight_decay = 0,0005`;
- **Warmup:** 3 epoci de warmup pentru stabilizarea gradientilor la începutul antrenării.

Scriptul include și o rutină de selecție automată a GPU-ului cu cea mai multă memorie liberă (`pick_gpu()`, via `pynvml` sau, în lipsa lui, `nvidia-smi`). Antrenarea a fost realizată pe un GPU NVIDIA A100.

4.6.7 Augmentările folosite

Pentru a crește robustețea modelului și a reduce overfitting-ul, în timpul antrenării au fost aplicate multiple transformări de augmentare a datelor:

- **Variații cromatice HSV** – nuanță ($\pm 0,015$), saturație ($\pm 0,7$) și luminositate ($\pm 0,4$), pentru a simula condiții de iluminare diferite;
- **Transformări geometrice** – rotație până la 10° , translație de 10%, scalare de 50% și oglindire orizontală (`fliplr=0.5`);
- **Mosaic** (`mosaic=1.0`) – combinarea a patru imagini într-una singură, expunând modelul la contexte și scale variate;
- **MixUp** (`mixup=0.1`) – amestecarea a două imagini, ca regularizare suplimentară.

4.6.8 Rezultatele antrenării

Antrenarea a parcurs toate cele 100 de epoci (early stopping nu s-a declanșat). Modelul a fost evaluat în două regimuri: pe partiția de *validare* (monitorizată la fiecare epocă în timpul antrenării) și, separat, pe partiția de *test* – date pe care modelul nu le-a întâlnit niciodată, nici la antrenare, nici la selecția epocii optime. Evaluarea pe partiția de test este realizată separat de scriptul `04_evaluate_test.py`, care încarcă ponderile finale `best.pt` și rulează o singură rundă de validare pe această partiție. Ambele seturi de metrice sunt sintetizate comparativ în tabelul 4.2.

Figura 4.7 ilustrează evoluția procesului de antrenare pe parcursul celor 100 de epoci, prin intermediul celor zece curbe generate automat de framework-ul Ultralytics. În partea superioară sunt prezentate cele trei componente ale funcției de cost calculate pe setul de antrenare (*training set*) — eroarea de localizare a casetelor de delimitare (`box_loss`), eroarea de clasificare (`cls_loss`) și eroarea distribuției casetelor (`dfl_loss`) — împreună cu metricile de precizie și sensibilitate. Partea inferioară conține aceleași componente ale funcției de cost evaluate pe setul de validare (*validation set*), alături de indicatorii `mAP@0,5` și `mAP@0,5:0,95`.

Analiza curbelor evidențiază mai multe aspecte relevante privind comportamentul modelului în timpul antrenării. În primul rând, toate cele șase componente ale funcției de cost, atât pentru setul de antrenare, cât și pentru cel de validare, prezintă o tendință descendentă și se stabilizează către finalul procesului de optimizare. În același timp, valorile costului pe setul de validare nu înregistrează o creștere sistematică, ceea ce sugerează absența fenomenului de supraînvățare (*overfitting*).

În al doilea rând, evoluția curbelor de validare este similară cu cea observată pentru setul de antrenare, indicând o capacitate adecvată de generalizare a modelului asupra datelor nevăzute anterior. O particularitate poate fi observată în jurul epocii 90, unde apare o discontinuitate la nivelul curbelor de cost aferente antrenării. Aceasta este explicată de dezactivarea augmentării de tip *mosaic* în ultimele zece epoci, conform configurației `close_mosaic = 10`. Utilizarea exclusivă a imaginilor originale, mai apropiate de distribuția întâlnită în etapa de inferență, determină o reducere bruscă a valorii funcției de cost și contribuie la rafinarea finală a performanței modelului.

În ceea ce privește metricile de evaluare, acestea înregistrează cea mai accentuată creștere în primele 20–30 de epoci, după care evoluția lor tinde către un regim de platou. Precizia crește de la aproximativ 0,38 la 0,94, sensibilitatea de la 0,36 la aproximativ 0,89, iar mAP@0,5 de la 0,30 la 0,947. De asemenea, faptul că mecanismul de oprire prematură (*early stopping*, configurat cu o toleranță de 20 de epoci) nu s-a declanșat demonstrează că modelul a continuat să își îmbunătățească marginal performanța până la epuizarea epocilor programate.

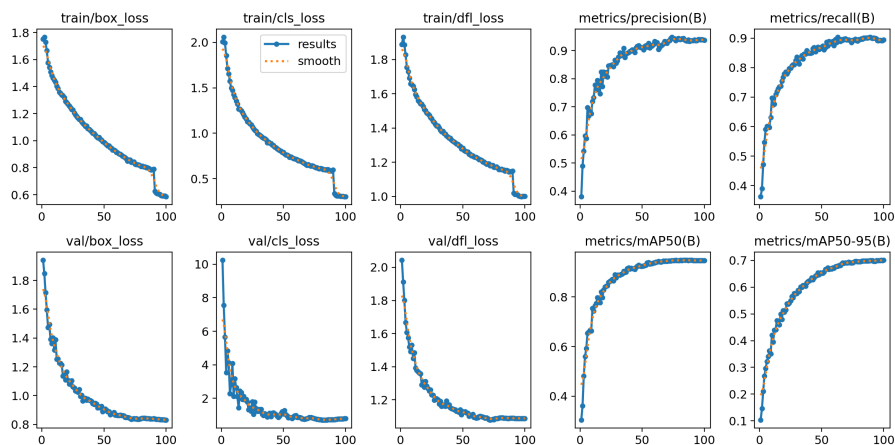


Figura 4.7: Curbele de antrenare ale detectorului de arme pe parcursul celor 100 de epoci. Rândul superior: componentele funcției de cost pe partiția de antrenare (`box_loss`, `cls_loss`, `dfl_loss`), precizia și sensibilitatea. Rândul inferior: aceleași componente de cost pe partiția de validare, împreună cu metricile mAP@0,5 și mAP@0,5:0,95. Punctele reprezintă valorile per epocă, iar linia punctată roșie reprezintă varianta netezită. Scăderea bruscă a costului de antrenare în jurul epocii 90 corespunde dezactivării augmentării *mosaic* în ultimele zece epoci.

Tabel 4.2: Metricile modelului de detecție a armelor, pe partițiile de validare și de test.

Metrică	Validare	Test
Precizie (<i>precision</i>)	0,937	0,943
Sensibilitate (<i>recall</i>)	0,894	0,902
mAP@0,5	0,947	0,943
mAP@0,5:0,95	0,700	0,696

Pe setul de testare (2.191 de imagini nevăzute, dintre care 444 imagini de fundal fără nicio armă), modelul atinge o precizie de 94,3% și o sensibilitate de 90,2%, cu un mAP@0,5 de 0,943. Valoarea mAP@0,5:0,95 (media mAP peste

pragurile IoU cuprinse între 0,5 și 0,95) este de 0,696. Acest rezultat indică o localizare spațială bună, deși mai puțin exactă la pragurile IoU foarte stricte (un comportament tipic pentru detecția obiectelor cu forme și orientări foarte variate, cum sunt armele). În esență, metricile pe test sunt practic identice cu cele de validare (diferențe sub un punct procentual, în ambele sensuri). Aceasta confirmă o bună capacitate de generalizare a modelului și absența fenomenului de supraînvațare (*overfitting*).

Figura 4.8 prezintă relația dintre precizie și sensibilitate sub forma curbei *precision-recall*, obținută prin variația pragului de încredere utilizat în procesul de detecție. Aria de sub această curbă corespunde valorii $\text{mAP}@0,5$, egală cu 0,947. Forma curbei indică o performanță ridicată a modelului pe întregul interval analizat. Se observă că precizia se menține la valori apropiate de 1,0 pentru majoritatea nivelurilor de sensibilitate, înregistrând o scădere accentuată doar atunci când sensibilitatea depășește aproximativ 0,9. Acest comportament sugerează existența unui compromis favorabil între cele două metrici, modelul reușind să atingă valori ridicate ale sensibilității fără o degradare semnificativă a preciziei. Rezultatele sunt în concordanță cu valorile metricilor prezentate anterior și evidențiază capacitatea modelului de a identifica un număr mare de obiecte relevante, menținând în același timp un nivel redus al detecțiilor eronate. Din această perspectivă, modelul poate fi considerat adecvat pentru aplicații de supraveghere și monitorizare, în care limitarea alarmelor false reprezintă o cerință importantă.

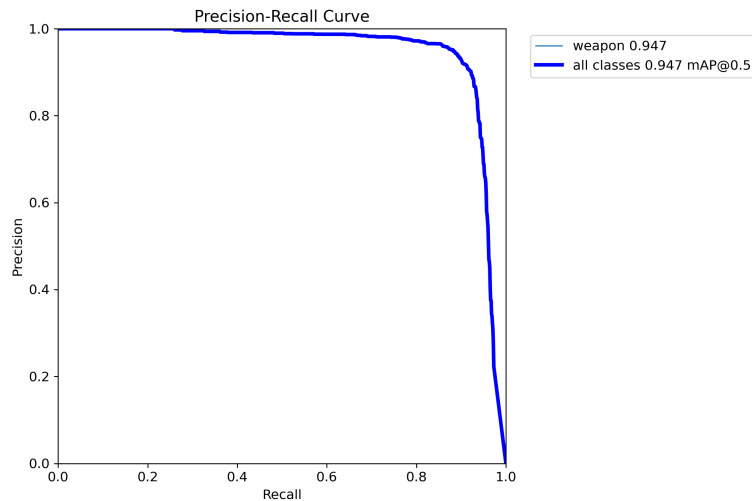


Figura 4.8: Curba precizie-sensibilitate (*precision-recall*) obținută de detectorul de arme pe setul de validare. Aria de sub curbă reflectă valoarea $\text{mAP}@0,5 = 0,947$. Precizia rămâne cvasi-constantă, aproape de nivelul maxim, până la o sensibilitate de $\approx 0,9$, demonstrând capacitatea sistemului de a identifica majoritatea țintelor cu o incidență minimă a alarmelor false.

4.6.9 Discuție asupra rezultatelor

Valorile obținute sunt solide pentru o sarcină dificilă: detecția unei clase vizual eterogene (o armă de foc, un cuțit și o bătă de baseball au foarte puține trăsături comune), pe un set de date combinat din surse cu stiluri de adnotare diferite. Diferența dintre precizie (94,3%) și sensibilitate (90,2%) arată că modelul este ușor mai conservator: ratează unele arme, dar generează relativ puține detecții false. Acesta este un compromis dorit pentru a evita alarmele false într-un sistem de securitate.

Acest comportament este susținut la inferență de o decizie de proiectare la nivelul modulului de rulare (**WeaponDetectionSystem**). Deși modelul produce candidați peste un prag de confidență permisiv (0,3), o detecție este reținută doar dacă trece de un filtru ulterior mai strict de confidență (0,7), iar alerta finală necesită confirmare temporală pe mai multe cadre consecutive (mecanism analog celui de la detecția de foc, detaliat în Secțiunea 4.5.2.4). Astfel, sensibilitatea brută a modelului este sacrificată intenționat în favoarea unei precizii operaționale mai mari, prioritară fiind reducerea falselor alarme. Aportul exemplor negative (telefoane, portofele, carduri din setul de date SOHAS, lăsate intenționat fără adnotări) se reflectă tocmai în această precizie ridicată, modelul fiind forțat să ignore obiectele uzuale care pot fi confundate cu arme.

4.7 Modulul de recunoaștere a acțiunilor umane (ajustare fină SlowFast)

Cel mai complex modul de inteligență artificială al sistemului este cel de recunoaștere a acțiunilor umane (HAR – *Human Action Recognition*), care clasifică *secvențe video* (nu cadre izolate) în trei categorii comportamentale: **normal** (activitate obișnuită), **fight** (lupte, altercații fizice) și **vandalism** (acte de distrugere intenționată a proprietății). Spre deosebire de detectoarele de obiecte, care operează pe un singur cadru, recunoașterea acțiunilor necesită modelarea dimensiunii temporale – aceeași imagine poate aparține atât unei îmbrățișări, cât și unei altercații, distincția fiind dată de mișcare.

Acest modul reprezintă, alături de detectorul de arme, una dintre cele două componente antrenate în cadrul proiectului. Antrenarea (ajustarea fină) a fost realizată în proiectul separat **security_system**, iar modelul rezultat este consumat de aplicația principală ca un fișier checkpoint (**best_model.pth**), încărcat la rulare de clasa **HumanActionRecognitionSystem** din **har_system.py** (*Thread 5*). Subcapitolele care urmează descriu alegerea arhitecturii, construcția setului de date (inclusiv crearea de la zero a unui set de date propriu pentru clasa **vandalism**), procesul de ajustare fină, parametrii și rezultatele, iar la final modul de integrare în aplicație.

Figura 4.9 oferă o imagine de ansamblu a întregului proces, de la cele trei surse de date (inclusiv setul propriu pentru clasa **vandalism**) până la modelul antrenat și utilizarea lui la inferență. Spre deosebire de detectoarele de obiecte, intrarea modelului nu este un cadru izolat, ci o pereche de secvențe de cadre

(căile *Slow* și *Fast*) eșantionate dintr-un clip; aceeași procedură de pregătire a intrărilor este folosită identic la antrenare și la inferență. Subcapitolele care urmează detaliază fiecare etapă a acestui proces.

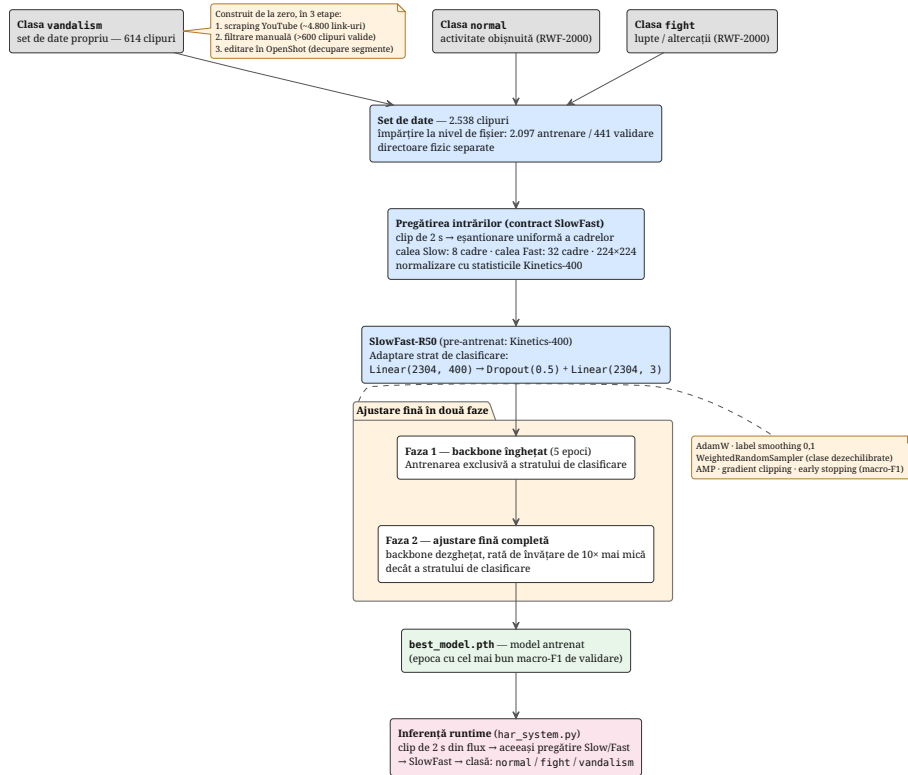


Figura 4.9: Procesul modulului de recunoaștere a acțiunilor umane. Setul de date este alcătuit din trei clase: **normal**, **fight** și **vandalism**. Clipurile sunt împărțite la nivel de fișier în antrenare și validare, iar fiecare clip este transformat, conform contractului **SlowFast**, într-o pereche de intrări **Slow/Fast**. Modelul **SlowFast R50** pre-antrenat pe **Kinetics-400** primește un strat nou de clasificare cu trei ieșiri și este ajustat fin în două faze. Modelul rezultat (**best_model.pth**) este apoi folosit la inferență în aplicație, unde clasifică, prin aceeași pregătire a intrărilor, clipuri de două secunde.

4.7.1 Justificarea alegerii **SlowFast R50**

Arhitectura aleasă este **SlowFast R50**, dezvoltată de Facebook AI Research. Ea se bazează pe două căi de procesare paralele: o cale *Slow*, care procesează puține cadre la rezoluție temporală redusă și captează informația semantică spațială, și o cale *Fast*, care procesează multe cadre și captează dinamica

mișcării, cele două fiind unite prin conexiuni laterale. Această separare explicită a aspectului spațial de cel temporal o face deosebit de potrivită pentru distingerea acțiunilor pe baza mișcării – exact problema de față.

Varianta R50 (cu backbone ResNet-50) oferă un compromis bun între capacitate și cost computațional, permițând inferența în timp real alături de celelalte modele. În plus, modelul este disponibil pre-antrenat pe setul de date Kinetics-400 (prin PyTorchVideo / PyTorch Hub), ceea ce oferă un punct de pornire solid: reprezentările video generice învățate pe 400 de clase de acțiuni pot fi adaptate prin ajustare fină la cele trei clase ale problemei, cu un set de date mult mai mic decât ar fi necesar pentru antrenarea de la zero.

4.7.2 Construcția setului de date

Setul de date pentru ajustarea fină a fost construit din trei surse, câte una pentru fiecare clasă. Structura este de tip folder-per-clasă (`data/train/<clasă>/` și `data/test/<clasă>/`), iar clasele sunt descoperite automat din numele folderelor de către `configs/config.py`, cu convenția ca eticheta `normal` să primească întotdeauna indexul 0.

4.7.2.1 Clasa *normal*

Clasa `normal` conține videoclipuri de supraveghere cu activitate cotidiană, fără incidente, preluate din setul de date public **RWF-2000** (disponibil pe Kaggle). Aceste secvențe oferă modelului o referință pentru ceea ce înseamnă comportament obișnuit, esențială pentru a delimita clasele de interes (`fight`, `vandalism`) de fundalul normal.

4.7.2.2 Clasa *fight*

Clasa `fight` conține videoclipuri cu lupte și altercații fizice, preluate tot din setul de date **RWF-2000**, care include scene de confruntare fizică între persoane, capturate de camere de supraveghere. Folosirea aceluiași set de date pentru clasele `normal` și `fight` asigură un stil vizual consistent (rezoluție, unghiuri, calitate a imaginii) între cele două, astfel încât modelul să învețe diferența de *acțiune*, nu de domeniu vizual.

4.7.2.3 Clasa *vandalism* – set de date propriu

4.7.2.3.1 Justificarea creării unui set de date propriu. Spre deosebire de primele două clase, pentru `vandalism` nu există un set de date public adecvat de videoclipuri de supraveghere. Din acest motiv, a fost necesară construirea unui set de date propriu, de la zero, printr-un proces în trei etape descris în continuare.

4.7.2.3.2 Etapa 1: Scraping automat YouTube. A fost dezvoltat un script Python dedicat care folosește biblioteca `yt-dlp` pentru a căuta și colecta

automat link-uri către videoclipuri candidate de pe YouTube. Scriptul generează sute de interogări de căutare variate, în mai multe limbi, acoperind acte specifice de vandalism (graffiti, spargere de geamuri, distrugere de mașini, intrare prin efracție etc.) combinate cu termeni asociați camerelor de supraveghere. În urma acestui proces au fost colectate aproximativ 4.800 de link-uri către potențiale videoclipuri.

4.7.2.3.3 Etapa 2: Filtrare manuală. Toate cele aproximativ 4.800 de videoclipuri au fost vizualizate și evaluate manual, pentru a selecta doar clipurile care conțin efectiv acte de vandalism filmate de camere de supraveghere, eliminându-le pe cele irelevante. În urma filtrării au rămas peste 600 de videoclipuri valide. Această etapă, deși laborioasă, este esențială pentru calitatea setului de date: garantează că modelul învață din exemple reale și relevante.

4.7.2.3.4 Etapa 3: Editare în OpenShot. Videoclipurile selectate au fost editate individual cu software-ul OpenShot Video Editor, pentru a extrage doar segmentele relevante. Fiecare clip a fost decupat astfel încât să conțină secvența propriu-zisă a actului de vandalism, eliminând porțiunile irelevante (intro-uri, text suprapus, secvențe fără activitate).

4.7.2.3.5 Setul final. Setul de date final pentru clasa `vandalism` conține **614 videoclipuri** cu acte de vandalism, acoperind o gamă largă de acțiuni: graffiti / street art, spargere de geamuri și vitrine, distrugere de autoturisme (zgâriere, lovire), intrare prin efracție, distrugere de mobilier urban, aruncare cu obiecte și alte forme de distrugere intenționată a proprietății.

4.7.3 Împărțirea în seturile de antrenare și validare

Un aspect esențial pentru validitatea rezultatelor raportate este modul în care datele au fost partiționate între antrenare și validare. Orice suprapunere între aceste două mulțimi ar conduce la o supraestimare a performanțelor (*data leakage*). Modul de organizare a datelor, vizibil în fișierul `data/dataset.py`, garantează imposibilitatea unei astfel de scurgeri de informații din două motive complementare.

Unitatea de antrenare este videoclipul integral, nu un fragment al acestuia. Fiecare exemplu din setul de date corespunde unui singur fișier video complet. Clasa `ActionVideoDataset` parcurge directoarele și construiește o listă de perechi (`cale_video`, `etichetă`), generând câte o intrare pentru fiecare fișier. La încărcare, întregul videoclip este decodat o singură dată, iar din durata sa totală sunt eșantionate *uniform* cadrele necesare pentru a construi perechea de intrări (*slow/fast*). Nu se extrag subsecvențe multiple din același videoclip și nu se folosește o tehnică de tip fereastră glisantă (*sliding window*); un videoclip produce strict un singur exemplu. În consecință, este exclusă situația în care un fragment dintr-un videoclip să ajungă la antrenare, iar o altă secțiune

a *aceluiași* fișier să ajungă la validare. Problema este eliminată prin însăși concepția sistemului, videoclipul fiind o unitate indivizibilă.

Împărțirea se face la nivel de fișier, în directoare fizic separate. Datele de antrenare și cele de validare provin din directoare complet distincte (`data/train/` și, respectiv, `data/test/`), fiecare având propriile subdirectoare per clasă. Repartizarea videoclipurilor în cele două locații a fost realizată *înainte* de faza de antrenare, la nivel de fișier complet, asigurând că un videoclip aparține exclusiv antrenării sau validării, niciodată amândurora. La execuție, funcția `build_dataloaders()` încarcă informațiile din `data/train/` pentru antrenament și folosește direct `data/test/` drept set de validare.

În total, setul de date cuprinde **2538 de videoclipuri**, repartizate în **2097 de videoclipuri pentru antrenare** (normal: 829, fight: 771, vandalism: 497) și **441 de videoclipuri pentru validare** (normal: 169, fight: 155, vandalism: 117). Cifrele aferente setului de validare corespund coloanei *Support* din analiza per clasă (Tabelul 4.4) și reprezintă numărul de videoclipuri pe care s-au calculat metricile finale. Pentru clasele `normal` și `fight` s-a păstrat împărțirea predefinită a setului original RWF-2000. Pentru clasa `vandalism` (care folosește setul de date propriu, cele 614 videoclipuri menționate anterior), fișierele au fost repartizate manual în cele două directoare, respectând aceeași regulă de separare la nivel de fișier.

4.7.4 Pregătirea modelului pentru ajustarea fină

Pregătirea modelului pornește de la SlowFast R50 pre-antrenat pe Kinetics-400, al cărui cap de clasificare original – un strat `Linear(2304, 400)` corespunzător celor 400 de clase Kinetics – este înlocuit cu un cap nou, adaptat problemei: o secvență `Dropout(0.5)` urmată de un `Linear(2304, 3)`, pentru cele trei clase. Această înlocuire este realizată în `build_slowfast_model()` din `models/slowfast_model.py`.

4.7.4.1 Formatul intrării (contractul SlowFast)

Fiecare videoclip este transformat într-o pereche de tensori, câte unul pentru fiecare cale: calea Slow primește 8 cadre, iar calea Fast 32 de cadre, ambele decupate spațial la 224×224 pixeli. Cadrele sunt eșantionate uniform dintr-un clip corespunzător unei durate de 2.0 secunde. Aceste valori sunt fixate în `configs/config.py` și trebuie respectate identic la inferență (în aplicație), altfel modelul ar primi intrări incompatibile cu cele pe care a fost antrenat.

Listing-ul 4.6 prezintă procedura de generare a intrărilor pentru arhitectura SlowFast, implementată în funcția `pack_frames_slowfast()`: din totalul de `T` cadre ale secvenței video se eșantionează uniform 8 indici pentru ramura Slow și 32 de indici pentru ramura Fast. Fiecare cadru extras este redimensionat la rezoluția de 224×224 pixeli și normalizat utilizând media și abaterea standard aferente setului de date Kinetics-400. Aceeași procedură de structurare a datelor este aplicată în mod identic și în etapa de inferență, prin intermediul metodei `har_system._pack_slowfast()`.

```

1 # T cadre (T, H, W, 3) -> pereche de tensori pentru cele doua cai SlowFast
2 def pack_frames_slowfast(frames, num_slow=8, num_fast=32, crop_size=224):
3     T_total = frames.shape[0]
4     # Esantionare uniforma a indicilor de-a lungul clipului (2.0 s)
5     # 8 cadre
6     slow_idx = np.linspace(0, T_total - 1, num_slow, dtype=np.int64)
7     # 32 cadre
8     fast_idx = np.linspace(0, T_total - 1, num_fast, dtype=np.int64)
9     slow_tensor = frames_to_tensor(frames[slow_idx], crop_size)
10    fast_tensor = frames_to_tensor(frames[fast_idx], crop_size)
11    # (3, 8, 224, 224) si (3, 32, 224, 224)
12    return [slow_tensor, fast_tensor]
13
14 def frames_to_tensor(frames, crop_size):
15     imgs = []
16     # fiecare cadru (H, W, 3)
17     for f in frames:
18         # (3, H, W), valori in [0, 1]
19         img = to_tensor(f).float() / 255.0
20         # decupare spatiala 224x224
21         img = resize(img, [crop_size, crop_size])
22         imgs.append(img)
23     # (3, T, 224, 224)
24     tensor = torch.stack(imgs, dim=1)
25     # Normalizare cu media / deviatia standard Kinetics-400
26     mean = torch.tensor([0.45, 0.45, 0.45]).view(3, 1, 1, 1)
27     std = torch.tensor([0.225, 0.225, 0.225]).view(3, 1, 1, 1)
28     return (tensor - mean) / std

```

Listing 4.6: Construcția perechii de intrări slow/fast (`pack_frames_slowfast` din `data/dataset.py`).

4.7.5 Parametrii de antrenare

Antrenarea, implementată în `train.py`, folosește o strategie de ajustare fină în *două faze*, gândită pentru a adapta modelul la noile clase fără a distruge reprezentările pre-antrenate:

- **Faza 1 – backbone înghețat** (primele 5 epoci): toți parametrii backbone-ului sunt înghețați (`requires_grad=False`), antrenându-se *doar* capul de clasificare nou. Astfel, capul aleator inițializat se stabilizează fără a perturba caracteristicile vizuale învățate pe Kinetics-400;
- **Faza 2 – ajustare fină completă**: backbone-ul este dezghețat și întregul model este antrenat, dar cu *rate de învățare diferențiate* – backbone-ul primește o rată de 10 ori mai mică decât capul (`lr_backbone_factor=0.1`), pentru a-l ajusta fin fără a-l deteriora.

Listing-ul 4.7 prezintă structura de bază a celor două etape de antrenare din modulul `train.py`: în Faza 1, optimizatorul actualizează exclusiv parametrii straturilor finale de clasificare (capul rețelei), în timp ce în Faza 2, rețeaua de

extracție a trăsăturilor (*backbone-ul*) este deblocată și antrenată utilizând rate de învățare diferențiate. Acest proces este gestionat de un planificator de tip `StepLR` și include un mecanism de oprire timpurie (*early stopping*) evaluat pe baza metricii `macro_f1`.

```
1 # Faza 1: backbone înghețat -- se antrenează doar capul de clasificare
2 # requires_grad = False pe backbone
3 model.freeze_backbone()
4 optimizer = AdamW(
5     # doar parametrii activi
6     filter(lambda p: p.requires_grad, model.parameters()),
7     lr=LR, weight_decay=WEIGHT_DECAY,
8 )
9 # primele 5 epoci
10 for epoch in range(1, FREEZE_EPOCHS + 1):
11     train_one_epoch(model, train_loader, criterion, optimizer)
12     metrics = validate(model, val_loader, criterion)
13     # checkpoint pe cel mai bun macro-F1
14     save_best_and_last(model, metrics)
15
16 # Faza 2: ajustare fină completă, cu rate de învățare diferențiate
17 model.unfreeze_backbone()
18 param_groups = model.get_optimizer_param_groups(
19     # backbone: LR de 10x mai mic decât capul
20     lr=LR, lr_backbone_factor=0.1,
21 )
22 optimizer = AdamW(param_groups, weight_decay=WEIGHT_DECAY)
23 # 10x la 10 epoci
24 scheduler = StepLR(optimizer, step_size=LR_STEP_SIZE, gamma=LR_GAMMA)
25
26 for epoch in range(FREEZE_EPOCHS + 1, EPOCHS + 1):
27     train_one_epoch(model, train_loader, criterion, optimizer)
28     metrics = validate(model, val_loader, criterion)
29     scheduler.step()
30     save_best_and_last(model, metrics)
31     # oprire timpurie pe macro-F1
32     if early_stopping(metrics["macro_f1"]):
33         break
```

Listing 4.7: Ajustarea fină în două faze a modelului `SlowFast` (`train.py`).

Alți parametri și tehnici folosite, conform `config.py` și `train.py`:

- **Optimizator:** `AdamW`, rată de învățare 10^{-3} , `weight_decay` = 10^{-4} , cu o programare de tip `StepLR` (reducere de $10\times$ la fiecare 10 epoci);
- **Funcția de cost:** `CrossEntropyLoss` cu *label smoothing* de 0,1, pentru a reduce supraîncrederea modelului;
- **Dezechilibrul de clase:** deoarece numărul de clipuri diferă între categorii, dezechilibrul a fost tratat la nivelul *eșantionării*, nu al funcției de

pierdere, printr-un `WeightedRandomSampler`. Concret, se numără clipurile fiecărei clase din setul de antrenare, iar fiecărui clip i se atribuie o pondere invers proporțională cu frecvența clasei sale ($w_c = 1/n_c$, unde n_c este numărul de clipuri din clasa c). La fiecare epocă, eșantionatorul extrage același număr total de clipuri, dar *cu revenire* și conform acestor ponderi, astfel încât clipurile claselor minoritare sunt selectate de mai multe ori (supra-eșantionare), iar cele ale claselor majoritare mai rar (sub-eșantionare). În acest fel, fiecare clasă ajunge să fie reprezentată aproximativ egal în loturi, fără a modifica numărul de iterații per epocă și fără a duplica fizic fișierele pe disc. (Întrucât `PyTorch` nu permite folosirea simultană a unui eșantionator și a amestecării aleatoare, `shuffle` este dezactivat automat când eșantionatorul este activ, ordinea aleatoare fiind deja asigurată de extragerea ponderată.)

- **Tehnici de stabilizare:** antrenare cu precizie mixtă (AMP, utilizând `GradScaler` și `autocast`), limitarea gradientului (*gradient clipping*) la norma maximă de 5,0 și oprire timpurie (*early stopping*) cu o toleranță de 7 epoci, evaluată pe baza scorului macro-F1 de validare;
- **Augmentare spațială:** oglindire orizontală, *color jitter* și *random crop*.

Din totalul de 30 de epoci planificate, procesul de antrenare s-a oprit automat la epoca 28 datorită mecanismului de oprire timpurie (*early stopping*). Cele mai bune rezultate (salvate în fișierul `best_model.pth`) au fost obținute la epoca 21. La nivel de iterație, s-au utilizat 262 de loturi de date (*batches*) pentru faza de antrenament și 56 pentru cea de validare, cu o durată medie de execuție de aproximativ 15 minute per epocă.

4.7.6 Rezultatele

Metricile globale obținute pe setul de validare de către cel mai bun model (epoca 21) sunt sintetizate în Tabelul 4.3. Valoarea finală a funcției de cost (*loss*) pentru antrenare a fost de 0,3065, iar pentru validare a înregistrat 0,4306.

Tabel 4.3: Metricile globale de validare ale modelului de recunoaștere a acțiunilor.

Metrică	Valoare
Acuratețe (<i>accuracy</i>)	0,9433
Macro Precision	0,9478
Macro Recall	0,9436
Macro F1-Score	0,9456

Modelul atinge o acuratețe globală de 94,33% și un scor macro-F1 de 94,56%, valori foarte bune pentru o problemă de clasificare a acțiunilor pe trei clase, în condiții reale de supraveghere video.

Figura 4.10 prezintă evoluția procesului de antrenare pe parcursul celor 28 de epoci. Graficul din stânga ilustrează funcțiile de cost aferente seturilor de antrenare și validare, iar graficul din dreapta evidențiază variația scorului macro-F1 calculat pe setul de validare.

Se observă o scădere constantă a funcției de cost pentru ambele seturi, costul de validare stabilizându-se în jurul valorii de 0,43 după aproximativ 19–20 de epoci. Absența unei creșteri ulterioare a acestuia indică faptul că modelul nu manifestă fenomenul de supraînvățare (*overfitting*). Diferența moderată dintre costul final de antrenare (0,3065) și cel de validare (0,4306) este compatibilă cu dimensiunea relativ redusă a setului de date utilizat.

În primele epoci se remarcă o variație mai accentuată a performanței, reflectată prin creșterea temporară a costului de validare și scăderea scorului macro-F1 în intervalul epocilor 4–6. Acest comportament poate fi asociat tranziției dintre cele două etape ale antrenării. În primele cinci epoci a fost antrenat exclusiv stratul de clasificare adăugat, în timp ce parametrii rețelei de bază au fost menținuți fixați. Începând cu epoca a șasea, rețeaua de bază (*backbone*) a fost deblocată, ceea ce a generat o perturbare tranzitorie a procesului de optimizare, urmată de o îmbunătățire rapidă a performanței pe măsură ce întregul model s-a adaptat sarcinii de clasificare.

Scorul macro-F1 pe setul de validare a crescut de la aproximativ 0,81 la peste 0,94, valoarea maximă fiind atinsă în epoca 21, corespunzătoare modelului salvat în fișierul `best_model.pth`. După acest moment, performanța s-a menținut relativ constantă, fără îmbunătățiri semnificative, ceea ce a condus la activarea mecanismului de oprire timpurie (*early stopping*) în epoca 28.

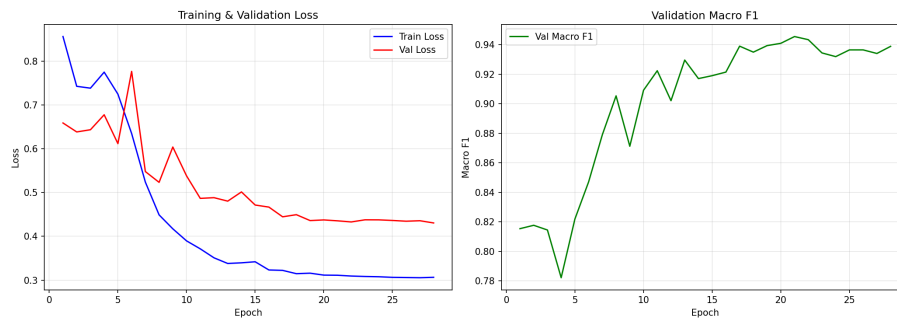


Figura 4.10: Evoluția antrenării modelului de recunoaștere a acțiunilor pe durata a 28 de epoci. Stânga: funcția de cost de antrenare (albastru) și de validare (roșu). Dreapta: scorul macro-F1 obținut pe setul de validare. Perturbarea din jurul epocilor 4–6 corespunde deblocării parametrilor rețelei de bază (tranziția de la ponderi fixe în epocile 1–5 la optimizare globală începând cu epoca 6). Ponderile optime s-au obținut la epoca 21, mecanismul de oprire timpurie (*early stopping*) declanșându-se la epoca 28.

4.7.7 Analiza per clasă

Rezultatele detaliate pe fiecare clasă sunt prezentate în tabelul 4.4.

Tabel 4.4: Metricile per clasă ale modelului de recunoaștere a acțiunilor.

Clasă	Precision	Recall	F1-Score	Suport
normal	0,920	0,947	0,933	169
fight	0,942	0,935	0,939	155
vandalism	0,982	0,949	0,965	117

Datele de evaluare evidențiază performanța optimă obținută pe clasa **vandalism** (o precizie de 98,2% și un scor F1 de 0,965), pentru care a fost utilizat setul de date propriu. Nivelul ridicat al metricilor indică faptul că secvențele video agregate și verificate manual oferă o reprezentare suficient de stabilă pentru a permite modelului o clasificare sigură. Categoriile **normal** și **fight** mențin, de asemenea, performanțe ridicate (scoruri F1 de 0,933, respectiv 0,939). Erorile de clasificare reziduale se manifestă, în mod previzibil, preponderent între aceste două clase, ambele provenind din setul de date RWF-2000. Acest comportament poate fi atribuit prezenței unor secvențe cu ambiguitate vizuală ridicată, situate la granița dintre o activitate umană rapidă și o altercație fizică propriu-zisă.

Distribuția erorilor de clasificare este ilustrată prin matricea de confuzie normalizată prezentată în Figura 4.11, unde liniile corespund claselor reale, iar coloanele claselor prezise de model. Valorile ridicate de pe diagonală (0,95 pentru **normal**, 0,94 pentru **fight** și 0,95 pentru **vandalism**) indică o rată ridicată de clasificare corectă pentru toate cele trei categorii analizate.

Erorile de clasificare se manifestă predominant între clasele **normal** și **fight**: aproximativ 5% dintre exemplele din clasa **normal** sunt încadrate eronat în clasa **fight**, în timp ce 6% dintre exemplele din clasa **fight** sunt clasificate ca **normal**. Acest comportament este explicabil prin faptul că ambele categorii provin din același set de date, RWF-2000, și prezintă caracteristici vizuale similare.

În schimb, clasa **vandalism** este confundată foarte rar cu celelalte două clase, procentul de clasificări eronate fiind sub 1% în ambele direcții. Acest rezultat sugerează că modelul a reușit să învețe reprezentări distinctive pentru această categorie, diferențiind-o eficient de acțiunile asociate claselor **normal** și **fight**.

4.7.7.1 Integrarea în aplicație

La rulare, modelul antrenat este folosit de **HumanActionRecognitionSystem**, care nu poate clasifica un singur cadru, ci are nevoie de o secvență. De aceea, sistemul acumulează cadrele primite într-un *ring buffer* (*deque*) și rulează inferența doar o dată la fiecare 30 de cadre (`clip_interval_frames`), atunci când bufferul conține destule cadre. La fiecare inferență, se eșantionează uniform până la 64 de cadre din buffer, se construiește perechea *slow/fast* (8/32 cadre, 224×224) exact ca la antrenare, se aplică **softmax** peste logits și se reține clasa cu probabilitatea maximă. O acțiune non-normală este raportată ca alertă doar dacă

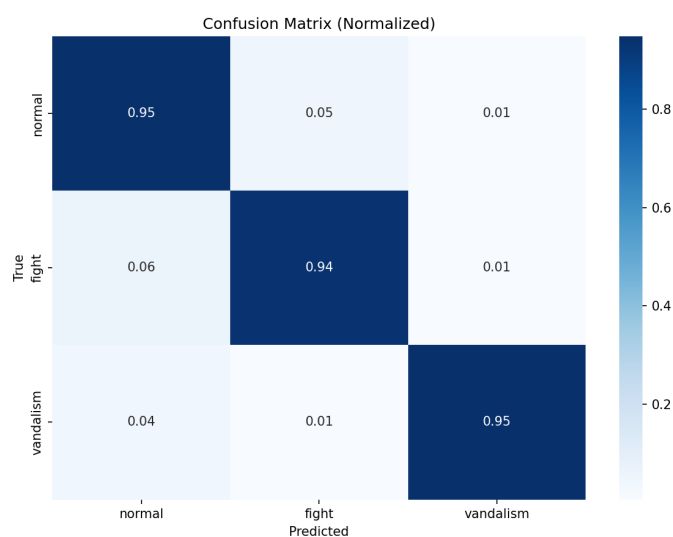


Figura 4.11: Matricea de confuzie normalizată a modelului de recunoaștere a acțiunilor, evaluată pe partiția de validare. Liniile indică apartenența reală la clasă (*ground truth*), în timp ce coloanele reprezintă predicțiile modelului, valorile fiind normalizate pe fiecare linie. Dominanța valorilor de pe diagonala principală confirmă acuratețea ridicată a clasificării, erorile reziduale manifestându-se cu precădere sub forma unor confuzii între clasele `normal` și `fight`.

încrederea depășește pragul `confidence_threshold` (0,5), cu un *cooldown* de alertă de 5 secunde pentru a evita notificările repetate. Predicția curentă este păstrată între inferențe, astfel încât adnotarea afișată pe flux rămâne coerentă chiar și pe cadrele dintre două rulări de inferență.

4.8 Fuziunea rezultatelor pe cadru

Cele cinci module de inteligență artificială descrise anterior nu rulează izolat, ci sunt orchestrate de aplicația principală (`app.py`) într-un *pipeline multi-threaded* care procesează același flux video în paralel și apoi recombina rezultatele într-un singur cadru adnotat. Această secțiune descrie modul în care detecțiile provenite de la modele diferite, care rulează pe fire de execuție separate și se termină la momente diferite, sunt sincronizate, suprapuse pe cadru, transformate în alarme și difuzate către clienți.

Decizia arhitecturală fundamentală este aceea de a folosi *fire de execuție* (**threading**) cu cozi (`queue.Queue`), nu programare asincronă: apelurile de inferență ale modelelor sunt operații blocante, intensive computațional, care nu trebuie să blocheze bucla de evenimente a serverului FastAPI. Pipeline-ul folosește în total **șapte fire de execuție**: captură (Thread 1), recunoaștere facială cu demografie (Thread 2), plăcuțe (Thread 3), foc și fum (Thread 4), acțiuni umane (Thread 5), arme (Thread 6) și un fir final de fuziune (Thread 7).

4.8.1 Dispecerizarea cadrelor și procesarea în paralel

Firul de execuție responsabil de captură (Thread 1) preia cadrele de la toate camerele active. Fiecărui cadru *i* se atribuie un identificator unic, monoton crescător, denumit `frame_id`, care va servi drept cheie de sincronizare pe parcursul întregului flux de prelucrare. Ulterior, cadrul este distribuit de metoda `_dispatch_frame()`, care inserează câte o copie a acestuia în coada de intrare aferentă fiecărui modul activ (`face_processing_queue`, `plate_processing_queue` etc.) și, în mod distinct, în coada cadrelor brute (`raw_frame_queue`), necesară pentru recompunerea imaginii finale.

Fiecare modul de inteligență artificială operează într-un fir de execuție independent, extrăgând cadre din propria coadă, rulând algoritmul de inferență și publicând rezultatele în coada de ieșire specifică (`face_results_queue` etc.). Deoarece aceste cozi sunt implementate cu o capacitate limitată (`maxsize=10`), în situația în care un modul necesită un timp de procesare mai lung și nu mai poate prelua date noi, funcția de distribuție nu se blochează, ci omite cadrul respectiv (incrementând un contor intern de tip `queue_skips` / `frames_dropped`). Această decizie arhitecturală asigură o degradare controlată a ratei de procesare a cadrelor, garantând continuitatea execuției pentru restul sistemului.

4.8.2 Sincronizarea rezultatelor după *frame_id*

Firul de execuție responsabil de fuziunea datelor (Thread 7) gestionează procesul complex de recombinaire a rezultatelor generate *asincron*. De exemplu, modulul pentru identificarea plăcuțelor poate finaliza procesarea cadrului N înainte ca modulul facial să termine cadrul $N - 2$. Pentru a rezolva această asimetrie temporală, sistemul menține un dicționar, **pending_results**, care include câte o structură de reținere pentru fiecare categorie de rezultat (**frames**, **face**, **plate**, **fire**, **har**, **weapon**), indexată pe baza identificatorului **frame_id**. La fiecare iterație, acest fir extrage toate datele disponibile din cozile de ieșire, populând dicționarul, după care încearcă să *agregheze* fiecare cadru pentru care a fost recepționată imaginea brută.

Un cadru este validat pentru agregare în momentul în care au sosit rezultatele furnizate de modulele *esențiale* activate (recunoașterea facială și a plăcuțelor). Rezultatele provenite de la modulele pentru foc, acțiuni și arme sunt tratate ca *opționale*: acestea sunt integrate în imagine doar dacă sunt deja disponibile, fără ca absența lor să blocheze procesul de asamblare. Pentru a preveni stările de așteptare infinită cauzate de rezultate lipsă (de exemplu, pe fondul omiterii anterioare a unui cadru), este implementat un mecanism de limitare a așteptării (*timeout*): dacă decalajul dintre **frame_id**-ul curent și cel al cadrului evaluat depășește 60 de indici, cadrul este asamblat forțat utilizând exclusiv datele parțiale disponibile la acel moment. Adicional, o rutină de mentenanță (**_cleanup_old_results()**) elimină periodic intrările cu o vechime mai mare de 90 de cadre, prevenind astfel creșterea necontrolată a dicționarului în caz de desincronizări severe.

Listing-ul 4.8 prezintă ciclul de execuție al firului de fuziune și politica de selecție *cel mai recent cadru disponibil*: la fiecare iterație, conținutul tuturor cozilor de ieșire este preluat într-o manieră non-blocantă în structurile locale (operațiune de tip **drain**). Ulterior, fiecare cadru ce conține imaginea brută este compus de îndată ce rezultatele modulelor esențiale sunt finalizate sau perioada de așteptare (*timeout*) a expirat. Rezultatele detectoarelor opționale sunt suprapuse exclusiv dacă acestea se află deja la dispoziție în momentul asamblării.

```

1 # Thread 7 (fuziune): structuri locale, indexate pe frame_id
2 pending = {"frames": {}, "face": {}, "plate": {},
3           "fire": {}, "har": {}, "weapon": {}}
4
5 while True:
6     # 1. Goleste nebloant toate cozile de iesire in structurile locale
7     # (camera_id, frame)
8     drain(raw_frame_queue, into=pending["frames"])
9     if face_enabled: drain(face_results_queue, into=pending["face"])
10    if plate_enabled: drain(plate_results_queue, into=pending["plate"])
11    if fire_enabled: drain(fire_results_queue, into=pending["fire"])
12    if har_enabled: drain(har_results_queue, into=pending["har"])
13    if weapon_enabled: drain(weapon_results_queue, into=pending["weapon"])
14
15    # 2. Incearca sa assembleze fiecare cadru pentru care exista imaginea
16    ↪ bruta
17    for frame_id in list(pending["frames"]):
18        # asamblare fortata
19        timeout = is_frame_too_old(frame_id, max_age=60)
20
21        # Module esentiale: gata SAU expirat; foc/HAR/arme sunt optionale
22        ready = True
23        if face_enabled: ready = ready and (frame_id in pending["face"] or
24        ↪ timeout)
25        if plate_enabled: ready = ready and (frame_id in pending["plate"] or
26        ↪ timeout)
27        if not (face_enabled or plate_enabled):
28            ready = True
29        if not ready:
30            # mai asteapta acest cadru
31            continue
32
33        camera_id, final_frame = pending["frames"].pop(frame_id)
34
35        # 3. Suprapune rezultatul disponibil de la fiecare detector
36        ↪ (optional)
37        for det in ("face", "plate", "fire", "har", "weapon"):
38            if frame_id in pending[det]:
39                final_frame = draw(det, final_frame,
40                ↪ pending[det].pop(frame_id))
41
42        # vezi logica de alarmare
43        create_alarms_and_logs(camera_id, final_frame)
44        # catre clientii WebSocket
45        broadcast_frame(camera_id, final_frame)

```

Listing 4.8: Firul de fuziune și politica *ultimul cadru disponibil* (merging_thread din app.py).

4.8.3 Compunerea cadrului adnotat

Odată ce un cadru este gata de asamblare, adnotările celor cinci module sunt desenate *în straturi* peste imagine, într-o ordine fixă. Punctul de plecare este

cadru deja adnotat de modulul facial (care conține chenarele fețelor și etichetele demografice), peste care se aplică succesiv: chenarele plăcuțelor (`plate_system.draw_outputs()`), detecțiile de foc confirmate (`fire_system.draw_detections()`), eticheta acțiunii recunoscute (`har_system.draw_detections()`) și, la final, chenarele armelor (`weapon_system.draw_detections()`). Rezultatul este un singur cadru (`final_frame`) care concentrează vizual toate detecțiile celor cinci module pentru acel moment de timp.

4.8.4 Generarea alarmelor și deduplicarea

În cadrul aceleiași etape de fuziune, detecțiile de interes sunt convertite în două categorii de înregistrări persistente: *jurnale de detecție* (un istoric exhaustiv, generat prin apelul funcției `_log_detection_if_needed()`) și *alarme* propriu-zise (evenimente critice care necesită atenția imediată a operatorului). Sunt clasificate drept evenimente de alarmă următoarele situații: prezența persoanelor neidentificate, detectarea confirmată a focului (marcată cu parametrul `alert`), identificarea unor acțiuni diferite de clasa `normal`, detecția armamentului, prezența vehiculelor neautorizate (plăcuțe lizibile care nu figurează în lista de acces) și pătrunderea unei persoane neautorizate într-o zonă restricționată.

Problema principală în gestionarea alarmelor o constituie evitarea saturării operatorului cu notificări redundante: un focar de incendiu activ timp de 10 secunde nu trebuie să declanșeze o avertizare la fiecare cadru analizat. Mecanismul de deduplicare, implementat în metoda `_create_alarm_if_needed()`, utilizează o cheie de indexare compusă, formată din identificatorul camerei, tipul evenimentului și *subiectul* alarmei (`camera_id:alarm_type:dedup_subject`), impunând o perioadă de latență (*cooldown*) de 30 de secunde pentru fiecare cheie unică. Includerea subiectului în cheie este esențială, deoarece permite coexistența unor alarme distincte de *același* tip – de exemplu, două persoane necunoscute diferite (diferențiate pe baza coordonatelor spațiale), un cuțit și un pistol, sau două autovehicule neautorizate distincte – fără ca acestea să fie comasate eronat într-o singură alertă. Complementar acestui mecanism de filtrare în memoria volatilă, sistemul execută o verificare redundantă în baza de date (prin metoda `get_recent_alarm()`). Această abordare previne o cascadă de alarme duplicate ce ar putea surveni imediat după o repornire a serverului, moment în care istoricul stărilor de latență din memorie este neinițializat.

4.8.5 Difuzarea către clienți prin WebSocket

Ultima etapă a procesului de fuziune constă în transmiterea cadrului agregat către interfețele clienților. Imaginea `final_frame` este redimensionată (la o lățime maximă de 800 px), comprimată în format JPEG cu un factor de calitate de 75 și convertită în format Base64 prin intermediul funcției `_encode_and_queue_frame()`. Datele sunt apoi încapsulate într-un obiect JSON care conține, pe lângă imaginea propriu-zisă, listele serializate cu rezultatele tuturor modulelor (`face_results`, `plate_results`, `fire_results`, `har_results`, `weapon`

`_results`), identificatorul camerei, un marcaj temporal (*timestamp*) și starea curentă (activat/dezactivat) a fiecărui modul.

Difuzarea mesajelor (`_broadcast_frame()`) utilizează un model de distribuție de tip *fan-out*, implementat cu cozi independente pentru fiecare client. Mai exact, fiecare utilizator conectat prin WebSocket dispune de propria coadă de ieșire, iar firul de fuziune inserează fiecare cadru procesat în toate aceste cozi. Această izolare arhitecturală este esențială: în cazul în care un client are o conexiune mai lentă și coada i se umple, acesta va elimina automat doar cel mai vechi cadru din propriul flux, fără a interfera cu pachetele destinate celorlalți clienți și fără a genera întârzieri la nivelul sistemului. Astfel, fluxul video adnotat este livrat fiecărui operator în mod independent, finalizând ciclul de procesare inițiat la captura cadrului brut.

4.9 Contribuții proprii

Capitolele de proiectare (Capitolul 3) și de implementare a modulelor de inteligență artificială (capitolul de față) descriu sistemul în ansamblul său, integrând atât componente utilizate în forma lor inițială din ecosistemul *open-source*, cât și soluții dezvoltate specific în cadrul acestui proiect. Întrucât aportul personal este distribuit pe parcursul ambelor capitole și este, prin natura sa, dificil de izolat în cadrul unei lecturi secvențiale, această secțiune îl recapitulează și îl prezintă explicit, sub forma unor contribuții numerotate (C $\mathfrak{x}$). Fiecare contribuție face trimitere, prin referință încrucișată, la secțiunea în care este tratată în detaliu, astfel încât aportul propriu să poată fi delimitat fără ambiguitate de utilizarea modelelor și a bibliotecilor preexistente.

Pentru claritate, contribuțiile sunt grupate în trei categorii: contribuții de *proiectare și arhitectură* a sistemului integrat, contribuții *algoritmice și de procesare* la nivelul modulelor, respectiv contribuții privind *modelele antrenate și seturile de date* construite. Pe ansamblu, contribuția majoră nu constă în dezvoltarea de la zero a unor noi arhitecturi neuronale, ci în *integrarea* a cinci familii eterogene de detectoare într-un singur sistem de timp real, coerent și operațional. Acestui efort i se adaugă construirea celor două componente pentru care nu existau soluții standard pre-antrenate: detectorul de arme și modulul de recunoaștere a acțiunilor umane.

4.9.1 Contribuții de proiectare și arhitectură

C1 – Arhitectură concurentă pentru rularea simultană a celor cinci module de inteligență artificială. Proiectarea unui flux de procesare cu șapte fire de execuție (captură, cinci detectoare și fuziune), pe baza principiului de separare a responsabilităților la nivel de fir. Toate modelele sunt inițializate o singură dată și partajate pe același GPU, în cadrul unui proces unic. Alocarea sarcinilor respectă atât criteriul omogenității computaționale (recunoașterea facială fiind grupată cu analiza demografică, iar detecția plăcuțelor cu recunoașterea optică a caracterelor), cât și

pe cel al izolării costurilor de procesare și a frecvenței de execuție (§3.4, §3.4.3, §4.8).

- C2 – Mecanism de fuziune a rezultatelor bazat pe identificatorul de cadru (`frame_id`).** Proiectarea unui fir de fuziune care sincronizează ieșirile asincrone ale detectoarelor utilizând un identificator de cadru monoton crescător. Logica se bazează pe structura de date `pending_results` și introduce o distincție arhitecturală clară între modulele *esențiale* și cele *opționale*. Totodată, mecanismul integrează politici de limitare a timpului de așteptare (*timeout*) și de curățare periodică a memoriei, menite să prevină blocajele (așteptarea nedeterminată) și consumul necontrolat al resurselor (§4.8.1–§4.8).
- C3 – Mecanism de *backpressure* utilizând cozi cu capacitate limitată.** Implementarea unei strategii de contrapresiune (*backpressure*) fundamentate pe cele unsprezece cozi cu dimensiune maximă fixă (`queue.Queue(maxsize=10)`). Aceasta presupune o gestionare diferențiată a stărilor de saturare a resurselor: omiterea cadrelor la intrare (eveniment monitorizat prin contoarele `queue_skips` / `frames_dropped`) și aplicarea politicii de eliminare a celui mai vechi element (*drop-oldest*) la ieșire. Ambele abordări au ca obiectiv central prioritizarea prelucrării celui mai recent cadru și respectarea strictă a bugetului de latență (§3.4).
- C4 – Mecanism de tip *cel mai recent rezultat disponibil* pentru decuplarea ratei de redare de modulul cu cea mai mare latență.** Proiectarea unei strategii de compunere a cadrelor prin care firul de fuziune nu așteaptă finalizarea tuturor inferențelor pentru imaginea curentă, ci stochează și actualizează continuu cele mai recente rezultate per cameră, suprapunându-le peste ultimul cadru capturat. Acest mecanism asigură menținerea fluidității fluxului video la frecvența sursei (~30 FPS), independent de constrângerile unui detector cu procesare temporală la nivel de secvență (precum HAR, care produce ieșiri la ~2 s) sau de latența ridicată inerentă modulului de recunoaștere facială (§3.4, §4.8).
- C5 – Structurarea modelului conceptual de evenimente în detecții, jurnale și alarme.** Proiectarea unei arhitecturi a evenimentelor bazată pe trei niveluri distincte de persistență, cu cicluri de viață specifice: detecția volatilă (la nivel de cadru), jurnalul pasiv interogabil (optimizat printr-o fereastră de deduplicare de 5 s) și alarma operațională (deduplicată la un interval de 30 s, asociată unui nivel de severitate, unei capturi de imagine/*snapshot* și unor stări terminale de validare: `resolved` / `false_alarm`). În cadrul acestui model, mecanismele de jurnalizare și alarmare funcționează ca două fluxuri informaționale *paralele* și complementare (§3.9.1).
- C6 – Mecanism centralizat de deduplicare a alarmelor utilizând cheia compusă (cameră, tip, subiect).** Proiectarea unei logici de (*cool-down*) care utilizează parametrul de identificare `dedup_subject` (cuanti-

zare spațială pentru persoanele necunoscute, `employee_id` pentru personalul autorizat, eticheta acțiunii sau numărul de înmatriculare) pentru a distinge și separa detecțiile concurente de același tip. Acest mecanism este consolidat printr-o validare redundantă (*backstop*) la nivelul bazei de date, concepută special pentru a preveni generarea în cascadă a înregistrărilor duplicate în eventualitatea unei reporniri a sistemului (§3.9.5, §4.8.4).

- C7 – Configurarea dinamică a modulelor de inteligență artificială în timpul execuției, fără repornirea serverului.** Proiectarea unui mecanism de activare și dezactivare individuală a celor șase module prin intermediul unor puncte terminale API dedicate (*endpoints* de tip comutator/*toggle*). Această soluție exploatează eficient arhitectura concurrentă a aplicației: dezactivarea unui modul se traduce prin întreruperea direcționării cadrelor către coada aferentă detectorului, firul de execuție intrând într-o stare de așteptare pasivă (blocare la nivelul operațiunii `get()`) fără a consuma resurse computaționale (§3.4, cerințele F7bis și NF5 din Capitolul 3).
- C8 – Mecanism de autorizare spațială integrat cu recunoașterea facială și asocierea cameră–zonă.** Proiectarea unei logici de control al accesului la nivel de individ (gestionată prin parametrul `authorized_zones`), evaluată direct în firul de fuziune în raport cu zona atribuită camerei curente. Performanța interogărilor este asigurată printr-un sistem de memorie intermediară (*cache*), actualizat pe baza unui timp de viață prestabilit (TTL) și invalidat explicit la orice modificare a drepturilor de acces. Acest mecanism permite generarea unor alarme diferențiate pentru persoanele neautorizate din zonele restricționate: persoane complet necunoscute, respectiv personal identificat, dar neautorizat pentru locația respectivă (§3.3.2, §3.9).
- C9 – Arhitectură de distribuție WebSocket de tip *fan-out* cu cozi dedicate per client.** Proiectarea unui canal de transmisie în care fiecare operator conectat dispune de o coadă de mesaje individuală (gestionată prin politica de eliminare a pachetelor vechi, *drop-oldest*). Această topologie garantează că un client cu limitări de lățime de bandă sau de procesare va suferi o degradare exclusiv a propriului flux video (omiterea cadrelor), fără a penaliza performanța celorlalți utilizatori. Securitatea canalului este asigurată prin autentificarea pe bază de tokenuri de acces (JWT), starea de autorizare fiind revalidată continuu în raport cu baza de date (§3.6, §4.8.5).
- C10 – Consolidarea arhitecturii de securitate și controlul accesului.** Implementarea unui model de control al accesului bazat pe roluri (RBAC) cu profilurile `admin` și `user`, integrat cu sistemul de autentificare pe bază de tokenuri (JWT). Această componentă introduce un mecanism preventiv de tip *fail-fast*, care blochează pornirea serverului în absența cheii criptografice (JWT_SECRET) în mediul de producție. Securitatea este completată

de o listă de revocare (*blocklist*) pentru invalidarea activă a sesiunilor, securizarea parolilor prin funcții de dispersie criptografică (*hashing* cu algoritmul bcrypt) și revalidarea stării de autorizare direct pe canalul de comunicație WebSocket (§3.8).

C11 – Arhitectura stratului de persistență și reconstrucția deterministă a indexului FAISS. Proiectarea schemei relaționale (PostgreSQL) pentru toate entitățile sistemului, acompaniată de o strategie de sincronizare prin care indexul FAISS (dedicat vectorilor de trăsături faciale de 512 dimensiuni) este reconstruit determinist, direct din baza de date, la fiecare inițializare a serverului. Această abordare decuplează stocarea permanentă de memoria operațională, asigurând toleranța la defecte: pierderea sau coruperea fișierului local de index nu afectează consistența și integritatea datelor biometrice (§3.7, §4.2.2).

4.9.2 Contribuții algoritmice și de procesare

C12 – Strategie de comparare facială utilizând un sistem cu prag dublu și procedură de calibrare. Implementarea algoritmului de identificare pe baza conversiei analitice a distanței euclidiene (L_2) în similaritate cosinus pentru vectorii de trăsături normalizați ($\cos \theta = 1 - \frac{L_2^2}{2}$). Metoda combină un prag restrictiv pentru distanța vectorială cu un prag minim pentru scorul de certitudine (*confidence score*), fiind completată de un mecanism de calibrare a distanțelor (§4.2.3).

C13 – Optimizarea condiționată a analizei demografice și a recunoașterii emoțiilor. Restrângerea calculului atributelor vizuale secundare (vârstă, gen, stare emoțională) *exclusiv la nivelul fețelor necunoscute*. Această decizie elimină o etapă de inferență redundantă prin rețeaua dedicată recunoașterii emoțiilor pentru persoanele a căror identitate a fost deja confirmată, reducând semnificativ costul computațional la nivelul fiecărui cadru procesat (§4.2.4).

C14 – Integrarea decuplată a clasificatorului de emoții Mini-Xception. Eliminarea dependenței de detectorul intern în cascadă (tip Haar) al pachetului *fer*, prin furnizarea directă către model a regiunii faciale deja extrase de modulul InsightFace. Această abordare a presupus implementarea unui flux de preprocesare personalizat (conversie în tonuri de gri, redimensionare spațială și normalizare în intervalul $[-1, 1]$) și direcționarea intenționată a inferenței către procesorul central (CPU), având ca scop eliberarea resurselor computaționale ale plăcii grafice (GPU) (§4.3.2, §4.3.4).

C15 – Fluxul de preprocesare și procedura de rezervă pentru recunoașterea optică a plăcuțelor de înmatriculare. Implementarea unui lanț de prelucrare a imaginilor optimizat pentru extragerea textului din cadre degradate sau cu rezoluție redusă. Acesta include mărirea rezoluției prin interpolare bicubică (INTER_CUBIC), aplicarea unui filtru bilateral,

egalizarea adaptivă a histogramei (CLAHE) și accentuarea contururilor (*unsharp mask*), cu omiterea intenționată a binarizării. Procesul este completat de un mecanism alternativ (*fallback*), care reevaluează secțiunea pe baza imaginii originale color în situațiile în care scorul de certitudine al detecției inițiale este insuficient (§4.4.2–§4.4.5).

C16 – Stabilizarea recunoașterii plăcuțelor de înmatriculare prin urmărirea spațială și agregare temporală. Implementarea unui modul de urmărire (*tracker*) bazat pe metrica intersecției peste reuniune (IoU), acompaniat de o validare a formatului pe baza unei expresii regulate specifice standardului românesc. Decizia finală este obținută printr-un mecanism de vot majoritar, ponderat cu scorul de certitudine, aplicat pe o fereastră temporală glisantă. Rezultatul este validat și interogată în baza de date exclusiv după atingerea unui consens pe minimum trei cadre, eliminând astfel erorile tranzitorii de recunoaștere (§4.4.6, §4.4.8).

C17 – Flux de filtrare pe cinci niveluri pentru detecția focului și a fumului. Proiectarea și implementarea unui strat de validare suprapus peste modelul YOLO de bază, alcătuit din: praguri de certitudine per clasă, filtre dimensionale, analiză cromatică în spațiul de culoare HSV, confirmare temporală prin urmărire spațială (IoU) de-a lungul unei secvențe de cadre și un mecanism de suprimare temporală a alertelor recurente pe aceeași locație. Această arhitectură introduce o separare semantică clară între stările **confirmed** și **alert**, având ca obiectiv reducerea drastică a detecțiilor fals pozitive (§4.5.2, §4.5.3).

C18 – Compoziția stratificată a cadrelor adnotate și orchestrarea procesului de fuziune. Implementarea unui mecanism de suprapunere ordonată a adnotărilor provenite de la cele cinci module pe o singură imagine rezultantă. Această etapă integrează procesul complet de codificare (compresie JPEG, serializare Base64) și distribuție către clienții conectați, funcționând ca punct central de convergență pentru întregul flux de procesare (§4.8.3, §4.8.5).

4.9.3 Contribuții privind modelele antrenate și seturile de date

C19 – Detector de arme YOLOv8m optimizat suplimentar pe o clasă unică. Antrenarea prin transfer de învățare (*fine-tuning*) a unui model pentru clasa unică **weapon**, pornind de la integrarea a două seturi de date eterogene (Zenodo *Dangerous Items* și Roboflow *SOHAS*). Procesul de pregătire a datelor a implicat unificarea tuturor etichetelor de armament la indexul 0, eliminarea adnotărilor irelevante (păstrând însă imaginile ca exemple negative pentru a spori robustețea modelului) și aplicarea unei convenții de prefixare a fișierelor (**zen_ / rf_**) pentru evitarea coliziunilor de nume. Modelul rezultat atinge o valoare mAP@0,5 de 0,947 (§4.6, §4.6.3, §4.6.8).

C20 – Constituirea unui set de date dedicat pentru recunoașterea actelor de vandalism. Dezvoltarea integrală a unui corpus de date video pentru clasa **vandalism** — categorie insuficient reprezentată în resursele publice actuale — prin proceduri de colectare automată (*web scraping*) de pe platforma YouTube. Procesul a inclus agregarea a aproximativ 4800 de adrese URL pe baza unor interogări multilingve, urmată de o validare manuală riguroasă și segmentarea secvențelor de interes (utilizând editorul OpenShot). Rezultatul constă într-un set de 614 secvențe video validate, care acoperă o tipologie diversificată de incidente (graffiti, spargerea geamurilor, distrugerea autovehiculelor, efracție, vandalizarea mobilierului urban) (§4.7.2.3, §4.7.2.3.2–§4.7.2.3.5).

C21 – Antrenarea prin transfer de învățare a modelului SlowFast-R50 pentru recunoașterea acțiunilor. Adaptarea rețelei SlowFast-R50 (pre-antrenată pe setul de date Kinetics-400) la specificul domeniului de supraveghere video, vizând trei clase țintă (**normal**, **fight**, **vandalism**). Modificarea arhitecturală a constat în înlocuirea stratului final de clasificare (**Linear**(2304 → 3)) și aplicarea unei strategii de antrenare în două etape: blocarea inițială a ponderilor rețelei de bază (*backbone*), urmată de deblocarea acestora și antrenarea cu o rată de învățare diminuată. În urma acestui proces, modelul a atins o acuratețe de 94,33% (§4.7.4, §4.7.5, §4.7.6).

Spre deosebire de elementele menționate anterior, componentele preluate și utilizate în forma lor originală (modulul de recunoaștere facială InsightFace, modelele pentru estimarea demografică, motorul EasyOCR, detectorul YOLOv10 pre-antrenat pentru foc/fum și rețeaua Mini-Xception pentru clasificarea emoțiilor) nu sunt revendicate drept contribuții originale la nivel de arhitectură neuronală. Contribuția personală se reflectă în modul de integrare și optimizare a acestora în cadrul sistemului, precum și în proiectarea algoritmilor de prelucrare, filtrare și decizie care le valorifică funcționalitățile, așa cum a fost prezentat în secțiunile precedente.

Capitolul 5:

Implementarea Aplicației Web

Acest capitol prezintă implementarea componentelor software fundamentale ale sistemului, respectiv arhitectura backend dezvoltată cu ajutorul framework-ului FastAPI și interfața web realizată în React. Sunt analizate exclusiv modulele independente de componenta de inteligență artificială, aceasta fiind detaliată în Capitolul 4. Pentru a evita repetarea aspectelor discutate în etapa de analiză și proiectare (Capitolul 3), accentul este pus pe deciziile de implementare și pe modul concret în care acestea au fost transpuse la nivelul codului sursă.

În continuare sunt prezentate organizarea proiectului software, implementarea mecanismelor de persistență a datelor, autentificare și sincronizare a camerelor video, precum și arhitectura aplicației web și structura principalelor pagini destinate utilizatorilor. Aspectele conceptuale definite anterior — inclusiv structura endpoint-urilor REST (Secțiunea 3.5), protocolul de comunicație WebSocket (Secțiunea 3.6), schema bazei de date (Secțiunea 3.7), modelul de control al accesului (Secțiunea 3.8) și mecanismul de generare a alertelor (Secțiunea 3.9) — nu sunt reluate în detaliu, fiind menționate doar în măsura în care contribuie la înțelegerea implementării practice a sistemului.

5.1 Organizarea codului backend

Arhitectura componentei backend este structurată în jurul unei clase principale, `EnhancedSecuritySystemAPI`, definită în fișierul `app.py`. Aceasta înglobează instanța `FastAPI`, modulele de inteligență artificială, firele de execuție aferente fluxului de procesare (Capitolul 3, Figura 3.6) și starea partajată a aplicației.

O abordare specifică de implementare constă în modul de definire a rutelor API. În loc de a fi separate în module externe de tip `APIRouter`, acestea sunt înregistrate ca funcții asincrone imbricate (*closures*) în cadrul metodei `setup_routes()`. Din motive de claritate, rutele sunt organizate logic pe categorii funcționale: autentificare, fluxuri video, controlul modulelor de analiză, gestiunea entităților (persoane, plăcuțe de înmatriculare, zone, camere, utilizatori), alerte, jurnale de sistem și statistici.

Această abordare permite rutelor să acceseze direct starea internă a instanței curente, inclusiv colecția camerelor active, modulele de analiză AI și mecanis-

mele utilizate pentru distribuirea datelor către clienți. În consecință, este redusă necesitatea introducerii unor mecanisme suplimentare de transmitere a dependențelor către fiecare rută, simplificând astfel implementarea și accesul la resursele partajate ale aplicației.

5.1.1 Injectarea dependențelor și validarea cererilor

Mecanismul de injectare a dependențelor pus la dispoziție de framework-ul FastAPI este utilizat exclusiv pentru gestionarea funcționalităților transversale, evitându-se folosirea acestuia pentru starea internă a aplicației. Astfel, verificarea autentificării și a permisiunilor asociate rolurilor este integrată în mod declarativ, prin intermediul parametrului `Depends`. Mai exact, fiecare rută protejată include `current_user: dict = Depends(get_current_user)`, în timp ce rutele cu acces restricționat destinate administratorilor utilizează `Depends(require_admin)`, aspecte detaliate în Secțiunea 5.2.

În ceea ce privește validarea conținutului cererilor HTTP, aceasta este realizată cu ajutorul unor modele Pydantic dedicate, precum `LoginRequest`, `PersonRegistration`, `LicensePlateRegistration`, `CreateUserRequest` și `UpdateUserRequest`. Aceste structuri garantează tipizarea și verificarea automată a datelor de intrare înainte ca acestea să fie procesate de logica aplicației. În situația în care datele furnizate sunt incorect formate, sistemul interceptează automat cererea și returnează un cod de eroare de tip 422.

5.1.2 Middleware, jurnalizare și interacțiunea cu firele de execuție

Singura componentă de tip *middleware* configurată în mod explicit este `CORSMiddleware`, definită prin metoda `setup_cors()`. Aceasta permite interacțiunea aplicației client din mediul de dezvoltare (`http://localhost:3000`), acceptând credențiale pentru toate metodele și anteturile HTTP. Jurnalizarea evenimentelor nu este realizată printr-un *middleware* dedicat, ci utilizând modulul standard `logging`. Astfel, sunt generate mesaje marcate semantic, vizând evenimente precum autentificările reușite, respingerea jetoanelor de acces, înregistrarea noilor camere video sau declanșarea alertelor. Aceste informații sunt deosebit de utile pentru diagnosticarea erorilor și pentru monitorizarea funcționării sistemului în regim concurent.

Un detaliu esențial de implementare, specific arhitecturii bazate pe fire de execuție multiple (Capitolul 3), constă în modul de gestionare a interacțiunii dintre bucla de evenimente asincronă a framework-ului FastAPI și operațiile de intrare/ieșire blocante. Funcțiile care execută operații blocante, precum inițializarea unei surse video prin `cv2.VideoCapture` (proces care poate dura câteva secunde în cazul unei camere IP), nu sunt apelate direct pe bucla principală de evenimente. Pentru a preveni blocarea aplicației, aceste operații sunt delegate unui fir de execuție secundar prin intermediul metodei `asyncio.to_thread(...)`. În absența acestei separări, o singură inițializare lentă ar întârzia sau ar bloca complet procesarea celorlalte cereri HTTP și a traficului de rețea. Transmiterea

cadrelor procesate către utilizatori respectă același principiu de decuplare: fi-rele responsabile cu analiza video depun mesajele rezultate în cozi de așteptare dedicate fiecărui client (`client_queues`), de unde sunt preluate în mod asin-cron de funcția de gestionare a conexiunilor `WebSocket`, conform descrierii din Secțiunea 4.8, subcapitolul 4.8.5.

5.2 Concretizarea proiectării: persistență, auten-tificare, sincronizarea camerelor

Această secțiune detaliază modul în care arhitectura propusă în Capitolul 3 este transpusă la nivelul codului, concentrându-se exclusiv pe aspectele practice de implementare și evitând reluarea argumentelor teoretice deja discutate.

5.2.1 Stratul de persistență

Mecanismul de persistență a fost implementat fără a utiliza un sistem de ma-pare obiect-relațională (ORM) sau un instrument dedicat pentru migrarea bazei de date. În schimb, s-a optat pentru un nivel simplificat de acces la date, de-finit prin clasa `DatabaseManager` din fișierul `database_manager.py`. Aceasta interacționează direct cu driverul `psycpg2`, utilizând componenta `RealDict Cursor` pentru a extrage înregistrările sub formă de dicționare. Deoarece o conexiune `psycpg2` nu suportă în mod nativ accesul concurent (*thread-safe*), iar aplicația utilizează multiple fire de execuție, conexiunea și cursorul aferent sunt izolate la nivelul fiecărui fir în parte, prin intermediul unui obiect de tip `threading.local` (`self._local`). Astfel, fiecare fir de execuție își inițializează propria conexiune la prima interogare, prevenind coliziunile între firele dedicate analizei video și cele responsabile cu gestionarea cererilor HTTP.

Structura bazei de date, a cărei proiectare a fost detaliată în Secțiunea 3.7, este inițializată automat la pornirea aplicației. Procesul utilizează instrucțiuni SQL idempotente de tipul `CREATE TABLE IF NOT EXISTS`. Această metodă garantează executarea lor sigură la fiecare inițializare și reprezintă o soluție eficientă care suplinește lipsa unui sistem complex de versionare a schemei relaționale.

În ceea ce privește stocarea datelor biometrice, vectorii de caracteristici fa-ciale sunt salvați în coloane de tip `BYTEA`, fiind transformați în format binar (serializați) cu ajutorul modulului `pickle`, procedură explicată în Capitolul 4. Pentru a asigura integritatea bazei de date, operațiile concurente de scriere sunt sincronizate, acolo unde este necesar, prin mecanisme de excludere mutuală (*locks*). Totodată, conflictele apărute la inserarea unor date duplicate sunt ges-tionate în mod explicit prin capturarea excepției `psycpg2.IntegrityError`.

5.2.2 Implementarea autentificării și a controlului de rol

Modelul de control al accesului bazat pe roluri (*Role-Based Access Control* – RBAC) a fost fundamentat teoretic în Secțiunea 3.8. Această subsecțiune

descrie transpunerea sa la nivelul codului sursă. Pentru a garanta securitatea datelor, parolele nu sunt stocate niciodată în format text (*in cleartext*). Metoda `hash_password()` generează un rezumat criptografic utilizând algoritmul `bcrypt`, apelând `bcrypt.hashpw` împreună cu o componentă aleatoare generată prin `gensalt()`. Verificarea credențialelor la momentul autentificării se realizează exclusiv prin funcția `bcrypt.checkpw` în cadrul metodei `verify_password()`, evitându-se complet orice formă de decriptare a valorii stocate.

Gestiunea sesiunilor utilizatorilor este realizată prin intermediul token-urilor JWT (*JSON Web Tokens*), configurate cu algoritmul HS256 și o perioadă de valabilitate de 8 ore. Cheia criptografică necesară semnării este preluată exclusiv din variabila de mediu `JWT_SECRET`, fiind impusă o lungime minimă de 32 de caractere. Pentru a preveni vulnerabilitățile, în regim de producție inițializarea aplicației este oprită dacă această variabilă lipsește, evitându-se astfel emiterea token-urilor cu o cheie temporară. În plus față de revendicările standard (*claims*), fiecare token integrează un identificator unic (`jti`), facilitate care permite revocarea sa instantanee în caz de necesitate.

Mecanismul de verificare a permisiunilor este implementat printr-o ierarhie de dependențe specifice framework-ului FastAPI:

- Funcția `get_current_user()` decodifică token-ul și îl validează riguros la fiecare cerere HTTP, interogând direct baza de date. Astfel, sunt respinse token-urile aflate pe lista de revocare, verificate prin intermediul funcției `is_token_revoked(jti)` pe structura `revoked_tokens`, precum și cele care aparțin unor utilizatori eliminați din sistem. Mai mult, nivelul de acces este extras din baza de date, ignorându-se valoarea stocată inițial în token. Această abordare garantează că modificările de rol sau ștergerea unui cont produc efecte imediat, la următoarea cerere, fără a aștepta expirarea naturală a sesiunii.
- Funcția `require_admin()` reprezintă o dependență suplimentară, construită peste cea anterioară. Aceasta returnează codul de eroare HTTP 403 dacă rolul utilizatorului nu este `admin`. Aceeași regulă de autorizare guvernează atât accesul la nivelul API-ului, cât și afișarea condiționată a elementelor din interfața grafică, aspect detaliat în Secțiunea 5.3.

Deoarece browserele web nu permit configurarea antetului `Authorization` pentru conexiunile de tip WebSocket, autentificarea pe canalul de transmisie continuă (*streaming*) este gestionată diferit comparativ cu rutele REST. Token-ul de acces este transmis sub forma unui parametru de interogare în adresa URL (`/ws?token=...`). Funcția responsabilă cu gestionarea conexiunii validează acest token, verificând inclusiv apartenența la lista de revocare și starea contului de utilizator. În cazul în care validarea eșuează, conexiunea este respinsă înainte de a fi inițializată, returnându-se codul de eroare 1008 (*policy violation*).

5.2.3 Sincronizarea camerei dinamice cu registrul de camere

Sistemul face distincția între camera video locală (atașată direct echipamentului și rezervată sub identificadorul CAM-01, fiind înregistrată automat la inițializare) și camerele IP adăugate dinamic prin intermediul interfeței utilizator. Acestea din urmă sunt integrate apelând ruta `/api/streams`, utilizând o structură de date de forma `source: "ip", url, camera_id, location`.

Mecanismul de implementare gestionează în mod explicit scenariile de eroare, având în vedere posibilitatea ca o sursă IP să fie indisponibilă sau ca adresa de rețea specificată să fie incorectă. Inițializarea fluxului video se realizează în afara buclei principale de evenimente, apelând metoda `asyncio.to_thread(cv2.VideoCapture, url)`, în timp ce starea conexiunii este validată prin funcția `isOpenned()`. În situația în care deschiderea fluxului eșuează, resursele alocate sunt eliberate imediat, iar un mesaj de eroare este transmis aplicației client, prevenind astfel blocarea resurselor sistemului.

În cazul unei conexiuni reușite, dispozitivul este înregistrat simultan în două locații complementare. Prima locație este memoria volatilă a aplicației, reprezentată de dicționarul `self.cameras`, care este accesat direct de firele de execuție pentru procesarea imaginilor. A doua locație este baza de date, unde informația este salvată în mod persistent prin intermediul funcției `upsert_camera()`. Această dublă înregistrare a fost implementată intenționat. Salvarea persistentă garantează că asocierea dintre o cameră și o zonă monitorizată, element central pentru logica de generare a alertelor detaliată în Secțiunea 3.9, este păstrată inclusiv după repornirea sistemului, eliminând astfel necesitatea unei noi configurări manuale.

Pentru a optimiza performanța și a evita interogarea bazei de date la procesarea fiecărui cadru video, relația dintre dispozitiv și zona supravegheată este stocată într-o memorie temporară locală (`_camera_zone_cache`). Suplimentar, dacă un identificador `camera_id` există deja în sistem, fluxul video anterior este închis și eliberat în mod corespunzător înainte de a fi înlocuit, prevenindu-se astfel acumularea conexiunilor deschise în fundal.

5.3 Arhitectura frontend-ului React

Interfața grafică este dezvoltată sub forma unei aplicații React 19, inițializată prin intermediul utilitarului *Create React App*. Pentru stilizarea vizuală a fost utilizat framework-ul Tailwind CSS, în timp ce elementele grafice (pictogramele) sunt preluate din biblioteca `lucide-react`. La nivel arhitectural, aplicația nu depinde de o bibliotecă externă pentru gestionarea rutelor de navigare (*routing*). În schimb, componenta principală `App.js` menține o stare internă denumită `activeTab`, iar trecerea de la o secțiune la alta se realizează prin randare condiționată. Navigarea propriu-zisă este delegată componentei `Sidebar`, căreia îi este transferată funcția `onNavigate=setActiveTab`. În paralel, componenta `Header` nu influențează secțiunea activă, având exclusiv rolul de a extinde me-

niul lateral, de a afișa starea conexiunii și de a oferi opțiunea de deconectare. Această abordare este optimă pentru o interfață de tip panou de control (*dashboard*), caracterizată printr-un număr prestabilit și fix de secțiuni.

Gestiunea sesiunii utilizatorului se bazează pe stocarea locală a datelor în navigatorul web (`localStorage`), utilizând cheile `auth_token` și `auth_user`. Această metodă garantează menținerea stării de autentificare chiar și în eventualitatea reîncărcării paginii. Drepturile de acces în interfață sunt gestionate prin intermediul variabilei derivate `isAdmin`. Aceasta condiționează afișarea secțiunilor dedicate administratorilor, reprezentând o restricție de natură vizuală care dublează și completează verificarea de securitate `require_admin` implementată la nivelul serverului, conform descrierii din Secțiunea 5.2. Comunicația cu interfața de programare (API) este centralizată cu ajutorul constantei `API_BASE` și al funcției utilitare `getAuthHeaders()`, care are rolul de a include automat antetul de autorizare (`Authorization: Bearer`) în fiecare cerere HTTP de tip `fetch`.

Inițializarea conexiunii continue prin protocolul WebSocket se realizează în cadrul unei funcții `useEffect`, token-ul de autentificare fiind transmis sub forma unui parametru de interogare. În situația unei întreruperi, aplicația client inițiază automat o tentativă de reconectare după un interval de trei secunde. Face excepție scenariul în care serverul a terminat conexiunea returnând codul de eroare 1008 (token respins), caz în care reconectarea este anulată, fiind considerată inutilă. Un element arhitectural definitoriu este *rolul simplificat al aplicației client în redarea video*. Cadrele recepționate prin rețea sunt deja procesate vizual la nivelul serverului, adnotările (precum casetele de delimitare și etichetele descriptive) fiind aplicate direct pe imagine în etapa de fuziune a datelor (Secțiunea 4.8). Prin urmare, interfața web nu efectuează operații costisitoare de decodare a imaginii sau de desenare grafică. Aceasta se limitează la a atribui șirul de caractere codificat în format Base64 atributului `src` al unui element HTML de tip `<img>` (sub forma `data:image/jpeg;base64,...`). Metadatele asociate detecțiilor, care însoțesc fiecare cadru, sunt valorificate exclusiv pentru actualizarea componentelor adiacente din interfață (panourile destinate alertelor și activității recente), nefiind implicate în randarea propriu-zisă a fluxului video.

5.4 Paginile aplicației

Interfața grafică a aplicației este structurată în trei categorii de pagini: secțiunea de monitorizare în timp real, secțiunea destinată administrării entităților (care integrează operațiile standard de gestionare a datelor, de tip CRUD) și secțiunea operațională, dedicată alertelor, jurnalelor de sistem și statisticilor. Deoarece arhitectura generală de navigare a fost deja abordată în Secțiunea 3.10, subcapitolele următoare se concentrează exclusiv pe detalierea modului în care a fost implementată la nivel de cod fiecare dintre aceste categorii.

5.4.1 Monitorizare live și controlul modulelor (Dashboard)

Panoul principal de control, implementat prin intermediul componentei **CameraGrid**, prezintă o rețea de panouri destinate afișării fluxurilor video. Sursa video locală, identificată prin **CAM-01**, este disponibilă în mod implicit. În paralel, camerele IP sunt adăugate dinamic în interfață pe măsură ce primele cadre sunt recepționate prin protocolul WebSocket. Mai exact, funcția responsabilă cu procesarea mesajelor generează un nou panou de redare atunci când detectează un identificator **camera_id** nou. Fiecare secțiune redă fluxul video sub forma unui element HTML de tip `<img>`, utilizând imagini codificate în format Base64. Prin selectarea unui anumit panou, utilizatorul poate deschide o fereastră modală care mărește imaginea respectivă, în timp ce un indicator vizual semnalează imediat dispozitivele care au înregistrat alerte active.

Activarea și dezactivarea celor șase module de inteligență artificială sunt gestionate prin componenta **IntelligenceSettings**. Aceasta pune la dispoziție o serie de elemente de control pentru fiecare funcționalitate în parte (recunoașterea facială, citirea plăcuțelor de înmatriculare, analiza demografică, detectarea incendiilor și a fumului, recunoașterea acțiunilor umane și identificarea armelor). Fiecare acțiune de comutare inițiază o cerere către punctul terminal (*endpoint*) corespunzător, cum ar fi `/api/face/toggle`. Această modificare de stare se aplică *în timp real, fără a necesita repornirea sistemului*. Firele de execuție dedicate analizei video verifică, pentru fiecare cadru procesat, variabilele de stare partajate pe care aceste cereri le actualizează. Prin urmare, un modul dezactivat este pur și simplu exclus din fluxul curent de procesare, mecanism aliniat cu principiile de dezactivare selectivă argumentate în Capitolul 3.

5.4.2 Administrarea entităților (operații CRUD)

Paginile destinate administrării entităților (persoane, zone, camere video, plăcuțe de înmatriculare și utilizatori) sunt dezvoltate pe baza unei arhitecturi și a unui model de implementare unitar. Fiecare interfață integrează o listă prevăzută cu filtre de căutare, un formular pentru crearea sau modificarea înregistrărilor, precum și opțiuni pentru ștergerea acestora. Toate aceste componente comunică cu rutele REST, detaliate în Secțiunea 3.5, prin intermediul unor apeluri de rețea autentificate de tip **fetch**. Pentru a evita o prezentare repetitivă a acestor elemente comune, în continuare sunt evidențiate exclusiv particularitățile funcționale ale fiecărei pagini:

- **Persoane (PersonManagement)**: formularul de înregistrare preia date referitoare la nume, identificatorul de angajat, departament și zonele de acces permise. O funcționalitate specifică este posibilitatea de încărcare a mai multor imagini pentru același individ. Acestea sunt transmise către punctul terminal `/api/persons//faces` sub forma unui câmp multiplu de tip **files**. Vectorii de caracteristici faciale sunt generați la nivelul serverului, conform mecanismului descris în Capitolul 4. În plus, interfața

permite vizualizarea unui istoric detaliat al accesărilor pentru fiecare persoană înregistrată.

- **Zone (ZoneManagement):** secțiunea permite definirea numelui zonei și marcarea acesteia ca fiind *restricționată* prin intermediul atributului `is_restricted`. Este important de precizat că drepturile de acces ale angajaților nu sunt alocate din această pagină, ci din interfața dedicată gestionării persoanelor, unde se configurează lista zonelor autorizate pentru fiecare individ. Aceste atribute sunt ulterior valorificate de logica de generare a alertelor.
- **Camere (CameraManagement):** facilitează înregistrarea noilor dispozitive video, asocierea acestora cu o zonă specifică și configurarea parametrilor fluxului de date (nume, adresă URL). Sistemul garantează păstrarea acestor setări în mod persistent, inclusiv după repornirea aplicației, aspect detaliat în Secțiunea 5.2.3.
- **Plăcuțe de înmatriculare (PlateManagement):** asigură introducerea datelor vehiculelor, incluzând numărul de înmatriculare, identitatea proprietarului, tipul vehiculului și starea de autorizare.
- **Utilizatori (UserManagement):** această pagină pune la dispoziție instrumentele necesare pentru crearea noilor conturi (atribuirea rolurilor `admin` sau `user`), modificarea nivelurilor de acces, reinițializarea parolilor și eliminarea conturilor existente.

5.4.3 Paginile operaționale: alarme, jurnale și statistici

A treia categorie de pagini din cadrul interfeței web este proiectată pentru a sprijini direct activitatea operatorului de supraveghere. Componenta **AlarmManagement** afișează un registru al alertelor generate, permițând filtrarea acestora în funcție de starea curentă, tipul evenimentului și nivelul de severitate. Interfața oferă posibilitatea de a vizualiza imaginea captată (*snapshot*) exact în momentul detecției. Totodată, operatorul poate marca o alertă ca fiind *rezolvată* (`resolved`) sau ca o *alarmă falsă* (`false_alarm`), având și opțiunea de a adăuga observații operaționale specifice. Sistemul suportă actualizarea simultană a mai multor alerte selectate.

Componenta **Logs** oferă acces la istoricul complet al detecțiilor înregistrate de sistem. Aceasta integrează funcționalități de căutare și filtrare pe baza unor criterii precum tipul detecției, camera sursă, starea alarmei, intervalul temporal sau expresii introduse prin căutare liberă. De asemenea, rezultatele obținute în urma filtrării pot fi exportate în format CSV, facilitând prelucrarea și analiza ulterioară a datelor.

Componenta **Statistics** furnizează o sinteză a informațiilor colectate pentru intervale de timp predefinite (ultimele 24 de ore, ultimele 7 zile și ultimele 30 de zile). Interfața prezintă indicatori statistici relevanți, precum numărul total de detecții, distribuția acestora pe categorii, camere video și niveluri de

severitate, precum și evoluția temporală a evenimentelor. În plus, sunt calculați indicatori de performanță asociați sistemului de alertare, incluzând numărul total de alerte generate, numărul alertelor nerezolvate și incidența evenimentelor critice.

O imagine de ansamblu asupra celor mai recente evenimente este disponibilă în permanență prin intermediul componentei **RecentActivity**, care afișează în timp real activitatea recentă înregistrată de sistem.

Capitolul 6: Testarea și Evaluarea Sistemului

- 6.1 Metodologia de testare
- 6.2 Configurația hardware folosită
- 6.3 Evaluarea pipeline-ului integrat
 - 6.3.1 FPS la procesare concurentă
 - 6.3.2 Utilizarea memoriei GPU și RAM
 - 6.3.3 Latența end-to-end
- 6.4 Scenarii de test funcțional
- 6.5 Discuție asupra false pozitelor și false negative
- 6.6 Limitări observate

Capitolul 7: Concluzii și Direcții de Dezvoltare

- 7.1 Sinteza rezultatelor
- 7.2 Obiective atinse vs. obiective propuse
- 7.3 Contribuții originale
- 7.4 Limitări ale soluției actuale
- 7.5 Direcții de dezvoltare viitoare

Bibliografie

Cărți

Articole și lucrări științifice

- [3] G. Sreenu și M. A. Saleem Durai, “Intelligent video surveillance: a review through deep learning techniques for crowd analysis”, în *Journal of Big Data* 6.48 (2019), pp. 1–27, DOI: 10.1186/s40537-019-0212-5.
- [4] Noah Sulman et al., “How effective is human video surveillance performance?”, în *2008 19th International Conference on Pattern Recognition (ICPR)*, IEEE, 2008, pp. 1–3, DOI: 10.1109/ICPR.2008.4761655.
- [5] Erika Näsholm, Sarah Rohlfing și James D. Sauer, “Pirate Stealth or Inattentional Blindness? The Effects of Target Relevance and Sustained Attention on Security Monitoring for Experienced and Naïve Operators”, în *PLoS ONE* 9.1 (2014), e86157, DOI: 10.1371/journal.pone.0086157.
- [6] Jenna Burrell, “How the Machine ‘Thinks’: Understanding Opacity in Machine Learning Algorithms”, în *Big Data & Society* 3.1 (2016), pp. 1–12, DOI: 10.1177/2053951715622512.
- [8] Ahmed Shafee, Syed Rafay Hasan și Tasneem A. Awaad, “Privacy and Security Vulnerabilities in Edge Intelligence: An Analysis and Countermeasures”, în *Computers & Electrical Engineering* 123 (2025), p. 110146, DOI: 10.1016/j.compeleceng.2025.110146.
- [9] S. Q. Y. Al-Hashemi, M. S. Naghmash și A. Ghandour, “Improving IP Video Surveillance Systems: The Shift to Digital Networks and Security Challenges”, în *SHIFRA 2024* (2024), pp. 17–23, DOI: 10.70470/SHIFRA/2024/003.
- [10] Roman Odarchenko et al., “Optimization of the integrated video surveillance system with elements of data analysis”, în *CEUR Workshop Proceedings (Third International Conference on Cyber Hygiene & Conflict Management in Global Information Networks)*, vol. 3925, 2025, pp. 47–63, URL: <https://ceur-ws.org/Vol-3925/>.

- [11] David Abrams, “Video Content Analysis with Effective Response”, in *Proceedings of the 43rd Annual IEEE International Carnahan Conference on Security Technology (ICCST)*, IEEE, 2009.
- [12] Lokesh Jain și Nitin Saraswat, “Revolutionizing Modern Surveillance Systems: Harnessing the Power of Neural Networks for Advanced Video Analytics, Real-Time Object Detection, and Predictive Security Applications”, in *AI and Deep Learning Enabled Surveillance System Using Image Processing*, ed. de Jay Kumar Pandey, Mritunjay Rai și Faizan Ahmad, Emerald Publishing Limited, 2026, DOI: 10.1108/978-1-80592-815-7.
- [21] Carlos Julio Fierro Silva et al., “Comparative analysis of previous YOLO detectors and YOLOv26s for real-time weapon detection in video surveillance”, in *Frontiers in Computer Science* 8 (2026), DOI: 10.3389/fcomp.2026.1789702, URL: <https://doi.org/10.3389/fcomp.2026.1789702>.
- [23] Tareq Khan, “Audiovisual Gun Detection with Automated Lockdown and PA Announcing IoT System for Schools”, in *IoT* 7.1 (2026), p. 15, ISSN: 2624-831X, DOI: 10.3390/iot7010015, URL: <https://doi.org/10.3390/iot7010015>.
- [24] Jiankang Deng et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition”, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.
- [25] Face Recognition Technologies: A Comprehensive Review, “Face Recognition Technologies: A Comprehensive Review”, in *International Journal of Sciences: Basic and Applied Research (IJSBAR)* (2025), Analiză a 127 de publicații academice (2015–2024).
- [26] Comparative Evaluation of ArcFace and FaceNet, “Comparative Evaluation of ArcFace and FaceNet Models for Real-World Facial Recognition Using the Analytic Hierarchy Process”, in *Lecture Notes in Networks and Systems*, Springer Nature, 2026, DOI: 10.1007/978-3-032-19913-3_6.
- [27] Muhammad Hussain și Rahima Khanam, “YOLOv1 to YOLOv10: A Comprehensive Review of YOLO Variants and Their Application in the Agricultural Domain”, in *arXiv preprint arXiv:2406.10139* (2024).
- [28] Ranjan Sapkota, Rizwan Qureshi et al., “YOLOv10 to Its Genesis: A Decadal and Comprehensive Review of the You Only Look Once Series”, in *arXiv preprint arXiv:2406.19407* (2024).
- [29] Christoph Feichtenhofer et al., “SlowFast Networks for Video Recognition”, in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6202–6211.
- [30] Ming Cheng, Kunjing Cai și Ming Li, “RWF-2000: An Open Large Scale Video Database for Violence Detection”, in *2020 25th International Conference on Pattern Recognition (ICPR)*, 2021, pp. 4183–4190, DOI: 10.1109/ICPR48806.2021.9412502.

- [31] Literature Review of Deep-Learning-Based Detection of Violence in Video, “Literature Review of Deep-Learning-Based Detection of Violence in Video”, în *Sensors (MDPI)* (2024), Articol cu acces deschis, PMC11207446.
- [32] Aml Sarhan, R. Abdel-Rahem, B. Darwish et al., “Egyptian Car Plate Recognition Based on YOLOv8, Easy-OCR, and CNN”, în *Journal of Electrical Systems and Information Technology* 11.1 (2024), p. 32, DOI: 10.1186/s43067-024-00156-y.
- [33] A Deep Learning-Based System for Automatic License Plate Recognition, “A Deep Learning-Based System for Automatic License Plate Recognition Using YOLOv12 and PaddleOCR”, în *Applied Sciences (MDPI)* 15.14 (2025), p. 7833, DOI: 10.3390/app15147833.

Resurse web, rapoarte și legislație

- [1] Paul Bischoff, *Surveillance Camera Statistics: Which City has the Most CCTV Cameras?*, Comparitech, 2025, URL: <https://www.comparitech.com/vpn-privacy/the-worlds-most-surveilled-cities/> (accesat în 21.06.2026).
- [2] Parlamentul României, *Legea nr. 190/2018 privind măsuri de punere în aplicare a Regulamentului (UE) 2016/679 (GDPR)*, Monitorul Oficial al României, Partea I, nr. 651, 2018, URL: https://www.dataprotection.ro/index.jsp?page=Legea_nr_190_2018 (accesat în 19.06.2026).
- [7] Manuel Hoffmann, Frank Nagle și Yanuo Zhou, *The Value of Open Source Software*, Strategy Unit Working Paper 24-038, Harvard Business School, 2024, DOI: 10.2139/ssrn.4693148.
- [13] Verkada Inc., *Hybrid Cloud Video Security Overview*, Accesat: Iunie 2026, 2022, URL: https://www.mltnetworks.co.uk/wp-content/uploads/2022/07/Verkada_Video_Security_Overview.pdf.
- [14] Verkada Inc., *How Secure is your Video Surveillance System? Cybersecurity Guide*, Accesat: Iunie 2026, 2021, URL: <https://docs.verkada.com/docs/video-security-cybersecurity-guide.pdf>.
- [15] Motorola Solutions, *Avigilon Appearance Search Technology*, Accesat: Iunie 2026, 2021, URL: https://www.motorolasolutions.com/content/dam/msi/docs/Appearence_Search_AV0.pdf.
- [16] Avigilon, *Appearance Search - Avigilon Documentation*, Accesat: Iunie 2026, 2023, URL: <https://docs.avigilon.com/bundle/unity-video-client-8-1/page/using/appearance-search.htm>.
- [17] BriefCam, *BriefCam Platform - Datasheet*, Accesat: Iunie 2026, 2025, URL: <https://www.briefcam.com/wp-content/uploads/2025/06/Briefcam-datasheet.pdf>.

- [18] BCDVideo / BriefCam, *BriefCam Appliance Catalog*, Accesat: Iunie 2026, 2018, URL: <https://www.bcdvideo.com/wp-content/uploads/2018/10/Briefcam-Catalog.pdf>.
- [19] Frigate NVR, *Frigate: Open Source AI Object Detection for IP Cameras*, Accesat: Iunie 2026, 2026, URL: <https://frigate.video/>.
- [20] ZoneMinder, *ZoneMinder User and API Documentation*, Accesat: Iunie 2026, 2020, URL: <https://zoneminder.com/>.
- [22] Gajendra Mandal, J. P. Patra și Priyansh Mahant, *Real-Time Threat Detection from Surveillance Cameras using Machine Learning*, Preprint, 2026, DOI: 10.48550/arXiv.2606.05708, arXiv: 2606.05708 [cs.CV], URL: <https://arxiv.org/abs/2606.05708>.